Introduction to Statistics
18.05 Spring 2022

T T T H H T H H H T
H T H T H T H T H T
H T T T H T T T T H
H T T H H T H H T H
T T H H H H T H T H
T T T H T H H H H T
T T T H H H T T T H
H H H H H H H T T T
H T H H T T T H H T
H T H H H T T T H H

Announcements/Agenda

Announcements

- Changed due date on Pset 6. Now due on Wed. 3/16. (Only one problem.)

Agenda

- Start statistics
- Define statistic
- Define likelihood
- Maximum likelihood estimates (MLE)

Statistics

Statistics is about the analysis of data. In practice, it is an art. It uses the mathematics or probability, but we will see that the conclusions we are able to draw are not rigorous in a mathematical sense.

- Data Collection:
  Informal Investigation / Observational Study / Formal Experiment
- Descriptive statistics : e.g. mean, median, variance of the data, histograms, box plots etc.
- Inferential statistics (the focus in 18.05): Draw conclusions or make decisions based on data.

To consult a statistician after an experiment is finished is often merely to ask him to conduct a post-mortem examination. He can perhaps say what the experiment died of.

R.A. Fisher

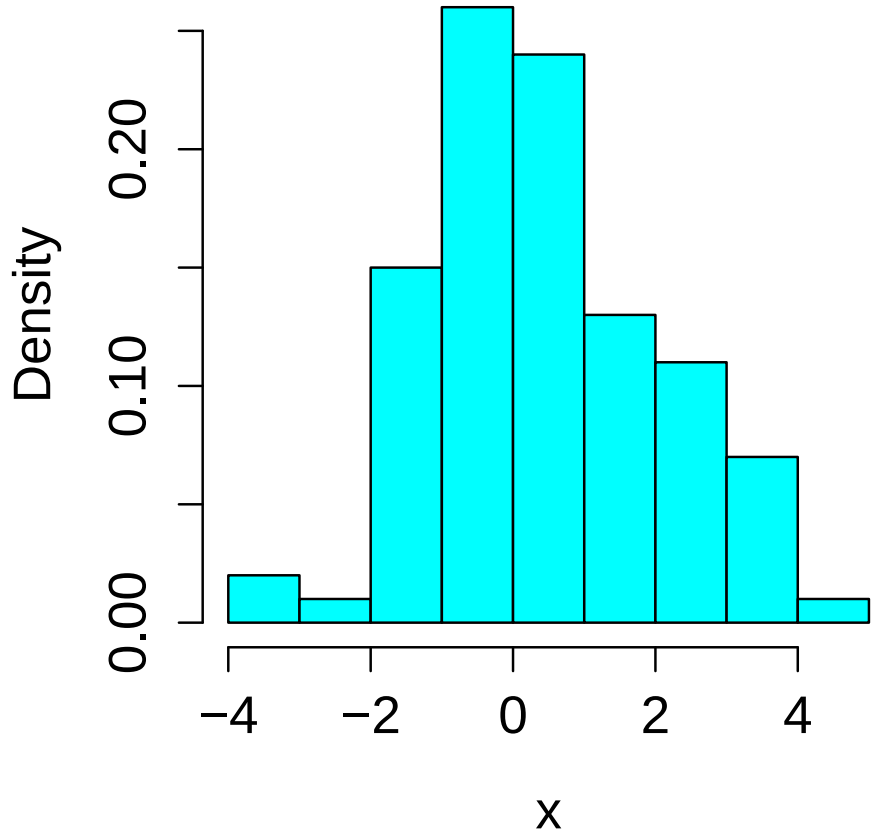
Question about data: Is it fair?

100 tosses of a coin. Is the coin fair?

T	T	T	H	H	T	H	H	H	T
H	T	H	T	H	T	H	T	H	T
H	T	T	T	H	T	T	T	T	H
H	T	T	H	H	T	H	H	T	H
T	T	H	H	H	H	T	H	T	H
T	T	T	H	T	H	H	H	H	T
T	T	T	H	H	H	T	T	T	H
H	H	H	H	H	H	H	T	T	T
H	T	H	H	T	T	T	H	H	T
H	T	H	H	H	T	T	T	H	H

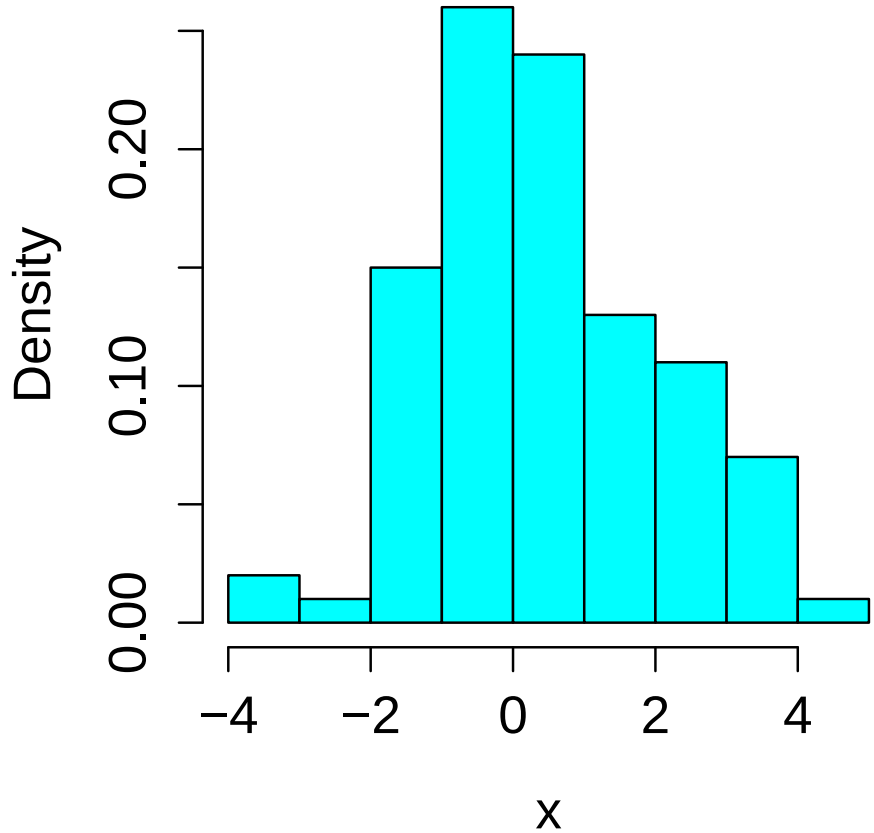
Question about data: Is it normal?

Does it have $\mu = 0$? Is it normal? Is it standard normal?



Question about data: Is it normal?

Does it have $\mu = 0$? Is it normal? Is it standard normal?



Sample mean $= 0.38$; sample standard deviation $= 1.59$

What is a statistic?

Definition. A statistic is anything that can be computed from the collected data. That is, a statistic must be observable.

- Point statistic: a single value computed from data, e.g sample average $\overline{x}_n$ or sample standard deviation s_n.
- Interval or range statistics: an interval $[a, b]$ computed from the data. (Just a pair of point statistics.) Often written as $\overline{x} \pm s$.
- **Important:** A statistic is itself a random variable since a new experiment will produce new data to compute it.

Concept question: Is it a statistic?

You believe that the lifetimes of a certain type of lightbulb follow an exponential distribution with parameter λ. To test this hypothesis you measure the lifetime of 5 bulbs and get data $x_1, \ldots x_5$.

Which of the following are statistics?

(a) The sample average $\overline{x} = \frac{x_1+x_2+x_3+x_4+x_5}{5}$.

(b) The expected value of a sample, namely $1/\lambda$.

(c) The difference between $\overline{x}$ and $1/\lambda$.

1. (a)
2. (b)
3. (c)
4. (a) and (b)
5. (a) and (c)
6. (b) and (c)
7. all three
8. none of them

Notation

Big letters X, Y, X_i are random variables.

Little letters x, y, x_i are data (values) generated by the random variables.

Example. Experiment: 10 flips of a coin:

X_i is the random variable for the ith flip: either 0 or 1.

x_i is the actual result (data) from the ith flip.
e.g. $x_1, \ldots, x_{10} = 1, 1, 1, 0, 0, 0, 0, 0, 1, 0$.

Reminder of Bayes' theorem

Bayes's theorem is the key to our view of statistics.

(Much more next week!)

$$P(\mathcal{H}|\mathcal{D}) = \frac{P(\mathcal{D}|\mathcal{H})P(\mathcal{H})}{P(\mathcal{D})}.$$

$$P(\text{hypothesis}|\text{data}) = \frac{P(\text{data}|\text{hypothesis})P(\text{hypothesis})}{P(\text{data})}$$

Standard statistics question: Estimating a parameter

Example. Suppose we want to know the percentage p of people for whom cilantro tastes like soap.

Parameter of interest: p

Experiment: Ask n random people to taste cilantro.

Model: X_i is the response of the ith person (1 if yes, 0 if no). $X_i \sim \text{Bernoulli}(p)$.

Data: $x_1, \ldots, x_n$ are the results of the experiment

Inference: Estimate p from the data.

Hypotheses: We can hypothesize that p has a specific value or a range of values.

Parameters of interest

Example. You ask 100 people to taste cilantro and 55 say it tastes like soap. Use this data to estimate p the fraction of all people for whom it tastes like soap.

So, p is the parameter of interest.

Don't know the value of p – can only hypothesize values.

Likelihood

You ask 100 people and 55 say cilantro tastes like soap. For a given p, the probability of 55 'successes' is the binomial probability

$$P(55 \text{ soap}|p) = \binom{100}{55} p^{55}(1-p)^{45}.$$

This the likelihood of p given the data. Often denoted $L(p)$.

In general,

$$\text{The likelihood of } p \ = L(p) = P(\text{data}|p)$$

NOTICE: The likelihood takes the data as fixed and computes the probability of the data for a given hypothetical value of$p$.

Maximum likelihood estimate (MLE)

The maximum likelihood estimate (MLE) is a way to estimate the value of a parameter of interest from the data.

The MLE is the value of p that maximizes the likelihood.

Different problems call for different methods of finding the maximum.

Here are a few –there are others:

1. Calculus: To find the MLE, solve $\frac{d}{dp}P(\text{data} \mid p) = 0$ for p. (We should also check that the critical point is a maximum.)

2. Sometimes the derivative is never 0 and the MLE is at an endpoint of the allowable range.

3. If the parameter takes only a finite number of values, you can compute the likelihood for each one and see which is the biggest.

Cilantro tasting MLE: Method 1

The MLE for the cilantro tasting experiment can be found by setting the derivative to 0:

$$\frac{dL(p)}{dp} = 0.$$

The calculation is not hard.

In this case, log likelihoods are easier, so we do that calculation instead.

Cilantro tasting MLE: Method 2: log likelihood

Because the log function turns multiplication into addition it is often convenient to use the log of the likelihood function

$$\text{log likelihood} = l(p) = \ln(\text{likelihood}) = \ln(P(\text{data} \mid p)).$$

Example.

$$\text{Likelihood } L(p) = P(\text{data}|p) = \binom{100}{55} p^{55}(1-p)^{45}$$

$$\text{Log likelihood } l(p) = \ln\left(\binom{100}{55}\right) + 55\ln(p) + 45\ln(1-p).$$

(Note first term is just a constant.)

The computation is posted with today's problems. The answer is

$$\hat{p} = 0.55$$

Adding a hat, is a standard way of indicating it is an estimate of the true value of p.

Board Question: Coins

(a) A box contains 3 coins. They land heads with, respectively, probability $p = 1/3, 1/2, 2/3$.

A coin is taken from the box. The mystery coin is tossed 80 times, resulting in 49 heads and 31 tails.

What is the likelihood of this data for each type of coin? Which coin gives the maximum likelihood?

(b) Now suppose you found a bent coin. It has an unknown probability p of landing heads. To estimate p you toss it 80 times getting 49 heads. Find the likelihood and log likelihood functions given this data. What is the maximum likelihood estimate for p?

Board question: continuous likelihood

For continuous likelihood: use the pdf instead of the pmf

Box of light bulbs.

Lifetime of each bulb $\sim \exp(\lambda)$, with unknown parameter λ.

For multiple independent data points, the likelihood is the product of the individual likelihoods.

(a) We test 5 light bulbs and find they have lifetimes of 2, 3, 1, 3, 4 years respectively. We assume the tests are independent.

(i) Find the likelihood and log likelihood functions (as functions of λ.)

(ii) What is the maximum likelihood estimate (MLE) for λ?

Reminder: An exponential distribution has pdf $f(x|\lambda) = \lambda \mathrm{e}^{-\lambda x}$

(b) Suppose we test 5 bulbs and find they have lifetimes x_1, x_2, x_3, x_4, x_5 years respectively. Redo Part (a) using these lifetimes.

MIT OpenCourseWare
https://ocw.mit.edu

18.05 Introduction to Probability and Statistics
Spring 2022

For information about citing these materials or our Terms of Use, visit:
https://ocw.mit.edu/terms.

Conditional Probability, Independence and Bayes' Theorem
Class 3, 18.05
Jeremy Orloff and Jonathan Bloom

1 Learning Goals

1. Know the definitions of conditional probability and independence of events.
2. Be able to compute conditional probability directly from the definition.
3. Be able to use the multiplication rule to compute the total probability of an event.
4. Be able to check if two events are independent.
5. Be able to use Bayes' formula to 'invert' conditional probabilities.
6. Be able to organize the computation of conditional probabilities using trees and tables.
7. Understand the base rate fallacy thoroughly.

2 Conditional Probability

Conditional probability answers the question 'how does the probability of an event change if we have extra information'. We'll illustrate with an example.

Example 1. Toss a fair coin 3 times.
(a) What is the probability of 3 heads?

Solution: Sample space $\Omega = \{HHH, HHT, HTH, HTT, THH, THT, TTH, TTT\}$.
All outcomes are equally probable, so $P(3 \text{ heads}) = 1/8$.

(b) Suppose we are told that the first toss was heads. Given this information how should we compute the probability of 3 heads?

Solution: We have a new (reduced) sample space: $\Omega' = \{HHH, HHT, HTH, HTT\}$.
All outcomes are equally probable, so

$$P(3 \text{ heads given that the first toss is heads}) = 1/4.$$

This is called conditional probability, since it takes into account additional conditions. To develop the notation, we rephrase (b) in terms of *events*.

Rephrased (b) Let A be the event 'all three tosses are heads' $= \{HHH\}$.
Let B be the event 'the first toss is heads' $= \{HHH, HHT, HTH, HTT\}$.

The conditional probability of A knowing that B occurred is written

$$P(A|B)$$

This is read as

'the conditional probability of A given B'

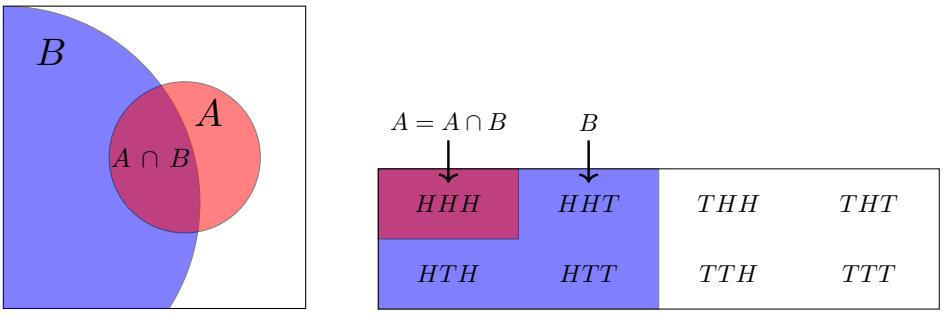
or

'the probability of A conditioned on B'

or simply

'the probability of A given B'.

We can visualize conditional probability as follows. Think of $P(A)$ as the proportion of the area of the *whole* sample space taken up by A. For $P(A|B)$ we restrict our attention to B. That is, $P(A|B)$ is the proportion of area of B taken up by A, i.e. $P(A \cap B)/P(B)$.



Conditional probability: Abstract visualization and coin example

Note, $A \subset B$ in the right-hand figure, so there are only two colors shown.

The formal definition of conditional probability catches the gist of the above example and visualization.

Formal definition of conditional probability
Let A and B be events. We define the conditional probability of A given B as

$$P(A|B) = \frac{P(A \cap B)}{P(B)}, \text{ provided } P(B) \neq 0. \tag{1}$$

Let's redo the coin tossing example using the definition in Equation (1). Recall $A =$ '3 heads' and $B =$ 'first toss is heads'. We have $P(A) = 1/8$ and $P(B) = 1/2$. Since $A \cap B = A$, we also have $P(A \cap B) = 1/8$. Now according to (1),

$$P(A|B) = \frac{P(A \cap B)}{P(B)} = \frac{1/8}{1/2} = 1/4,$$

which agrees with our answer in Example 1 b.

We start with a simple example where we can find all the probabilities directly by counting.

Example 2. Draw two cards from a deck. Define the events: $S_1 =$ 'first card is a spade' and $S_2 =$ 'second card is a spade'. What is the $P(S_2|S_1)$?

Solution: We'll use formula (1) to compute the conditional probability. We have to compute $P(S_1)$, $P(S_2)$ and $P(S_1 \cap S_2)$: We know that $P(S_1) = 1/4$ because there are 52 equally probable ways to draw the first card and 13 of them are spades. The same logic says that there are 52 equally probable ways the second card can be drawn, so $P(S_2) = 1/4$.

Aside: The probability $P(S_2) = 1/4$ may seem surprising since the value of first card certainly affects the probabilities for the second card. However, if we look at *all* possible two card sequences we will see that every card in the deck has equal probability of being

the second card. Since 13 of the 52 cards are spades we get $P(S_2) = 13/52 = 1/4$. Another way to say this is: if we are not given value of the first card then we have to consider all possibilities for the second card.

Continuing, we compute $P(S_1 \cap S_2)$ by counting:

Number of ways to draw a spade followed by a second spade: $13 \cdot 12$.
Number of ways to draw any card followed by any other card: $52 \cdot 51$.

Thus,

$$P(S_1 \cap S_2) = \frac{13 \cdot 12}{52 \cdot 51} = 3/51.$$

Now, using (1) we get

$$P(S_2|S_1) = \frac{P(S_2 \cap S_1)}{P(S_1)} = \frac{3/51}{1/4} = 12/51.$$

This case is simple enough that we can check our answer by computing the conditional probability directly: if the first card is a spade then of the 51 cards remaining, 12 are spades. So, the probability the second card is also a spade is

$$P(S_2|S_1) = 12/51.$$

Warning: In more complicated problems it will be be much harder to compute conditional probability by counting. Usually we have to use Equation (1).

Think: For S_1 and S_2 in the previous example, what is $P(S_2|S_1^c)$?

3 Multiplication Rule

The following formula is called the multiplication rule.

$$P(A \cap B) = P(A|B) \cdot P(B). \tag{2}$$

This is simply a rewriting of the definition in Equation (1) of conditional probability. We will see that our use of the multiplication rule is very similar to our use of the rule of product in counting. In fact, the multiplication rule is just a souped up version of the rule of product.

We start by verifying the multiplication rule for the previous example.

Example 3. Draw two cards from a deck. Define the events: S_1 = 'first card is a spade' and S_2 = 'second card is a spade'. Verify the multiplication rule.

Solution: From the previous example, we know $P(S_2|S_1) = 12/51$, $P(S_1 \cap S_2) = 3/51$, $P(S_1) = 1/4$. From this it is easy to check that

$$P(S_2|S_1) \cdot P(S_1) = \frac{12}{51} \cdot \frac{1}{4} = \frac{3}{51} = P(S_1 \cap S_2).$$

4 Law of Total Probability

The law of total probability will allow us to use the multiplication rule to find probabilities in more interesting examples. It involves a lot of notation, but the idea is fairly simple. We

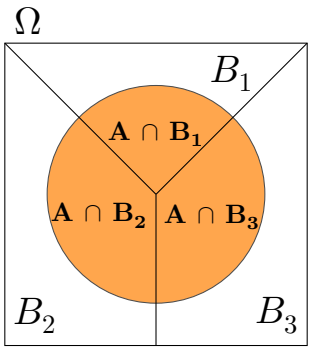
state the law when the sample space is divided into 3 pieces. It is a simple matter to extend the rule when there are more than 3 pieces.

Law of Total Probability

Suppose the sample space Ω is divided into 3 disjoint events B_1, B_2, B_3 (see the figure below). Then for any event A:

$$P(A) = P(A \cap B_1) + P(A \cap B_2) + P(A \cap B_3)$$
$$P(A) = P(A|B_1)\,P(B_1) + P(A|B_2)\,P(B_2) + P(A|B_3)\,P(B_3) \tag{3}$$

The top equation says 'if A is divided into 3 pieces then $P(A)$ is the sum of the probabilities of the pieces'. The bottom equation (3) is called the law of total probability. It is just a rewriting of the top equation using the multiplication rule.



The sample space Ω and the event A are each divided into 3 disjoint pieces.

The law holds if we divide Ω into any number of events, so long as they are *disjoint* and *cover* all of Ω. Such a division is often called a *partition* of Ω.

Our first example will be one where we already know the answer and can verify the law.

Example 4. An urn contains 5 red balls and 2 green balls. Two balls are drawn one after the other. What is the probability that the second ball is red?

Solution: The sample space is $\Omega = \{\text{rr, rg, gr, gg}\}$.

Let R_1 be the event 'the first ball is red', G_1 = 'first ball is green', R_2 = 'second ball is red', G_2 = 'second ball is green'. We are asked to find $P(R_2)$.

Let's compute this same value using the law of total probability (3). First, we'll find the conditional probabilities. This is a simple counting exercise.

$$P(R_2|R_1) = 4/6, \qquad P(R_2|G_1) = 5/6.$$

Since R_1 and G_1 partition Ω the law of total probability says

$$\begin{aligned} P(R_2) &= P(R_2|R_1)P(R_1) + P(R_2|G_1)P(G_1) \\ &= \frac{4}{6}\cdot\frac{5}{7} + \frac{5}{6}\cdot\frac{2}{7} \\ &= \frac{30}{42} = \frac{5}{7}. \end{aligned} \tag{4}$$

Of course, this example is simple enough that we could have computed $P(R_2)$ directly the same way we found $P(S_2)$ directly in the card example. But, we will see that in more complicated examples the law of total probability is truly necessary.

Probability urns

The example above used probability urns. Their use goes back to the beginning of the subject and we would be remiss not to introduce them. This toy model is very useful. We quote from Wikipedia: https://en.wikipedia.org/wiki/Urn_problem

> In probability and statistics, an urn problem is an idealized mental exercise in which some objects of real interest (such as atoms, people, cars, etc.) are represented as colored balls in an urn or other container. One pretends to draw (remove) one or more balls from the urn; the goal is to determine the probability of drawing one color or another, or some other properties. A key parameter is whether each ball is returned to the urn after each draw.

It doesn't take much to make an example where (3) is really the best way to compute the probability. Here is a game with slightly more complicated rules.

Example 5. An urn contains 5 red balls and 2 green balls. A ball is drawn. If it's green a red ball is added to the urn and if it's red a green ball is added to the urn. (The original ball is not returned to the urn.) Then a second ball is drawn. What is the probability the second ball is red?

Solution: The law of total probability says that $P(R_2)$ can be computed using the expression in Equation (4). Only the values for the probabilities will change. We have

$$P(R_2|R_1) = 4/7, \qquad P(R_2|G_1) = 6/7.$$

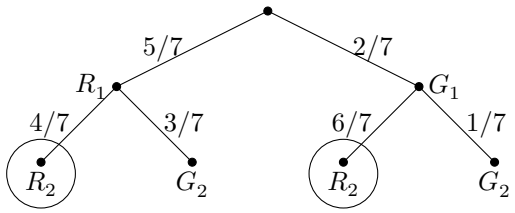
Therefore,

$$P(R_2) = P(R_2|R_1)P(R_1) + P(R_2|G_1)P(G_1) = \frac{4}{7}\cdot\frac{5}{7} + \frac{6}{7}\cdot\frac{2}{7} = \frac{32}{49}.$$

5 Using Trees to Organize the Computation

Trees are a great way to organize computations with conditional probability and the law of total probability. The figures and examples will make clear what we mean by a tree. As with the rule of product, the key is to organize the underlying process into a sequence of actions.

We start by redoing Example 5. The sequence of actions are: first draw ball 1 (and add the appropriate ball to the urn) and then draw ball 2.



You interpret this tree as follows. Each dot is called a node. The tree is organized by levels. The top node (root node) is at level 0. The next layer down is level 1 and so on. Each level shows the outcomes at one stage of the game. Level 1 shows the possible outcomes of the first draw. Level 2 shows the possible outcomes of the second draw starting from each node in level 1.

Probabilities are written along the branches. The probability of R_1 (red on the first draw) is 5/7. It is written along the branch from the root node to the one labeled R_1. At the next level we put in conditional probabilities. The probability along the branch from R_1 to R_2 is $P(R_2|R_1) = 4/7$. It represents the probability of going to node R_2 given that you are already at R_1.

The muliplication rule says that the probability of getting to any node is just the product of the probabilities along the path to get there. For example, the node labeled R_2 at the far left really represents the event $R_1 \cap R_2$ because it comes from the R_1 node. The multiplication rule now says

$$P(R_1 \cap R_2) = P(R_1) \cdot P(R_2|R_1) = \frac{5}{7} \cdot \frac{4}{7},$$

which is exactly multiplying along the path to the node.

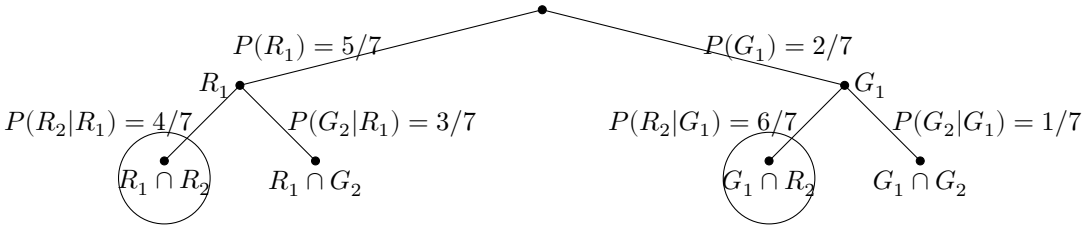
The law of total probability is just the statement that $P(R_2)$ is the sum of the probabilities of all paths leading to R_2 (the two circled nodes in the figure). In this case,

$$P(R_2) = \frac{5}{7} \cdot \frac{4}{7} + \frac{2}{7} \cdot \frac{6}{7} = \frac{32}{49},$$

exactly as in the previous example.

5.1 Shorthand vs. precise trees

The tree given above involves some shorthand. For example, the node marked R_2 at the far left really represents the event $R_1 \cap R_2$, since it ends the path from the root through R_1 to R_2. Here is the same tree with everything labeled precisely. As you can see this tree is more cumbersome to make and use. We usually use the shorthand version of trees. You should make sure you know how to interpret them precisely.



6 Independence

Two events are independent if knowledge that one occurred does not change the probability that the other occurred. Informally, events are independent if they do not influence one another.

Example 6. Toss a coin twice. We expect the outcomes of the two tosses to be independent of one another. In real experiments this always has to be checked. If my coin lands in honey and I don't bother to clean it, then the second toss might be affected by the outcome of the first toss.

More seriously, the independence of experiments can by undermined by the failure to clean or recalibrate equipment between experiments or to isolate supposedly independent observers from each other or a common influence. We've all experienced hearing the same 'fact' from different people. Hearing it from different sources tends to lend it credence until we learn that they all heard it from a common source. That is, our sources were not independent.

Translating the verbal description of independence into symbols gives

$$A \text{ is independent of } B \quad \text{if} \quad P(A|B) = P(A). \tag{5}$$

That is, knowing that B occurred does not change the probability that A occurred. In terms of events as subsets, knowing that the realized outcome is in B does not change the probability that it is in A.

If A and B are independent in the above sense, then the multiplication rule gives

$$P(A \cap B) = P(A|B) \cdot P(B) = P(A) \cdot P(B).$$

This justifies the following technical definition of independence.

Formal definition of independence: Two events A and B are independent if

$$P(A \cap B) \;=\; P(A) \cdot P(B) \tag{6}$$

This is a nice symmetric definition which makes clear that A is independent of B if and only if B is independent of A. Unlike the equation with conditional probabilities, this definition makes sense even when $P(B) = 0$. In terms of conditional probabilities, we have:

1. If $P(B) \neq 0$ then A and B are independent if and only if $P(A|B) = P(A)$.
2. If $P(A) \neq 0$ then A and B are independent if and only if $P(B|A) = P(B)$.

Independent events commonly arise as different trials in an experiment, as in the following example.

Example 7. Toss a fair coin twice. Let H_1 = 'heads on first toss' and let H_2 = 'heads on second toss'. Are H_1 and H_2 independent?

Solution: Since $H_1 \cap H_2$ is the event 'both tosses are heads' we have

$$P(H_1 \cap H_2) = 1/4 = P(H_1)P(H_2).$$

Therefore the events are independent.

We can ask about the independence of any two events, as in the following two examples.

Example 8. Toss a fair coin 3 times. Let H_1 = 'heads on first toss' and A = 'two heads total'. Are H_1 and A independent?

Solution: We know that $P(A) = 3/8$. Since this is not 0 we can check if the formula in Equation 5 holds. Now, H_1 = {HHH, HHT, HTH, HTT} contains exactly two outcomes (HHT, HTH) from A, so we have $P(A|H_1) = 2/4$. Since $P(A|H_1) \neq P(A)$ these events are not independent.

Example 9. Draw one card from a standard deck of playing cards. Let's examine the independence of 3 events 'the card is an ace', 'the card is a heart' and 'the card is red'.

Define the events as A = 'ace', H = 'hearts', R = 'red'.

(a) We know that $P(A) = 4/52$ (4 out of 52 cards are aces), $P(A|H) = 1/13$ (1 out of 13 hearts are aces). Since $P(A) = P(A|H)$ we have that A is independent of H.

(b) $P(A|R) = 2/26 = 1/13 = P(A)$. So A is independent of R. That is, whether the card is an ace is independent of whether it is red.

(c) Finally, what about H and R? Since $P(H) = 1/4$ and $P(H|R) = 1/2$, H and R are not independent. We could also see this the other way around: $P(R) = 1/2$ and $P(R|H) = 1$, so H and R are not independent. That is, the suit of a card is not independent of the color of the card's suit.

6.1 Paradoxes of Independence

An event A with probability 0 is independent of itself, since in this case both sides of equation (6) are 0. This appears paradoxical because knowledge that A occurred certainly gives information about whether A occurred. We resolve the paradox by noting that since $P(A) = 0$ the statement 'A occurred' is vacuous.

Think: For what other value(s) of $P(A)$ is A independent of itself?

7 Bayes' Theorem

Bayes' theorem is a pillar of both probability and statistics and it is central to the rest of this course. For two events A and B Bayes' theorem (also called Bayes' rule and Bayes' formula) says

$$P(B|A) = \frac{P(A|B) \cdot P(B)}{P(A)}. \tag{7}$$

Comments: 1. Bayes' rule tells us how to 'invert' conditional probabilities, i.e. to find $P(B|A)$ from $P(A|B)$.
2. In practice, $P(A)$ is often computed using the law of total probability.

Proof of Bayes' rule
The key point is that $A \cap B$ is symmetric in A and B. So the multiplication rule says

$$P(B|A) \cdot P(A) = P(A \cap B) = P(A|B) \cdot P(B).$$

Now divide through by $P(A)$ to get Bayes' rule.

A common mistake is to confuse the meanings of $P(A|B)$ and $P(B|A)$. They can be very different. This is illustrated in the next example.

Example 10. Toss a coin 5 times. Let H_1 = 'first toss is heads' and let H_A = 'all 5 tosses are heads'. Then $P(H_1|H_A) = 1$ but $P(H_A|H_1) = 1/16$.

For practice, let's use Bayes' theorem to compute $P(H_1|H_A)$ using $P(H_A|H_1)$.The terms

are $P(H_A|H_1) = 1/16$, $P(H_1) = 1/2$, $P(H_A) = 1/32$. So,

$$P(H_1|H_A) = \frac{P(H_A|H_1)P(H_1)}{P(H_A)} = \frac{(1/16)\cdot(1/2)}{1/32} = 1,$$

which agrees with our previous calculation.

7.1 The Base Rate Fallacy

The base rate fallacy is one of many examples showing that it's easy to confuse the meaning of $P(B|A)$ and $P(A|B)$ when a situation is described in words. This is one of the key examples from probability and it will inform much of our practice and interpretation of statistics. You should strive to understand it thoroughly.

Example 11. The Base Rate Fallacy
Consider a routine screening test for a disease. Suppose the frequency of the disease in the population (base rate) is 0.5%. The test is fairly accurate with a 5% false positive rate and a 10% false negative rate.

You take the test and it comes back positive. What is the probability that you have the disease?

Solution: We will do the computation three times: using trees, tables and symbols. We'll use the following notation for the relevant events:
D^+ = 'you have the disease'
D^- = 'you do not have the disease
T^+ = 'you tested positive'
T^- = 'you tested negative'.

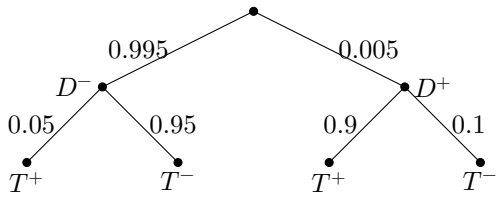
We are given $P(D+) = 0.005$ and therefore $P(D-) = 0.995$. The false positive and false negative rates are (by definition) conditional probabilities.

$$P(\text{false positive}) = P(T^+|D^-) = 0.05 \quad \text{and} \quad P(\text{false negative}) = P(T^-|D^+) = 0.1.$$

The complementary probabilities are known as the true negative and true positive rates:

$$P(T^-|D^-) = 1 - P(T^+|D^-) = 0.95 \quad \text{and} \quad P(T^+|D^+) = 1 - P(T^-|D^+) = 0.9.$$

Trees: All of these probabilities can be displayed quite nicely in a tree.



The question asks for the probability that you have the disease given that you tested positive, i.e. what is the value of $P(D^+|T^+)$. We aren't given this value, but we do know $P(T^+|D^+)$, so we can use Bayes' theorem.

$$P(D^+|T^+) = \frac{P(T^+|D^+)\cdot P(D^+)}{P(T^+)}.$$

The two probabilities in the numerator are given. We compute the denominator $P(T^+)$ using the law of total probability. Using the tree, we just have to sum the probabilities for each of the nodes marked T^+

$$P(T^+) = 0.995 \times 0.05 + 0.005 \times 0.9 = 0.05425$$

Thus,

$$P(D^+|T^+) = \frac{0.9 \times 0.005}{0.05425} = 0.082949 \approx 8.3\%.$$

Remarks: This is called the base rate fallacy because the base rate of the disease in the population is so low that the vast majority of the people taking the test are healthy, and even with an accurate test most of the positives will be healthy people. Ask your doctor for his/her guess at the odds.

To summarize the base rate fallacy with specific numbers

95% of all tests are accurate does not imply 95% of positive tests are accurate

We will refer back to this example frequently. It and similar examples are at the heart of many statistical misunderstandings.

Other ways to work Example 11

Tables: Another trick that is useful for computing probabilities is to make a table. Let's redo the previous example using a table built with 10000 total people divided according to the probabilites in this example.

We construct the table as follows. Pick a number, say 10000 people, and place it as the grand total in the lower right. Using $P(D^+) = 0.005$ we compute that 50 out of the 10000 people are sick (D^+). Likewise 9950 people are healthy (D^-). At this point the table looks like:

	$D+$	$D-$	total
T^+			
T^-			
total	50	9950	10000

Using $P(T^+|D^+) = 0.9$ we can compute that the number of sick people who tested positive as 90% of 50 or 45. The other entries are similar. At this point the table looks like the table below on the left. Finally we sum the T^+ and T^- rows to get the completed table on the right.

	D^+	D^-	total
T^+	45	498	
T^-	5	9452	
total	50	9950	10000

	D^+	D^-	total
T^+	45	498	543
T^-	5	9452	9457
total	50	9950	10000

Using the complete table we can compute

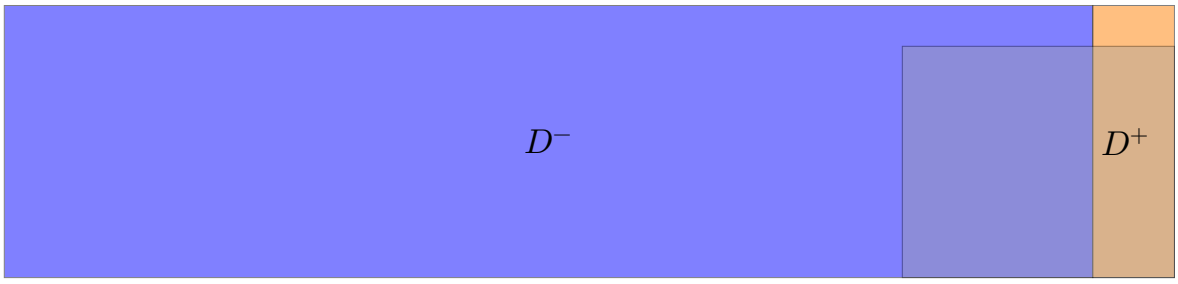
$$P(D^+|T^+) = \frac{|D^+ \cap T^+|}{|T^+|} = \frac{45}{543} = 8.3\%.$$

Symbols: For completeness, we show how the solution looks when written out directly in

symbols.

$$\begin{aligned} P(D^+|T^+) &= \frac{P(T^+|D^+) \cdot P(D^+)}{P(T^+)} \\ &= \frac{P(T^+|D^+) \cdot P(D^+)}{P(T^+|D^+) \cdot P(D^+) + P(T^+|D^-) \cdot P(D^-)} \\ &= \frac{0.9 \times 0.005}{0.9 \times 0.005 + 0.05 \times 0.995} \\ &= 8.3\% \end{aligned}$$

Visualization: The figure below illustrates the base rate fallacy. The large blue rectangle represents all the healthy people. The much smaller orange rectangle represents the sick people. The shaded rectangle represents the people who test positive. The shaded area covers most of the orange area and only a small part of the blue area. Even so, the most of the shaded area is over the blue. That is, most of the positive tests are of healthy people.



7.2 Bayes' rule in 18.05

As we said at the start of this section, Bayes' rule is a pillar of probability and statistics. We have seen that Bayes' rule allows us to 'invert' conditional probabilities. When we study statistics we will see that the art of statistical inference involves deciding how to proceed when one (or more) of the terms on the right side of Bayes' rule is unknown.

MIT OpenCourseWare
https://ocw.mit.edu

18.05 Introduction to Probability and Statistics
Spring 2022

For information about citing these materials or our Terms of Use, visit: https://ocw.mit.edu/terms.

Lecture 8: Time Series Analysis

MIT 18.S096

Dr. Kempthorne

Fall 2013

Outline

1. Time Series Analysis
   - Stationarity and Wold Representation Theorem
   - Autoregressive and Moving Average (ARMA) Models
   - Accommodating Non-Stationarity: ARIMA Models
   - Estimation of Stationary ARMA Models
   - Tests for Stationarity/Non-Stationarity

Stationarity and Wold Representation Theorem

A stochastic process $\{\ldots, X_{t-1}, X_t, X_{t+1}, \ldots\}$ consisting of random variables indexed by time index t is a **time series**.

The stochastic behavior of $\{X_t\}$ is determined by specifying the probability density/mass functions (pdf's)

$$p(x_{t_1}, x_{t_2}, \ldots, x_{t_m})$$

for all finite collections of time indexes

$$\{(t_1, t_2, \ldots, t_m), \quad m < \infty\}$$

i.e., all finite-dimensional distributions of $\{X_t\}$.

Definition: A time series $\{X_t\}$ is **Strictly Stationary** if

$$p(t_1 + \tau, t_2 + \tau, \ldots, t_m + \tau) = p(t_1, t_2, \ldots, t_m),$$
$$\forall \tau, \forall m, \forall (t_1, t_2, \ldots, t_m).$$

(Invariance under time translation)

Definitions of Stationarity

Definition: A time series $\{X_t\}$ is **Covariance Stationary** if

$$\begin{aligned} E(X_t) &= \mu \\ Var(X_t) &= \sigma_X^2 \\ Cov(X_t, X_{t+\tau}) &= \gamma(\tau) \end{aligned}$$

(all constant over time t)

The **auto-correlation function** of $\{X_t\}$ is

$$\begin{aligned} \rho(\tau) &= Cov(X_t, X_{t+\tau}) / \sqrt{Var(X_t) \cdot Var(X_{t+\tau})} \\ &= \gamma(\tau)/\gamma(0) \end{aligned}$$

Representation Theorem

Wold Representation Theorem: Any zero-mean covariance stationary time series $\{X_t\}$ can be decomposed as $X_t = V_t + S_t$ where

- $\{V_t\}$ is a linearly deterministic process, i.e., a linear combination of past values of V_t with constant coefficients.
- $S_t = \sum_{i=0}^{\infty} \psi_i \eta_{t-i}$ is an infinite moving average process of error terms, where
  - $\psi_0 = 1, \sum_{i=0}^{\infty} \psi_i^2 < \infty$
  - $\{\eta_t\}$ is linearly unpredictable white noise, i.e.,
    $$E(\eta_t) = 0,\ E(\eta_t^2) = \sigma^2,\ E(\eta_t \eta_s) = 0\ \forall t,\ \forall s \neq t,$$
    and $\{\eta_t\}$ is uncorrelated with $\{V_t\}$:
    $$E(\eta_t V_s) = 0,\ \forall t, s$$

Intuitive Application of the Wold Representation Theorem

Suppose we want to specify a covariance stationary time series $\{X_t\}$ to model actual data from a real time series

$$\{x_t, t = 0, 1, \ldots, T\}$$

Consider the following strategy:

- Initialize a parameter p, the number of past observations in the linearly deterministic term of the Wold Decomposition of $\{X_t\}$
- Estimate the linear projection of X_t on $(X_{t-1}, X_{t-2}, \ldots, X_{t-p})$
  - Consider an estimation sample of size n with endpoint $t_0 \leq T$.
  - Let $\{j = -(p-1), \ldots, 0, 1, 2, \ldots n\}$ index the subseries of $\{t = 0, 1, \ldots, T\}$ corresponding to the estimation sample and define $\{y_j : y_j = x_{t_0-n+j}\}$, (with $t_0 \geq n + p$)
  - Define the vector $\mathbf{Y}$ $_{(T \times 1)}$ and matrix $\mathbf{Z}$ $_{(T \times [p+1])}$ as:

- Estimate the linear projection of X_t on $(X_{t-1}, X_{t-2}, \ldots, X_{t-p})$ (continued)

$$\mathbf{y} = \begin{pmatrix} y_1 \\ y_2 \\ \vdots \\ y_n \end{pmatrix} \quad \mathbf{Z} = \begin{bmatrix} 1 & y_0 & y_{-1} & \cdots & y_{-(p-1)} \\ 1 & y_1 & y_0 & \cdots & y_{-(p-2)} \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ 1 & y_{n-1} & y_{n-2} & \cdots & y_{n-p} \end{bmatrix}$$

  - Apply OLS to specify the projection:

$$\begin{aligned} \hat{\mathbf{y}} &= \mathbf{Z}(\mathbf{Z}^T\mathbf{Z})^{-1}\mathbf{Z}\mathbf{y} \\ &= \hat{P}(Y_t \mid Y_{t-1}, Y_{t-2}, \ldots Y_{t-p}) \\ &= \hat{\mathbf{y}}^{(p)} \end{aligned}$$

- Compute the projection residual

$$\hat{\epsilon}^{(p)} = \mathbf{y} - \hat{\mathbf{y}}^{(p)}$$

- Apply time series methods to the time series of residuals $\{\hat{\epsilon}_j^{(p)}\}$ to specify a moving average model:
$$\epsilon_t^{(p)} = \sum_{i=0}^{\infty} \psi_j \eta_{t-i}$$
yielding $\{\hat{\psi}_j\}$ and $\{\hat{\eta}_t\}$, estimates of parameters and innovations.
- Conduct a case analysis diagnosing consistency with model assumptions
  - Evaluate orthogonality of $\hat{\epsilon}^{(p)}$ to Y_{t-s}, $s > p$.
    If evidence of correlation, increase p and start again.
  - Evaluate the consistency of $\{\hat{\eta}_t\}$ with the white noise assumptions of the theorem.
    If evidence otherwise, consider revisions to the overall model
    - Changing the specification of the moving average model.
    - Adding additional 'deterministic' variables to the projection model.

Note:

- Theoretically,
$$\lim_{p\to\infty} \hat{\mathbf{y}}^{(p)} = \hat{\mathbf{y}} = P(Y_t \mid Y_{t-1}, Y_{t-2}, \ldots)$$
but if $p \to \infty$ is required, then $n \to \infty$ while $p/n \to 0$.
- Useful models of covariance stationary time series have
  - Modest finite values of p and/or include
  - Moving average models depending on a parsimonious number of parameters.

Lag Operator $L()$

Definition The lag operator $L()$ shifts a time series back by one time increment. For a time series $\{X_t\}$:

$$L(X_t) = X_{t-1}.$$

Applying the operator recursively we define:

$$\begin{aligned} L^0(X_t) &= X_t \\ L^1(X_t) &= X_{t-1} \\ L^2(X_t) &= L(L(X_t)) = X_{t-2} \\ &\dots \\ L^n(X_t) &= L(L^{n-1}(X_t)) = X_{t-n} \end{aligned}$$

Inverses of these operators are well defined as:

$$L^{-n}(X_t) = X_{t+n}, \text{ for } n = 1, 2, \dots$$

Wold Representation with Lag Operators

The Wold Representation for a covariance stationary time series $\{X_t\}$ can be expressed as

$$\begin{aligned} X_t &= \sum_{i=0}^{\infty} \psi_i \eta_{t-i} + V_t \\ &= \sum_{i=0}^{\infty} \psi_i L^i(\eta_t) + V_t \\ &= \psi(L)\eta_t + V_t \end{aligned}$$

where $\psi(L) = \sum_{i=0}^{\infty} \psi_i L^i$.

Definition The **Impulse Response Function** of the covariance stationary process $\{X_t\}$ is

$$IR(j) = \frac{\partial X_t}{\partial \eta_{t-j}} = \psi_j.$$

The **long-run cumulative response** of $\{X_t\}$ is

$$\sum_{i=0}^{\infty} IR(j) = \sum_{i=0}^{\infty} \psi_i = \psi(L) \text{ with } L = 1.$$

Equivalent Auto-regressive Representation

Suppose that the operator $\psi(L)$ is invertible, i.e.,
$\psi^{-1}(L) = \sum_{i=0}^{\infty} \psi_i^* L^i$ such that
$\psi^{-1}(L)\psi(L) = I = L^0$.

Then, assuming $V_t = 0$ (i.e., X_t has been adjusted to $X_t^* = X_t - V_t$), we have the following equivalent expressions of the time series model for $\{X_t\}$

$$\begin{aligned} X_t &= \psi(L)\eta_t \\ \psi^{-1}(L)X_t &= \eta_t \end{aligned}$$

Definition When $\psi^{-1}(L)$ exists, the time series $\{X_t\}$ is **Invertible** and has an auto-regressive representation:

$$X_t = (\sum_{i=0}^{\infty} \psi_i^* X_{t-i}) + \eta_t$$

Outline

1. Time Series Analysis
   - Stationarity and Wold Representation Theorem
   - Autoregressive and Moving Average (ARMA) Models
   - Accommodating Non-Stationarity: ARIMA Models
   - Estimation of Stationary ARMA Models
   - Tests for Stationarity/Non-Stationarity

ARMA(p,q) Models

Definition: The times series $\{X_t\}$ follows the $ARMA(p,q)$ **Model** with auto-regressive order p and moving-average order q if

$$\begin{aligned} X_t &= \mu + \phi_1(X_{t-1} - \mu) + \phi_2(X_{t-1} - \mu) + \cdots \phi_p(X_{t-p} - \mu) \\ &\quad + \eta_t + \theta_1\eta_{t-1} + \theta_2\eta_{t-2} + \cdots \theta_q\eta_{t-q} \end{aligned}$$

where $\{\eta_t\}$ is $WN(0, \sigma^2)$, "**White Noise**" with

$$\begin{aligned} E(\eta_t) &= 0, \qquad \forall t \\ E(\eta_t^2) &= \sigma^2 < \infty, \quad \forall t \text{ , and } E(\eta_t\eta_s) = 0, \quad \forall t \neq s \end{aligned}$$

With lag operators

$$\phi(L) = (1 - \phi_1 L - \phi_2 L^2 - \cdots \phi_p L^P) \text{ and}$$
$$\theta(L) = (1 + \theta_1 L + \theta_2 L^2 + \cdots + \theta_q L^q)$$

we can write

$$\phi(L) \cdot (X_t - \mu) = \theta(L)\eta_t$$

and the Wold decomposition is

$$X_t = \mu + \psi(L)\eta_t, \text{ where } \psi(L) = [\phi(L)])^{-1}\theta(L)$$

AR(p) Models

Order-p Auto-Regression Model: AR(p)

$$\phi(L) \cdot (X_t - \mu) = \eta_t \text{ where}$$
$$\{\eta_t\} \text{ is } WN(0, \sigma^2) \text{ and}$$
$$\phi(L) = 1 - \phi_1 L - \phi_2 L^2 - \cdots + \phi_p L^p$$

Properties:

- Linear combination of $\{X_t, X_{t-1}, \ldots X_{t-p}\}$ is $WN(0, \sigma^2)$.
- X_t follows a linear regression model on explanatory variables $(X_{t-1}, X_{t-2}, \ldots, X_{t-p})$, i.e

$$X_t = c + \sum_{j=1}^{p} \phi_j X_{t-j} + \eta_t$$

where $c = \mu \cdot \phi(1)$, (replacing L by 1 in $\phi(L)$).

AR(p) Models

Stationarity Conditions

Consider $\phi(z)$ replacing L with a complex variable z.

$$\phi(z) = 1 - \phi_1 z - \phi_2 z^2 - \cdots \phi_p z^p.$$

Let $\lambda_1, \lambda_2, \ldots \lambda_p$ be the p roots of $\phi(z) = 0$.

$$\phi(L) = (1 - \tfrac{1}{\lambda_1} L) \cdot (1 - \tfrac{1}{\lambda_2} L) \cdots (1 - \tfrac{1}{\lambda_p} L)$$

Claim: $\{X_t\}$ is covariance stationary if and only if all the roots of $\phi(z) = 0$ (the "**characteristic equation**") lie outside the unit circle $\{z : |z| \leq 1\}$, i.e., $|\lambda_j| > 1, j = 1, 2, \ldots, p$

- For complex number λ: $|\lambda| > 1$,

$$\begin{aligned}(1 - \tfrac{1}{\lambda} L)^{-1} &= 1 + (\tfrac{1}{\lambda})L + (\tfrac{1}{\lambda})^2 L^2 + (\tfrac{1}{\lambda})^3 L^3 + \cdots \\ &= \textstyle\sum_{i=0}^{\infty} (\tfrac{1}{\lambda})^i L^i\end{aligned}$$

- $\phi^{-1}(L) = \prod_{j=1}^{p} \left[\left(1 - \frac{1}{\lambda_j} L\right)^{-1} \right]$

AR(1) Model

Suppose $\{X_t\}$ follows the $AR(1)$ process, i.e.,

$$X_t - \mu = \phi(X_{t-1} - \mu) + \eta_t,\ t = 1, 2, \ldots$$

where $\eta_t \sim WN(0, \sigma^2)$.

- The characteristic equation for the $AR(1)$ model is
$$(1 - \phi z) = 0$$
with root $\lambda = \frac{1}{\phi}$.
- The $AR(1)$ model is covariance stationary if (and only if)
$$|\phi| < 1 \quad \text{(equivalently } |\lambda| > 1\text{)}$$
- The first and second moments of $\{X_t\}$ are

$$\begin{aligned}
E(X_t) &= \mu \\
Var(X_t) &= \sigma_X^2 = \sigma^2/(1-\phi) && (= \gamma(0)) \\
Cov(X_t, X_{t-1}) &= \phi \cdot \sigma_X^2 \\
Cov(X_t, X_{t-j}) &= \phi^j \cdot \sigma_X^2 && (= \gamma(j)) \\
Corr(X_t, X_{t-j}) &= \phi^j = \rho(j) && (= \gamma(j)/\gamma(0))
\end{aligned}$$

AR(1) Model

- For $\phi : |\phi| < 1$, the Wold decomposition of the $AR(1)$ model is: $X_t = \mu + \sum_{j=0}^{\infty} \phi^j \eta_{t-j}$
  - For $\phi : 0 < \phi < 1$, the $AR(1)$ process exhibits exponential mean-reversion to μ
  - For $\phi : 0 > \phi > -1$, the $AR(1)$ process exhibits oscillating exponential mean-reversion to μ
- For $\phi = 1$, the Wold decomposition does not exist and the process is the simple random walk (non-stationary!).
- For $\phi > 1$, the $AR(1)$ process is explosive.

Examples of $AR(1)$ Models (mean reverting with $0 < \phi < 1$)

- Interest rates (Ornstein Uhlenbeck Process; Vasicek Model)
- Interest rate spreads
- Real exchange rates
- Valuation ratios (dividend-to-price, earnings-to-price)

Yule Walker Equations for AR(p) Processes

Second Order Moments of $AR(p)$ Processes

From the specification of the $AR(p)$ model:

$$(X_t - \mu) = \phi_1(X_{t-1} - \mu) + \phi_2(X_{t-1} - \mu) + \cdots + \phi_p(X_{t-p} - \mu) + \eta_t$$

we can write the **Yule-Walker Equations** $(j = 0, 1, \ldots)$

$$\begin{aligned} E[(X_t - \mu)(X_{t-j} - \mu)] &= \phi_1 E[(X_{t-1} - \mu)(X_{t-j} - \mu)] \\ &\quad + \phi_2 E[(X_{t-1} - \mu)(X_{t-j} - \mu)] + \\ &\quad \cdots + \phi_p E[(X_{t-p} - \mu)(X_{t-j} - \mu)] \\ &\quad + E[\eta_t(X_{t-j} - \mu)] \\ \gamma(j) &= \phi_1\gamma(j-1) + \phi_2\gamma(j-2) + \\ &\quad \cdots + \phi_p\gamma(j-p) + \delta_{0,j}\sigma^2 \end{aligned}$$

Equations $j = 1, 2, \ldots p$ yield a system of p linear equations in ϕ_j:

Yule-Walker Equations

$$\begin{pmatrix} \gamma(1) \\ \gamma(2) \\ \vdots \\ \gamma(p) \end{pmatrix} = \begin{bmatrix} \gamma(0) & \gamma(-1) & \gamma(-2) & \cdots & \gamma(-(p-1)) \\ \gamma(1) & \gamma(0) & \gamma(-1) & \cdots & \gamma(-(p-2)) \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ \gamma(p-1) & \gamma(p-2) & \gamma(p-3) & \cdots & \gamma(0)) \end{bmatrix} \begin{pmatrix} \phi_1 \\ \phi_2 \\ \vdots \\ \phi_p \end{pmatrix}$$

- Given estimates $\hat{\gamma}(j), j = 0, \ldots, p$ (and $\hat{\mu}$) the solution of these equations are the Yule-Walker estimates of the ϕ_j; using the property $\gamma(-j) = \gamma(+j)$, $\forall j$
- Using these in equation 0
$$\gamma(0) = \phi_1\gamma(-1) + \phi_2\gamma(-2) + \cdots + \phi_p\gamma(-p) + \delta_{0,0}\sigma^2$$
provides an estimate of σ^2
$$\hat{\sigma}^2 = \hat{\gamma}(0) - \sum_{j-1}^{p} \hat{\phi}_j\hat{\gamma}(j)$$
- When all the estimates $\hat{\gamma}(j)$ and $\hat{\mu}$ are unbiased, then the Yule-Walker estimates apply the **Method of Moments** Principle of Estimation.

MA(q) Models

Order-q Moving-Average Model: MA(q)

$$(X_t - \mu) = \theta(L)\eta_t, \text{ where}$$
$$\{\eta_t\} \text{ is } WN(0, \sigma^2) \text{ and}$$
$$\theta(L) = 1 + \theta_1 L + \theta_2 L^2 + \cdots + \theta_q L^q$$

Properties:

- The process $\{X_t\}$ is invertible if all the roots of $\theta(z) = 0$ are outside the complex unit circle.
- The moments of X_t are:

$$\begin{aligned} E(X_t) &= \mu \\ Var(X_t) &= \gamma(0) = \sigma^2 \cdot (1 + \theta_1^2 + \theta_2^2 + \cdots + \theta_q^2) \end{aligned}$$

$$Cov(X_t, X_{t+j}) = \begin{cases} 0, & j > q \\ \sigma^2 \cdot (\theta_j + \theta_{j+1}\theta_1 + \theta_{j+2}\theta_2 + \cdots \theta_q\theta_{q-j}), & 1 < j \leq q \end{cases}$$

Outline

1. Time Series Analysis
   - Stationarity and Wold Representation Theorem
   - Autoregressive and Moving Average (ARMA) Models
   - Accommodating Non-Stationarity: ARIMA Models
   - Estimation of Stationary ARMA Models
   - Tests for Stationarity/Non-Stationarity

Accommodating Non-Stationarity by Differencing

Many economic time series exhibit non-stationary behavior consistent with random walks. Box and Jenkins advocate removal of non-stationary trending behavior using

Differencing Operators:

$$\begin{aligned}\Delta &= 1 - L \\ \Delta^2 &= (1-L)^2 = 1 - 2L + L^2 \\ \Delta^k &= (1-L)^k = \sum_{j=0}^{k} \binom{k}{j} (-L)^j, \quad (\text{integral } k > 0)\end{aligned}$$

- If the process $\{X_t\}$ has a linear trend in time, then the process $\{\Delta X_t\}$ has no trend.
- If the process $\{X_t\}$ has a quadratic trend in time, then the second-differenced process $\{\Delta^2 X_t\}$ has no trend.

Examples of Non-Stationary Processes

Linear Trend Reversion Model: Suppose the model for the time series $\{X_t\}$ is:

$$X_t = TD_t + \eta_t, \text{ where}$$

- $TD_t = a + bt$, a deterministic (linear) trend
- $\eta_t \sim AR(1)$, i.e.,

$$\eta_t = \phi\eta_{t-1} + \xi_t, \text{ where } |\phi| < 1 \text{ and}$$
$$\{\xi_t\} \text{ is } WN(0, \sigma^2).$$

The moments of $\{X_t\}$ are:

$$E(X_t) = E(TD_t) + E(\eta_t) = a + bt$$
$$Var(X_t) = Var(\eta_t) = \sigma^2/(1-\phi).$$

The differenced process $\{\Delta X_t\}$ can be expressed as

$$\begin{aligned} \Delta X_t &= b + \Delta\eta_t \\ &= b + (\eta_t - \eta_{t-1}) \\ &= b + (1-L)\eta_t \end{aligned}$$

Non-Stationary Trend Processes

Pure Integrated Process I(1) for $\{X_t\}$:

$$X_t = X_{t-1} + \eta_t, \text{ where } \eta_t \text{ is } WN(0, \sigma^2).$$

Equivalently:

$$\Delta X_t = (1 - L)X_t + \eta_t, \text{ where } \{\eta_t\} \text{ is } WN(0, \sigma^2).$$

Given X_0, we can write $X_t = X_0 + TS_t$ where

$$TS_t = \sum_{j=0}^{t} \eta_j$$

The process $\{TS_t\}$ is a **Stochastic Trend** process with

$$TS_t = TS_{t-1} + \eta_t, \text{ where } \{\eta_t\} \text{ is } WN(0, \sigma^2).$$

Note:

- The Stochastic Trend process is not perfectliy predictable.
- The process $\{X_t\}$ is a **Simple Random Walk** with white-noise steps. It is non-stationary because given X_0:
  - $Var(X_t) = t\sigma^2$
  - $Cov(X_t, X_{t-j}) = (t-j)\sigma^2$ for $0 < j < t$.
  - $Corr = (X_t, X_{t-j}) = \sqrt{t-j}/\sqrt{t} = \sqrt{1 - j/t}$

ARIMA(p,d,q) Models

Definition: The time series $\{X_t\}$ follows an $ARIMA(p, d, q)$ model ("Integrated ARMA") if $\{\Delta^d X_t\}$ is stationary (and non-stationary for lower-order differencing) and follows an $ARMA(p, q)$ model.

Issues:

- Determining the order of differencing required to remove time trends (deterministic or stochastic).
- Estimating the unknown parameters of an $ARIMA(p, d, q)$ model.
- Model Selection: choosing among alternative models with different (p, d, q) specifications.

Outline

1. Time Series Analysis
   - Stationarity and Wold Representation Theorem
   - Autoregressive and Moving Average (ARMA) Models
   - Accommodating Non-Stationarity: ARIMA Models
   - Estimation of Stationary ARMA Models
   - Tests for Stationarity/Non-Stationarity

Estimation of ARMA Models

Maximum-Likelihood Estimation

- Assume that $\{\eta_t\}$ are i.i.d. $N(0, \sigma^2)$ r.v.'s.
- Express the $ARMA(p, q)$ model in state-space form.
- Apply the prediction-error decomposition of the log-likelihood function.
- Apply either or both of

Limited Information Maximum-Likelihood (LIML) Method

- Condition on the first p values of $\{X_t\}$
- Assume that the first q values of $\{\eta_t\}$ are zero.

Full Information Maximum-Likelihood (FIML) Method

- Use the stationary distribution of the first p values to specify the exact likelihood.

Model Selection

Statistical model selection critera are used to select the orders (p, q) of an ARMA process:

- Fit all $ARMA(p, q)$ models with $0 \leq p \leq p_{max}$ and $0 \leq q \leq q_{max}$, for chosen values of maximal orders.
- Let $\tilde{\sigma}^2(p, q)$ be the MLE of $\sigma^2 = Var(\eta_t)$, the variance of ARMA innovations under Gaussian/Normal assumption.
- Choose (p, q) to minimize one of:

  Akaike Information Criterion

  $$AIC(p, q) = log(\tilde{\sigma}^2(p, q)) + 2\frac{p+q}{n}$$

  Bayes Information Criterion

  $$BIC(p, q) = log(\tilde{\sigma}^2(p, q)) + log(n)\frac{p+q}{n}$$

  Hannan-Quinn Criterion

  $$HQ(p, q) = log(\tilde{\sigma}^2(p, q)) + 2log(log(n))\frac{p+q}{n}$$

Outline

1. Time Series Analysis
   - Stationarity and Wold Representation Theorem
   - Autoregressive and Moving Average (ARMA) Models
   - Accommodating Non-Stationarity: ARIMA Models
   - Estimation of Stationary ARMA Models
   - Tests for Stationarity/Non-Stationarity

Testing for Stationarity/Non-Stationarity

Dickey-Fuller (DF) Test : Suppose $\{X_t\}$ follows the $AR(1)$ model

$$X_t = \phi X_{t-1} + \eta_t, \text{ with } \{\eta_t\} \text{ a } WN(0, \sigma^2).$$

Consider testing the following hypotheses:

H_0: $\phi = 1$ (unit root, non-stationarity)

H_1: $|\phi| < 1$ (stationarity)

("Autoregressive Unit Root Test")

- Fit the $AR(1)$ model by least squares and define the test statistic: $$t_{\phi=1} = \frac{\hat{\phi}-1}{se(\hat{\phi})}$$ where $\hat{\phi}$ is the least-squares estimate of ϕ and $se(\hat{\phi})$ is the least-squares estimate of the standard error of $\hat{\phi}$.
  - If $|\phi| < 1$, then $\sqrt{T}(\hat{\phi} - \phi) \xrightarrow{d} N(0, (1 - \phi^2))$.
  - If $\phi = 1$, then $\hat{\phi}$ is super-consistent with rate $(1/T)$, $\sqrt{T}t_{\phi=1}$ has DF distribution.

References on Tests for Stationarity/Non-Stationarity*

Unit Root Tests (H_0 : Nonstationarity)

- Dickey and Fuller (1979): Dickey-Fuller (DF) Test
- Said and Dickey (1984): Augmented Dickey-Fuller (ADF) Test
- Phillips and Perron (1988) Unit root (PP) tests
- Elliot, Rothenberg, and Stock (2001) Efficient unit root (ERS) test statistics.

Stationarity Tests (H_0 : stationarity)

- Kwiatkowski, Phillips, Schmidt, and Shin (1922): KPSS test.

* Optional reading

MIT OpenCourseWare
http://ocw.mit.edu

18.S096 Topics in Mathematics with Applications in Finance

Fall 2013

For information about citing these materials or our Terms of Use, visit: http://ocw.mit.edu/terms.

FINANCIAL ECONOMETRICS

MODULE 4 | LESSON 1

TIME SERIES AND AUTOCORRELATION

Reading Time	60 minutes
Prior Knowledge	Regression analysis, Basic statistics
Keywords	Time Series Data, Autocovariance, Autocorrelation (ACF), Partial Autocorrelation (PACF), Conditional Probability, Stationarity

In this module, we are going to talk about data points with a time stamp. A lot of our examples in the previous modules actually used this type of data. We call them time series. Time series analysis is a very important area of statistical analysis in finance, and there are unique challenges when attempting to perform statistical inference. Hence, several analytical methods have been developed that focus on time series data. In the next few modules, we will be going through different topics in time series analysis. In this lesson, we will introduce what time series data is and some challenges we may encounter when handling time series data. We will use Google's stock price, the U.S. Dollar Index daily return, and U.S. 10-year Treasury bond yield from the beginning of 2016 to the end of 2021 as examples to demonstrate various concepts.

In [1]:
```
# Load packages
import matplotlib.pyplot as plt
import pandas as pd
import statsmodels.api as sm

plt.rcParams["figure.figsize"] = (16, 9)  # Figure size and width
```

In [2]:
```
# Download datasets
m4_data = pd.read_csv("M4. goog_eur_10.csv")
dxyr_data = pd.read_csv("M4. dxy_r_data.csv")

# Convert date variable to date format
m4_data["Date2"] = pd.to_datetime(m4_data["Date"], format="%m/%d/%Y")
dxyr_data["Date2"] = pd.to_datetime(dxyr_data["Date"], format="%m/%d/%Y")

# Selecting columns and setting index
goog = m4_data.loc[:, ["Date2", "GOOGLE"]].set_index("Date2")
ust10 = m4_data.loc[:, ["Date2", "UST10Y"]].set_index("Date2")
dxy = dxyr_data[["Date2", "DXY_R"]].set_index("Date2")
```

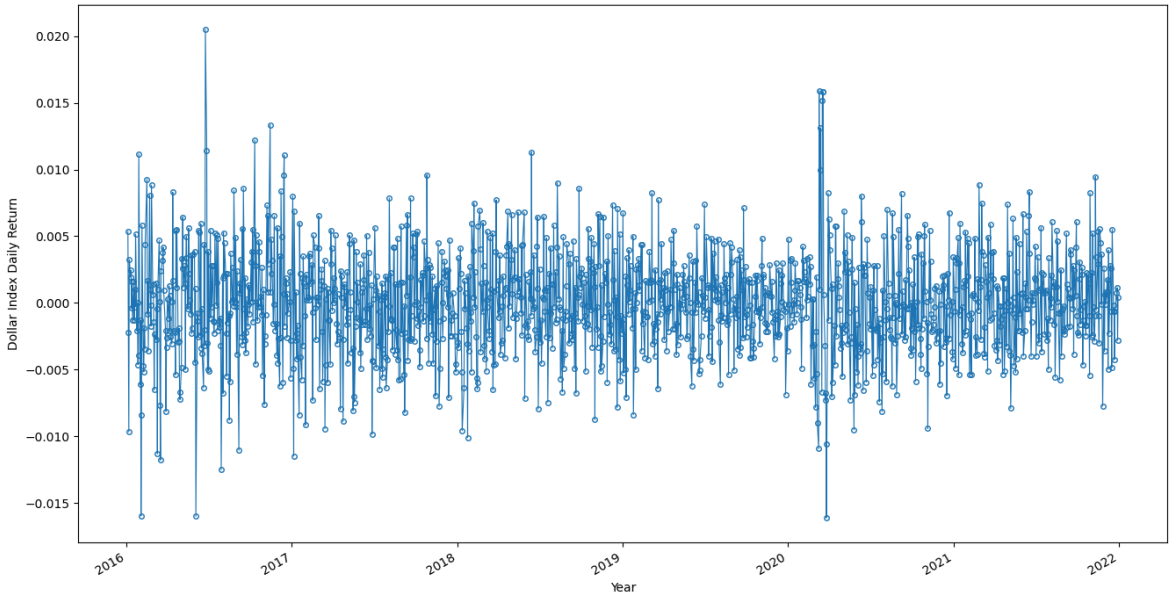
1. Time Series Data Overview

1.1 Time Series Data Examples and Some Characteristics

Time series data is a series of data points with date or time stamps attached to them. Let's look at some time series data examples in the following graphs and then we will formally define time series.

Figure 1: Dollar Index Daily Return Time Series Daily Chart

In [3]:
```
# Plot Dollar index time series chart
dxy["DXY_R"].plot(
    marker="o",
    markersize=4,
    markerfacecolor="none",
    linestyle="-",
    linewidth=0.8,
    xlabel="Year",
    ylabel="Dollar Index Daily Return",
)
plt.show()
```

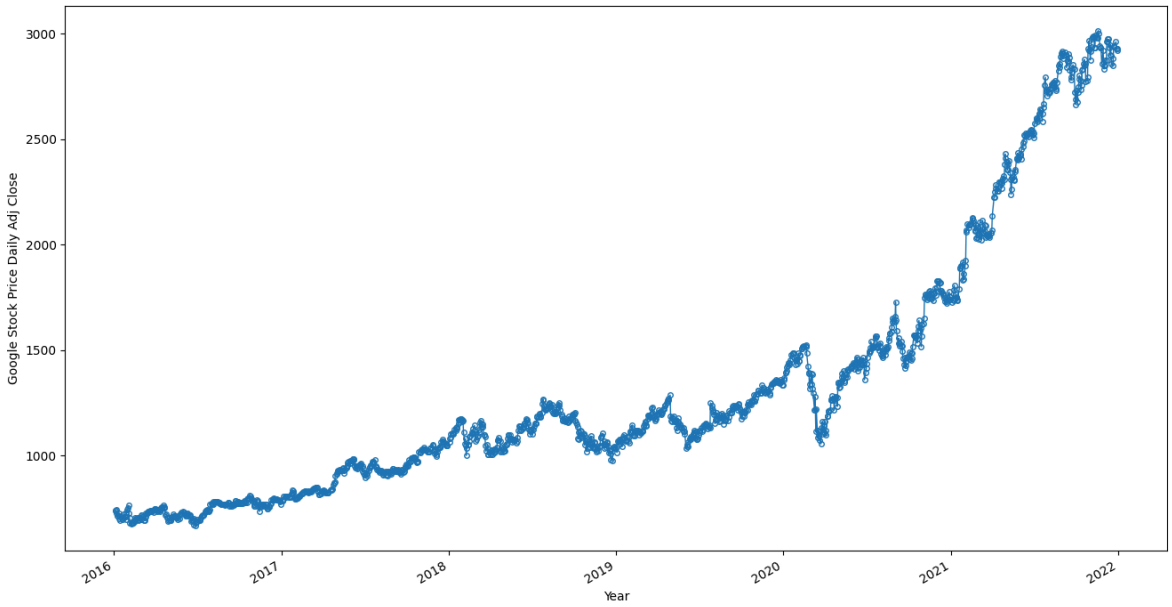


In figure 1, we can see the dollar index daily return moves around 0. It is common that a stock return usually oscillates around 0. On the other hand, the volatility of the return changes in different time periods. The year 2016 on average seems to have higher volatility than other years in the graph. The high volatility in 2020 appeared around the start of the COVID-19 pandemic. We can see that there are different volatility clusters in different years. 2016 has high volatility while the beginning of 2020 before COVID-19 witnessed a period of low volatility.

Figure 2: Google Stock Price Daily Close Time Series Daily Chart

In [4]:
```
# Plot Google price time series chart
goog["GOOGLE"].plot(
    marker="o",
    markersize=4,
    markerfacecolor="none",
    linestyle="-",
```

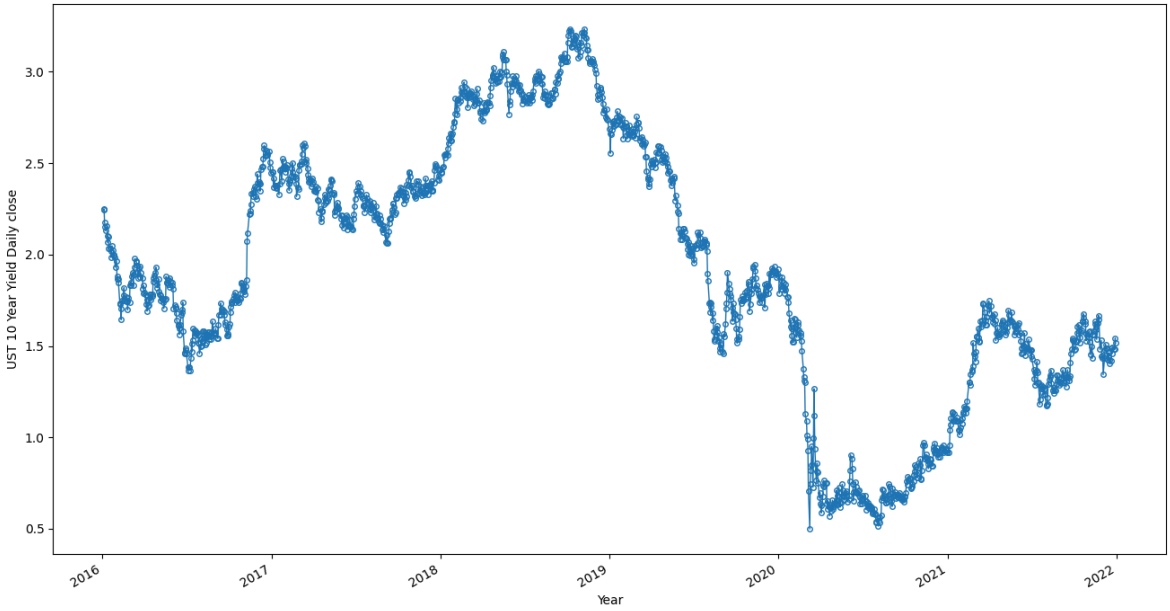
```
    linewidth=1,
    xlabel="Year",
    ylabel="Google Stock Price Daily Adj Close",
)
plt.show()
```



In figure 2, the graph of Google's stock price shows how the stock price has been moving during the period. Although the trend is up, we can also see there is a regular cyclic movement along the uptrend. We will learn how to analyze this type of time series data later.

Figure 3: U.S. 10-Year Treasury Bond Yield Time Series Daily Chart

In [5]:
```
# Plot UST10Y time series chart
ust10["UST10Y"].plot(
    marker="o",
    markersize=4,
    markerfacecolor="none",
    linestyle="-",
    linewidth=1,
    xlabel="Year",
    ylabel="UST 10 Year Yield Daily close",
)
plt.show()
```



In figure 3, there doesn't seem to be a clear pattern for the U.S. 10-year Treasury Bond yield. However, we can see that if the yield in one day is going down, it would keep going down the next day. It looks like the yield movement from the previous few days can predict where the yield would go today.

From the last three figures we have noticed several patterns:

> a. trend
> b. cyclical movement (seasonality)
> c. volatility clusters (volatilities can be different during different time periods)
> d. correlation between observations
> e. extreme values or outliers

These are some characteristics of time series data that we will address later in the course.

Since we would like to apply statistics to analyze time series data, we need to define time series data in a mathematical way.

2.2 Time Series Definition

The data series $\{x_1, x_2, \cdots, x_{t-1}, x_t\}$ is called a time series if

> a. t is an ordered time stamp (date, hour, year, etc.)
> b. x_t is from a random variable X_t

Let's use the following table to explain this definition.

Figure 4: Time Series Definition Example

Date	Price: Observed Data	Associated Random Variable
day 1	x_1	X_1
day 2	x_2	X_2
day 3	x_3	X_3
day 4	x_4	X_4
day 5	x_5	X_5

In figure 4, we assume this is the price data for Stock A for five days. We have the time series of Stock A's price for five days $\{x_1, x_2, x_3, x_4, x_5\}$. The price x_1 is from a random variable X_1 and the price x_2 is from another random variable X_2. Each price value at time t is a realization from its random variable at time t. In this course, we will limit t to be discrete numbers and X_t to be a continuous random variable.

For a time series, we already know that data observations do not necessarily need to be independent. Another thing to note is they also do not need to be identically distributed. What is important for a time series is that the data needs to be ordered by time. Many time series models provide results based on the order.

From now on, we will use an uppercase letter and $\{X_t\}$ to denote a time series that contains the data observations. We will use a lowercase letter and $\{x_t\}$ to denote a data observation from a time series. With this clarification, we will not have any confusion in the later course.

In our example above, the time series can be written as:

$$\{X_5\} = \{x_1, x_2, x_3, x_4, x_5\}$$

In general, the time series $\{X_t\}$ can be written as:

$$\{X_t\} = \{x_t, x_{t-1}, x_{t-2}, x_{t-3}, \cdots, x_1, x_0\}$$

For x_t and x_{t-1}, the observations are separate by one time unit. In general, we call this one **lag**. For x_t and x_{t-k}, the two observations are separated by k lags. The following table demonstrates how to express lag in our example.

Figure 5: Lag Expressions for Time Series

Time Series	Data Points				
X_5	x_5	x_4	x_3	x_2	x_1
$X_{(5-1)} = X_4$	x_4	x_3	x_2	x_1	
$X_{(5-2)} = X_3$	x_3	x_2	x_1		

We can see from figure 5, $\{X_4\}$ is lag 1 of $\{X_5\}$ and $\{X_3\}$ is lag 2 of $\{X_5\}$.

We can do time series difference calculations among the above time series. The following table shows a few examples.

Figure 6: Time Series Difference Calculation Example

Time Series Difference Calculation	Data Points			
$X_5 - X_4$	$x_5 - x_4$	$x_4 - x_3$	$x_3 - x_2$	$x_2 - x_1$
$X_5 - X_3$	$x_5 - x_3$	$x_4 - x_2$	$x_3 - x_1$	

In figure 6, it shows the calculation of one difference and two differences for time series $\{X_5\}$. One difference means the current time series minus the lag 1 time series. Two difference means the current time series minus the lag 2 time series. We notice that when we conduct one difference calculation for time series, we will lose one observation as indicated in the above table. We will lose 2 observations when we conduct two differences calculation. We will use this calculation a lot going forward.

3. Autocorrelation

In Module 2 Lesson 1, we discussed how one of the assumptions for OLS regression is that each observation should be independent of each other. If the data violates this assumption, this data series has **autocorrelation**. In our last section, we saw U.S. 10-Year Treasury bond yields have an autocorrelation issue because today's bond yield is likely to go down if the yields over the past few days have been inclining. The issue of correlated observations is one challenge in analyzing time series data. We cannot apply time series data to OLS properly without addressing the issue of autocorrelation. So how do we analyze the autocorrelation for a time series? We will introduce the autocorrelation function as a measure in this section. Before getting into autocorrelation function, we would like to introduce some concepts here.

3.1 The Mean Function of Time Series

First, let's look at the mean function for time series:

$$\mu_{X_t} = E(X_t) = \int_a^b x f_t(x) dx$$

Where f_t is the marginal density function for X_t and a and b are bounds for random variable X_t

The mean function is a measure to describe the center of the time series. It will play a key part in the later time series theories.

3.2 Autocovariance Function

The first metric we can use to study the strength of the relationship between two observations in a time series is **autocovariance function**. Autocovariance function measures the linear dependence of two observations. Autocovariance function plays a similar role to covariance for two data series, except that autocovariance function assesses the relationship among observations in one data series.

Assume X_t has finite variance where $t = 1, 2, \cdots, T$, autocovariance function is defined as:

$$\gamma_X(s,t) = cov(X_s, X_t) = E[(X_s - \mu_{X_s})(X_t - \mu_{X_t})]$$

for all s and t

Here are some properties for autocovariance function:

> a. It only measures the linear relationship between X_s and X_t
> b. $\gamma_X(s,t) = \gamma_X(t,s)$
> c. When $\gamma_X(s,t)$ is close to 0 and the distance between s and t is large, the time series graph would exhibit a choppy curve between s and t
> d. When the absolute value of $\gamma_X(s,t)$ is large and the distance between s and t is large, the time series graph would exhibit a smooth curve between s and t
> e. When $s = t$, the autocovariance becomes variance of X_t

3.3 Autocorrelation

Now we can introduce the autocorrelation function (ACF). Its definition is:

$$\rho_X(s,t) = \frac{\gamma_X(s,t)}{\sqrt{\gamma_X(s,s)\gamma_X(t,t)}}$$

Here are some properties for ACF:

> a. It only measures the linear relationship between X_s and X_t
> b. The value of $\rho_X(s,t)$ is between -1 and 1

By using autocorrelation, we can see if the current observation has any significant correlation with observations in different time lags.

4. Conditional Probability

Before we move to the next time series topic, let's review the concept of conditional probability. Conditional probability is an important concept incorporated in the topic we will discuss next.

Let's use an example to explain conditional probability. Say we have a cookie jar with 20 cookies. Using the cookies' ingredients, the distribution of cookies is as follows:

Figure 7. Cookie Distribution by Ingredients

	Dark Chocolate	Milk Chocolate	White Chocolate
With Nuts	6	4	2
No Nuts	3	4	1

Now, given that we get a cookie with dark chocolate, what is the probability that this cookie contains nuts? This is a conditional probability question. In mathematic form, we can write $P(\text{cookie has nuts} \mid \text{cookie has dark chocolate})$. The "|" stands for "given." Instead of trying to get the answer by checking all 20 cookies, we only need to focus on nine cookies with dark chocolate now. Among these nine cookies, there are six cookies that have nuts. Hence, $P(\text{cookie has nuts} \mid \text{cookie has dark chocolate}) = 6/9$. We can see the above math operation reduces the sample space from 20 cookies to nine cookies for conditional probability.

In the formal math form, assuming we know event A and event B are all in the same sample space, the **conditional probability** that event A will happen given that event B happens is:

$$P(A|B)$$

Please review the required reading on conditional probability to learn its properties and its relationship with joint probability and marginal probability.

5. Partial Autocorrelation

In the previous section, we talked about using autocorrelation to evaluate the relationship of two observations in a time series. Say if we have x_t and x_{t-3} from a time series, we can write their autocorrelation as follows:

$$\rho_X(t, t-3) = \frac{cov(x_t, x_{t-3})}{\sqrt{var(x_t)var(x_{t-3})}}$$

The above autocorrelation is to assess the relationship between x_t and x_{t-3} including the correlation explained by x_{t-1} and x_{t-2}. However, let's assume we want to evaluate the pure correlation between x_t and x_{t-3} stripped out correlation from x_{t-1} and x_{t-2}. In this case, we would need to use partial autocorrelation (PACF). Partial autocorrelation can be written as follows:

$$\phi_{_X}(t, t-3 \mid t-1, t-2) = \frac{cov(x_t, x_{t-3}|x_{t-1}, x_{t-2})}{\sqrt{var(x_t|x_{t-1}, x_{t-2})var(x_{t-3}|x_{t-1}, x_{t-2})}}$$

As you can see, partial autocorrelation is a conditional correlation. Unlike autocorrelation, partial correlation is giving the information about the pure correlation between x_t and x_{t-3} that x_{t-1} and x_{t-2} do not explain. For PACF notation, we can drop the condition part and simply write the notation as follows:

$$\phi_{_X}(t, t-3)$$

Both ACF and PACF are very useful when we talk about time series regression analysis later.

6. Stationarity

We introduced time series and discussed the mean and autocorrelation of time series. We have not yet discussed what kinds of behaviors or assumptions about time series make it possible to model them. In this section, we are going to introduce a concept called **stationarity**.

Let $\{X_t\}$ be a time series with finite variance, $\{X_t\}$ is stationary if

> a. the mean function $\mu_{_X}(t) = E(X_t)$ is constant and independent of t
> b. Autocovariance $\gamma_{_X}(t-h, t)$ is independent of t for each h.

This means autocovariance is only dependent on time difference h. For example, $Cov(X_t, X_{t+h}) = Cov(X_s, X_{s+h})$.

It also means when $h = 0$, autocovariance is independent of t. Since autocovariance is variance when $h = 0$, variance is constant. We therefore rule out the issue of heteroskedasticity in a stationary time series. The above stationarity definition is also called **weak stationarity**. There is also strict stationarity.

For time series $\{X_t\}$ to be **strictly stationary**, every finite joint probability distribution of $\{X_t\} : \{x_{t_1}, x_{t_2}, \cdots, x_{t_k}\}$ needs to have the identical probabilistic behavior of every collective value of $\{x_{t_1+h}, x_{t_2+h}, \cdots, x_{t_k+h}\}$. It means the probabilistic behavior of a time series will not change when the time series shifts time. We can also express strict stationarity in the following way.

$$P\{x_{t_1} \leq c_1, \cdots, x_{t_k} \leq c_k\} = P\{x_{t_1+h} \leq c_1, \cdots, x_{t_k+h} \leq c_k\}$$

for all $k = 1, 2, \cdots$ all time stamps $t_1, t_2, \ldots, t_k$, all numbers $c_1, c_2, \cdots, c_k$ and all time shifts h. It means the above joint cumulative probabilities have the same cumulative

functions.

It is clear that strict stationarity implies weak stationarity. The converse is not true. However, strict stationarity is a much harder requirement and much more difficult to check. Thankfully, most statistical procedures involving time series only require weak stationarity.

Because of the definition of strict stationary, both mean and covariances of a strictly stationary time series are independent of time.

Since the mean of $\{X_t\}$ is independent of t and is constant, we can write it as follows:

$$\mu_t = \mu$$

From (b), we know autocovariance for a stationary time series only depends on the distance between two time points, and we can write autocorrelation as follows:

$$\gamma_X(t-h,t) = \gamma_X(h)$$

Since autocovariance is independent of t, both ACF and PACF are independent of t, too. Hence, we can also write ACF as $\rho_X(h)$ and PACF as $\phi_X(h)$ or ϕ_{hh}.

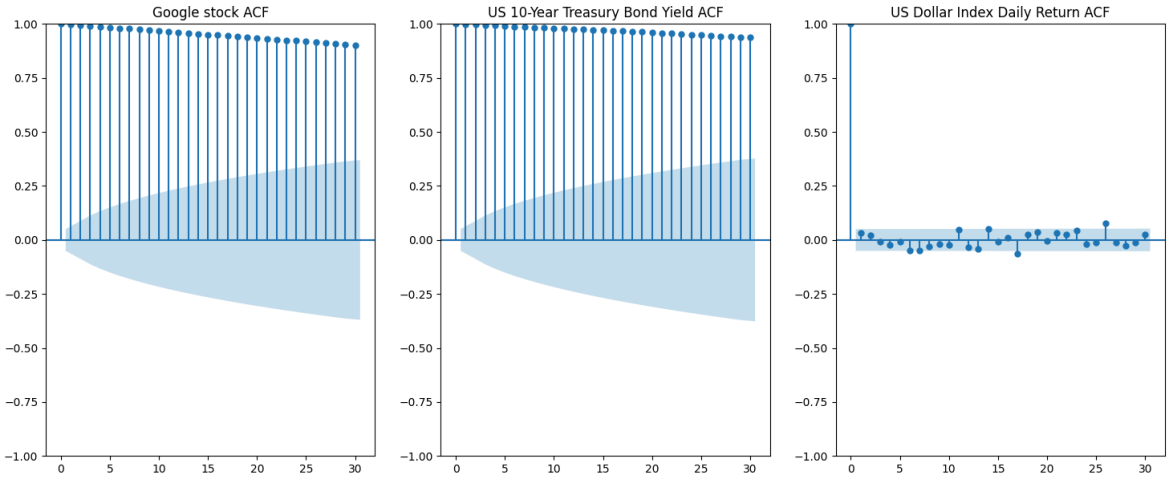
In the following time series analysis, we will introduce several models that require time series to be stationary before running the model. Hence, it is important to make sure the time series is stationary before conducting any time series analysis. Later in this course, we will introduce methods to check if a time series is stationary.

7. Plots of ACF and PACF

We introduced ACF and PACF in the last section. It is usually easier to use a plot to visualize information from ACF and PACF. Let's use our Google stock price, U.S. 10-year Treasury bond yield, and dollar index daily return as examples to show ACF and PACF plots.

Figure 8: ACF Plots for Google Stock Price, U.S. Treasury 10-Year Bond Yield and U.S. Dollar Index Daily Return

In [6]:
```
# ACF Plots for GOOGLE, U.S. UST10Y and DXY_R
fig, ax = plt.subplots(1, 3, figsize=(18, 7))
sm.graphics.tsa.plot_acf(goog["GOOGLE"], title="Google stock ACF", lags=30, ax=a
sm.graphics.tsa.plot_acf(
    ust10["UST10Y"], title="US 10-Year Treasury Bond Yield ACF", lags=30, ax=ax[
)
sm.graphics.tsa.plot_acf(
    dxy["DXY_R"], title="US Dollar Index Daily Return ACF", lags=30, ax=ax[2]
)
plt.show()
```



An ACF plot is a plot that shows a bar as the autocorrelation of a time series with different lags. You have 0 lag starting on the left end of the plot. Since autocorrelation with 0 lag is just correlation to itself, the number is 1. Then, you have autocorrelation with lag 1, lag 2, and so on and so forth. In the plot, we also have 95% confidence intervals as shaded area around 0. If a bar is over the confidence interval, it means this autocorrelation is statistically significant.

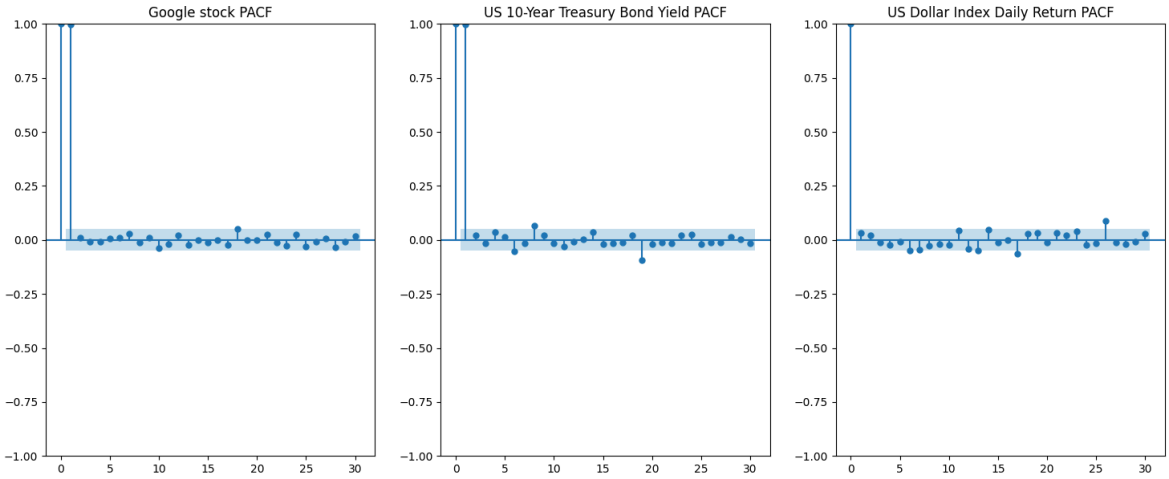
In figure 8, we can see that Google stock and U.S. Treasury 10-year bond yield have high and slowly declining autocorrelations, which usually means there is some sort of trend in the time series. It also means these time series are not stationary. We will usually proceed to detrend the time series to make it stationary before modeling. We will show how to detrend for a time series in the next lesson for the U.S. dollar index daily return.

We see from ACF for the U.S. dollar index daily return, lag 17 and lag 26 have significant autocorrelations. This piece of information will help us later to specify a time series model.

Now let's look at PACF plots for Google stock price, U.S. 10-year Treasury bond yield, and dollar index daily return.

Figure 9: PACF Plots for Google Stock Price, U.S. Treasury 10-Year Bond Yield, and U.S. Dollar Index Daily Return

In [7]:
```
# PACF plots for GOOGLE, U.S. UST10Y and DXY_R
fig, ax = plt.subplots(1, 3, figsize=(18, 7))
sm.graphics.tsa.plot_pacf(goog["GOOGLE"], title="Google stock PACF", lags=30, ax
sm.graphics.tsa.plot_pacf(
    ust10["UST10Y"], title="US 10-Year Treasury Bond Yield PACF", lags=30, ax=ax
)
sm.graphics.tsa.plot_pacf(
    dxy["DXY_R"], title="US Dollar Index Daily Return PACF", lags=30, ax=ax[2]
)
plt.show()
```



You read PACF plots as you read ACF plots except the bar now represents PACF. From figure 9, we see that for Google stock PACF lag18 is positive and significant. For U.S. Treasury 10-year bond yield, PACF lag8 and lag19 are significant. PACF lag 19 has a negative PACF. For U.S. dollar index daily return, PACF lag 16 is negative and significant. PACF lag 26 is positive and significant.

ACF and PACF plots will be applied frequently in the following lessons. They are important tools for providing information about a time series that can help us to specify a time series model.

8. Conclusion

The module introduced time series analysis. In this lesson, we started by observing a few time series graphs to give an idea of what time series are about. We then formally defined time series and showed some math operations on time series. Then, we introduced autocorrelation and partial autocorrelation of time series. Finally, we discussed what it means for a time series to be stationary. We completed the lesson with ACF and PACF plots and how to read the plots. In the next lesson, we will start to learn some basic time series statistical models.

Copyright 2025 WorldQuant University. This content is licensed solely for personal use. Redistribution or publication of this material is strictly prohibited.

FINANCIAL ECONOMETRICS

MODULE 4 | LESSON 2

TIME SERIES STATISTICAL MODELS

Reading Time	60 minutes
Prior Knowledge	Regression analysis, Basic statistics
Keywords	First difference, Backshift operator, White noise, Random walk, Random walk with a drift, Moving average

In this lesson, first, we will proceed from the previous lesson to introduce some methods to remove trends from a non-stationary time series. After that, we are going to introduce some time series models. They are foundational time series models that will help readers understand more complicated time series models to be introduced in later lessons and modules. In particular, we will cover white noise, random walk, random walk with a drift and moving average models. We will calculate their means and variances as wells as ACF and PACF. We will also check their stationarity.

In [1]:
```
# Load libraries
import matplotlib.pyplot as plt
import numpy as np
import pandas as pd
import statsmodels
import statsmodels.api as sm
from statsmodels.tsa.arima_process import arma_generate_sample

plt.rcParams["figure.figsize"] = (16, 9)  # Figure size and width
```

In [2]:
```
# Download the dataset
m4_data = pd.read_csv("M4. goog_eur_10.csv")
dxyr_data = pd.read_csv("M4. dxy_r_data.csv")

# Convert date variable to date format
m4_data["Date2"] = pd.to_datetime(m4_data["Date"], format="%m/%d/%Y")
dxyr_data["Date2"] = pd.to_datetime(dxyr_data["Date"], format="%m/%d/%Y")

# Selecting columns
goog = m4_data.loc[:, ["Date2", "GOOGLE"]].set_index("Date2")
```

1. Remove Trend from Non-Stationary Time Series

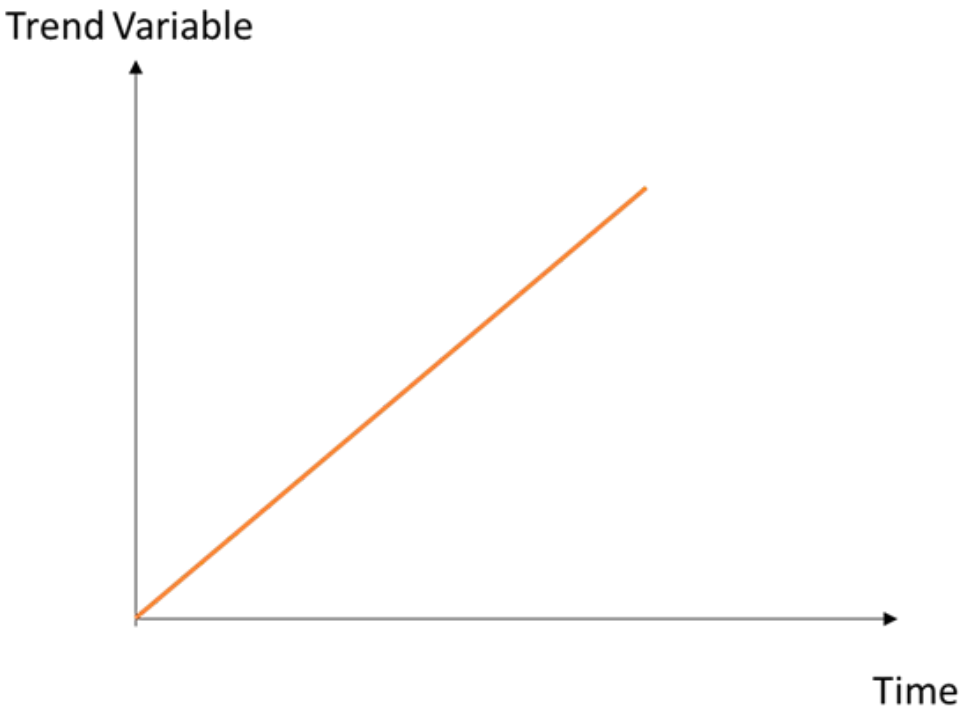
In the last lesson, we saw that one of the key points of time series analysis is to understand the relationship of data observations in different time points for a time series. We introduced ACF and PACF that are key metrics to understanding the relationship among the observations in a time series. We also introduced stationary time series. A stationary time series means that its mean and autocovariance are independent of absolute time location of data observations. For example, $Cov(X_t, X_s)$ must depend only on the difference $t - s$. The estimation of ACF and PACF will be accurate and stable without the worry of which section of data we choose to estimate. Hence, it is critical to obtain a stationary time series before doing modeling work.

Removing trends from a non-stationary time series is usually the first step to handling the time series data. In this section, we will go through two common methods to remove a trend: using a trend variable or differencing. We will use Google stock as an example to illustrate the process.

1.1 Remove Trend with a Trend Variable

The first method to remove trend from a non-stationary time series is to model the variable with a linear trend variable. Figure 1 shows the concept of a linear trend variable, a straight line against time in the graph.

Figure 1: A Trend Variable



Now, we can run a simple regression model for Google stock price as follows:

$$\text{Google stock price} = \text{intercept} + \beta \cdot \text{trend}$$

Figure 2 below shows the model result.

Figure 2: Regression Model for Google Stock Price and Trend

In [3]:
```
# Detrend
timeTrend = np.linspace(1, len(goog.GOOGLE), len(goog.GOOGLE))
timeTrend = sm.add_constant(timeTrend)

# Fit OLS
model = sm.OLS(goog.GOOGLE, timeTrend)
fit_g = model.fit()
fit_g.summary()
```

Out[3]:

OLS Regression Results

Dep. Variable:	GOOGLE	**R-squared:**	0.767
Model:	OLS	**Adj. R-squared:**	0.767
Method:	Least Squares	**F-statistic:**	4956.
Date:	Sat, 06 Dec 2025	**Prob (F-statistic):**	0.00
Time:	14:31:25	**Log-Likelihood:**	-10671.
No. Observations:	1505	**AIC:**	2.135e+04
Df Residuals:	1503	**BIC:**	2.136e+04
Df Model:	1		
Covariance Type:	nonrobust		

	coef	std err	t	P>|t|	[0.025	0.975]
const	413.4033	14.988	27.583	0.000	384.004	442.802
x1	1.2136	0.017	70.396	0.000	1.180	1.247

Omnibus:	101.043	**Durbin-Watson:**	0.006
Prob(Omnibus):	0.000	**Jarque-Bera (JB):**	120.227
Skew:	0.678	**Prob(JB):**	7.82e-27
Kurtosis:	3.277	**Cond. No.**	1.74e+03

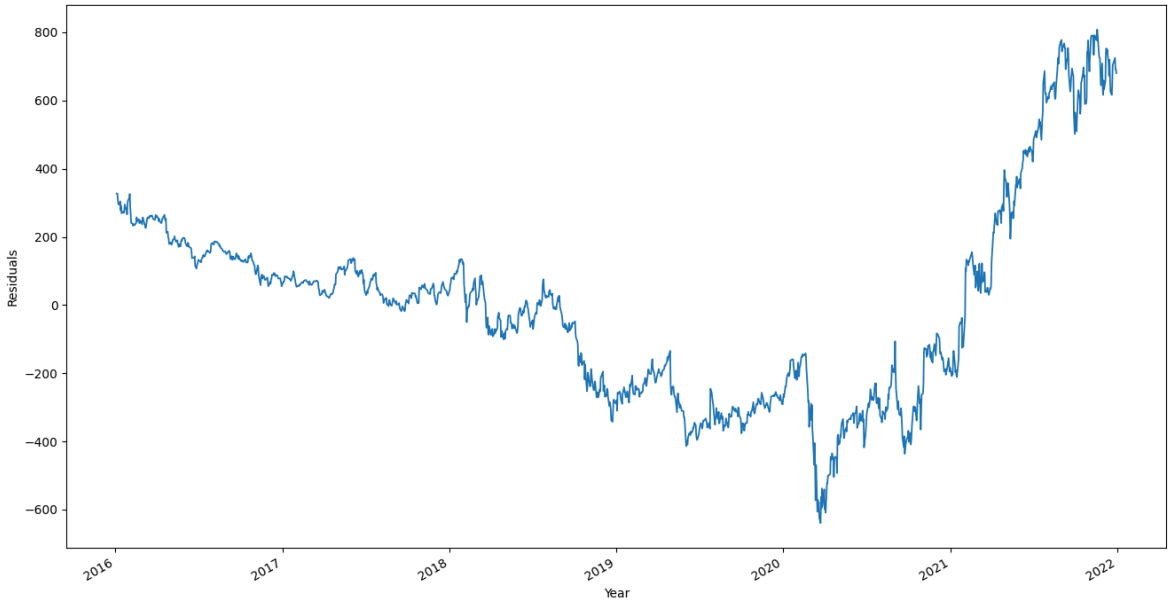
Notes:
[1] Standard Errors assume that the covariance matrix of the errors is correctly specified.
[2] The condition number is large, 1.74e+03. This might indicate that there are strong multicollinearity or other numerical problems.

Next, let's plot the residuals from the model against time in the following graph.

Figure 3: Residuals from Simple Regression of Google Stock Price on Trend

In [4]:
```
# Plot residuals
goog_res = fit_g.resid
```

```
goog_res.plot(linewidth=1.3, xlabel="Year", ylabel="Residuals")
plt.show()
```



From figure 3, we can see the residuals from the simple regression model with a trend variable does not appear to exhibit stationarity. The time series of residuals shows what resembles a "V pattern," and therefore, it does not appear that the mean is constant over time. Thus, the residuals from the regression do not form a stationary time series.

Adding a trend variable does not seem to be enough to make Google stock price into a stationary time series.

Let's look at the differencing method.

2.2 Removing Trend Using the Differencing Method

Another popular method to remove a trend in a time series is differencing. Before going into the differencing details, let's define some terms.

2.2.1 Notations

The first one we will introduce is first difference. First difference is defined as:

$$\nabla x_t = x_t - x_{t-1}$$

And second difference can be written as:

$$\nabla^2 x_t = \nabla(\nabla x_t) = \nabla(x_t - x_{t-1}) = (x_t - x_{t-1}) - (x_{t-1} - x_{t-2})$$

The next notation is backshift operator. The backshift operator is defined as:

$$Bx_t = x_{t-1}$$

And

$$B^2 x_t = B(Bx_t) = Bx_{t-1} = x_{t-2}$$

From above, we can write a general backshift operator as follows:

$$B^k x_t = x_{t-k}$$

With the backshift operator, we can also write the first difference with backshift operator:

$$\nabla x_t = x_t - x_{t-1} = x_t - Bx_t = (1 - B)x_t$$

Then, the second difference can be defined and written as follows:

$$\nabla^2 x_t = (1 - B)^2 x_t = (1 - 2B + B^2)x_t = x_t - 2x_{t-1} + x_{t-2}$$

And the difference of order d can be defined as:

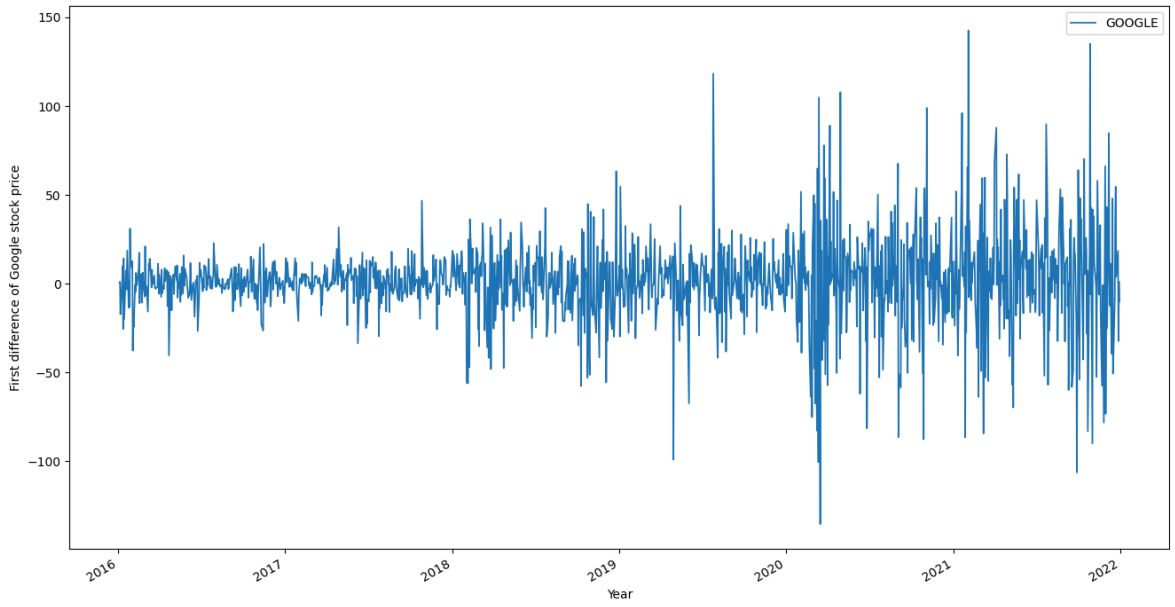
$$\nabla^d = (1 - B)^d$$

2.2.2. Differencing Method

The differencing method is used to generate first difference time series. Let's keep using Google stock price as an example. Figure 4 shows the first difference of Google stock price.

Figure 4: First Difference of Google Stock Price

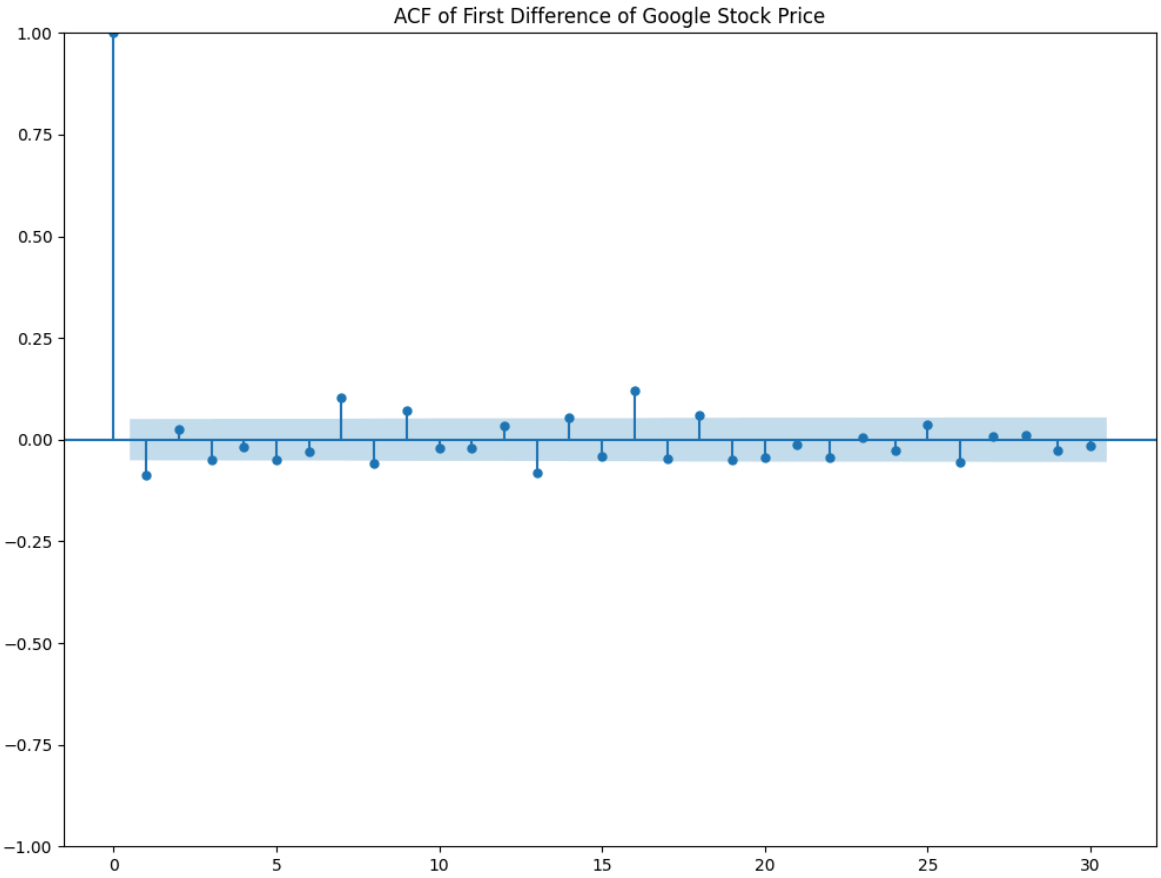
In [5]:
```
# Plot First Difference of Google Stock Price
goog.diff().plot(
    linewidth=1.3, xlabel="Year", ylabel="First difference of Google stock price
)
plt.show()
```



We can see that the first difference of Google stock price fluctuates around 0 during the whole period. However, the variation of the first difference increases as time goes on. It means that variance is dependent on time. Hence, the first differencing of Google stock price is still not enough to make it stationary. Let's look at its ACF.

Figure 5: ACF of First Difference of Google Stock Price

In [6]:
```
# ACF of First Difference of Google Stock Price
# (Note: [1:] as we need to skip first element since the model lost one data poi
fig, ax = plt.subplots(figsize=(12, 9))
sm.graphics.tsa.plot_acf(
    goog.diff()[1:],
    title="ACF of First Difference of Google Stock Price",
    lags=30,
    ax=ax,
)
plt.show()
```



From figure 5, we can see there are a few lags before lag 16 that are significant at the 95% level. We can use this piece of information for later modeling.

We used both the trend variable and differencing methods to remove the trend for Google stock price time series. Unfortunately, neither method did a good job of converting the time series to a stationary one. In the following sections, we will learn more models to see if they would be a better model choice for Google stock price.

3. White Noise

White noise is a special time series model. The white noise time series is generated from a collection of uncorrelated random variables. We can define white noise time series W_t as follows:

$$W_t \sim WN(0, \sigma^2)$$

Its autocovariance is given by:

$$\gamma_w(h) = \begin{cases} \sigma_2 & \text{for } h = 0 \\ 0 & \text{for } |h| > 0 \end{cases}$$

Since both mean and autocovariance of white noise are independent of t, white noise is a stationary time series.

If the white noise random variables are also independent and identical, we can denote it as follows:

$$W_t \sim iid(0, \sigma^2)$$

If all the random variables are i.i.d. and normally distributed, this is a normal white noise time series and we can denote it as follows:

$$W_t \sim iidN(0, \sigma^2)$$

Let's produce one simulation of a white noise process (see figure 6).

Figure 6: Normal White Noise Time Series Example

In [7]:
```
# Normal white noise ts example
plt.plot(np.random.normal(0, 3, 300))
plt.title("Normal White Noise with Mean=0 and Variance=9")
plt.show()
```

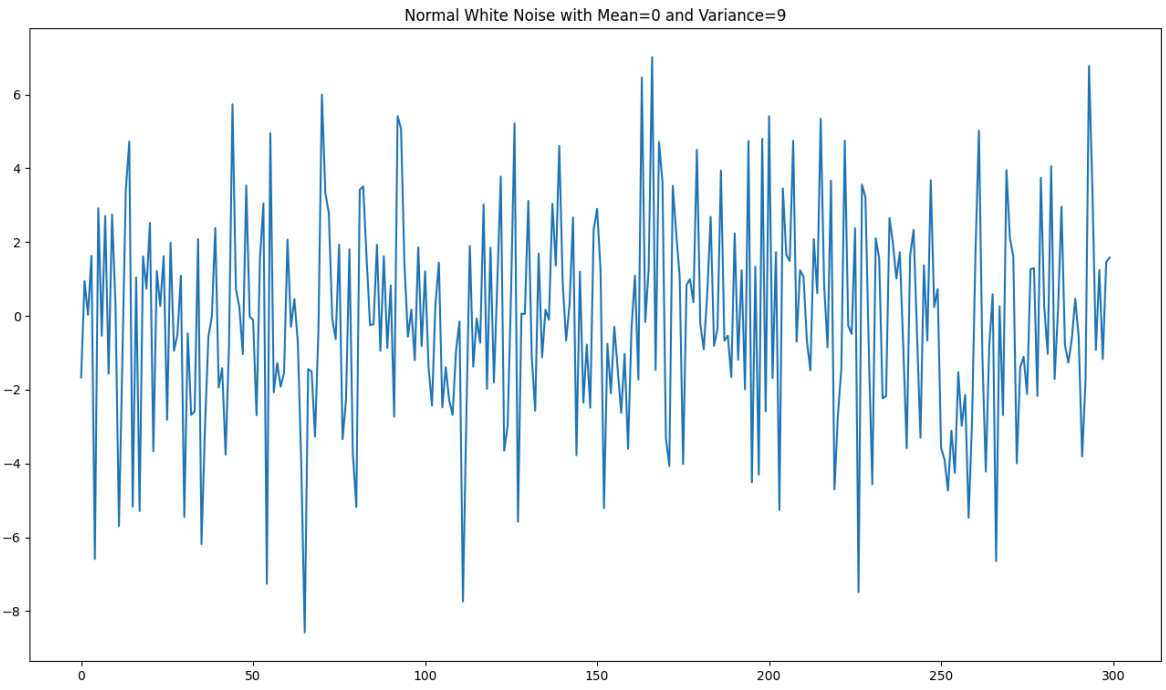


Figure 6 shows a white noise time series with mean = 0 and variance = 9. We can see the time series is pretty choppy, but it oscillates around 0.

Normal independent white noise is similar to the normal independent error terms in OLS regression. Usually, a time series can be broken down into a trend component, a seasonality component, and a residual component. If we can model the residual using a white noise model, then we will be able to model the whole time series and predict it. The random nature of the white noise makes it a good choice as a random term in the following time series models.

4. Random Walk

The next time series model we would like to talk about is the random walk time series. A random walk time series says that the current value depends only on the past value with a random component. We can write down the math definition as follows:

If $\{X_t\}$ is a random walk time series and $x_0 = 0$, then $X_t = X_{t-1} + W_t$ where $W_t \sim WN(0, \sigma^2)$

From the above definition, you can replace X_{t-1} with $X_{t-2} + W_{t-1}$ and continue the same replacement process until X is represented with W_i where $i = 1, \cdots, t$ and X_0. We call this a back substitution technique. After you do the process, you can replace the above definition with the following formula:

$$X_t = \sum_{i=1}^{t} W_i + X_0 = \sum_{i=1}^{t} W_i$$

The mean function:

$$E(X_t) = E(X_{t-1} + W_t) = E(\sum_{i=1}^{t} W_i + X_0) = X_0 = 0$$

From the mean function, we know it is not dependent on t but on the first time X_0. In this case, it is 0.

The autocovariance:

$$COV(X_{t+h}, X_t) = COV(X_t + W_{t+1} + W_{t+2} + \cdots + W_{t+h}, X_t) = Cov(X_t, X_t) = t\sigma^2$$

We can see the autocovariance is dependent on t. Hence, the random walk time series is not a stationary time series.

Figure 7: Random Walk Examples

In [8]:
```
# Four random walks

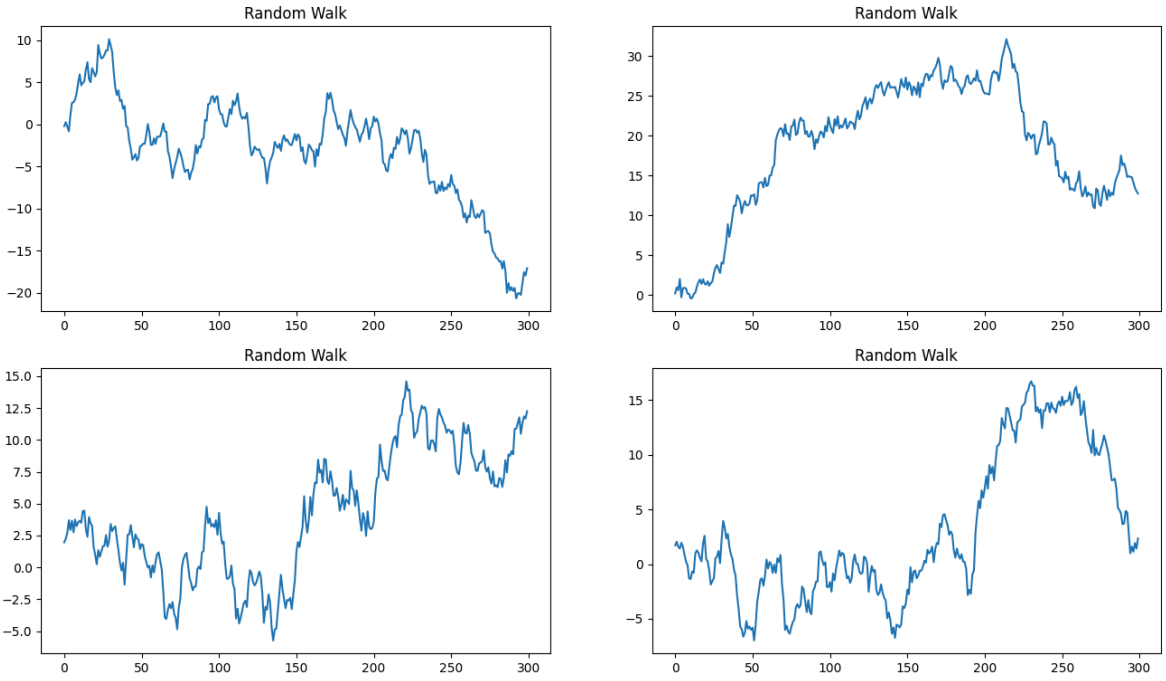

# Helper function
def RandWalkPlot(seed):
    np.random.seed(seed)
    wn = np.random.normal(0, 1, 300)
    rw = np.cumsum(wn)
    plt.plot(rw)
    plt.title("Random Walk")


# 4 random walk examples
ax1 = plt.subplot(221)
RandWalkPlot(12345)  # RW example 1

ax2 = plt.subplot(222)
RandWalkPlot(54321)  # RW example 2
```

```
ax3 = plt.subplot(223)
RandWalkPlot(222)  # RW example 3

ax4 = plt.subplot(224)
RandWalkPlot(333)  # RW example 4
plt.show()
```



In figure 7, we simulated four random walk time series of 300 data points with the same setup: normal white noise and $X_0 = 0$. Every time we generate a time series from the same random walk model, we'll get a very different random walk time series. There is no clear common pattern. We can also see random walk is not confined in a range as a stationary time series is.

One common random walk time series variation is **random walk with a drift**. Random walk with a drift is to add a constant term D to a random walk model. We can write random walk with a drift model as follows:

If $\{X_t\}$ is a random walk with a drift time series and $x_0 = 0$, then $X_t = D + X_{t-1} + W_t$ where $W_t \sim WN(0, \sigma^2)$ and D is a constant

We call D a drift. With the back substitution technique, we can rewrite the model as follows:

$$X_t = Dt + \sum_{i=1}^{t} W_i + X_0 = Dt + \sum_{i=1}^{t} W_i$$

The mean function:

$$E(X_t) = Dt + \sum_{i=1}^{t} E(W_i) = Dt$$

From the mean function, we see it is dependent on t.

The autocovariance:

$$COV(X_{t+h}, X_t) = COV(X_t + W_{t+1} + W_{t+2} + \cdots + W_{t+h}, X_t) = Cov(X_t, X_t) = t\sigma^2$$

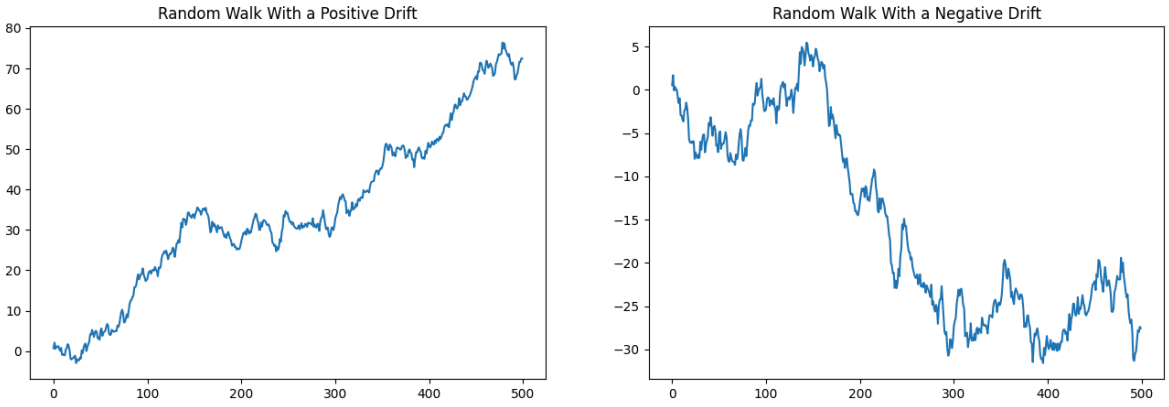
As the difference of random walk and random walk with a drift is only the addition of a constant drift, their autocovariance should be the same. We can see that random walk with a drift is not a stationary time series either. Both mean and autocovariance depend on t. Also because of the addition of drift, the mean is increasing as time increases. Hence, random walk with a drift is a choice to model real world time series that possess a trending pattern.

Figure 8: Random Walk with a Positive Drift and Random Walk with a Negative Drift Examples

In [9]:
```
# RW examples with drift
np.random.seed(99999)
wnd = np.random.normal(0, 1, 500)

plt.figure(figsize=(16, 5))
ax1 = plt.subplot(121)
plt.plot(np.cumsum(wnd + 0.1))
plt.title("Random Walk With a Positive Drift")

ax2 = plt.subplot(122)
plt.plot(np.cumsum(wnd - 0.1))
plt.title("Random Walk With a Negative Drift")
plt.show()
```



In figure 8, we generated one random walk with a positive drift time series and one random walk with a negative drift time series. We can see the positive drift time series has a clear upward trend as expected. The negative drift time series initially goes up then slowly moves down but the pattern is very choppy. From the above graphs, random walk with a drift can be a model choice for a time series with a trending pattern.

5. Moving Average

A moving average model tries to model a shock with impact in a finite time frame. If a shock in interest has an initial impact and the impact gradually fades in a finite time frame, the moving average model can be a model choice. A moving average with a shock lasting only for a total time range of q can be written as MA(q) and is defined as follows:

$$X_t = W_t + \theta_1 W_{t-1} + \theta_2 W_{t-2} + \cdots + \theta_q W_{t-q}$$

Where $\theta_1, \theta_2, \cdots, \theta_q$ are parameters and W_t is normally distributed white noise with mean = 0 and variance σ^2 is 1. There are q lags in the model.

We can also write the MA(q) model using backshift operator:

$$X_t = \theta(B) W_t$$

Where $\theta(B)$ is the **moving average operator** as follows:

$$\theta(B) = 1 + \theta_1 B + \theta_2 B^2 + \cdots + \theta_q B^q$$

The following plots show some simulated examples for moving average models.

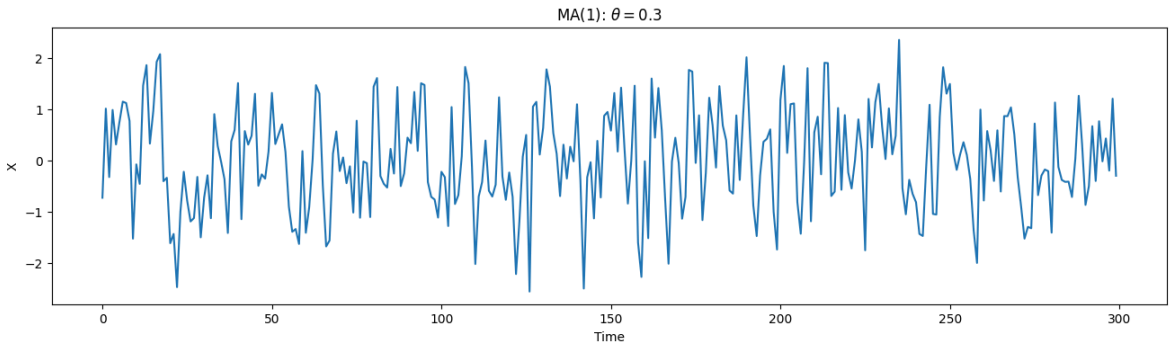
Figure 9: MA(1): $\theta = 0.3$ (Top) and MA(2): $\theta = -0.7$ (Bottom)

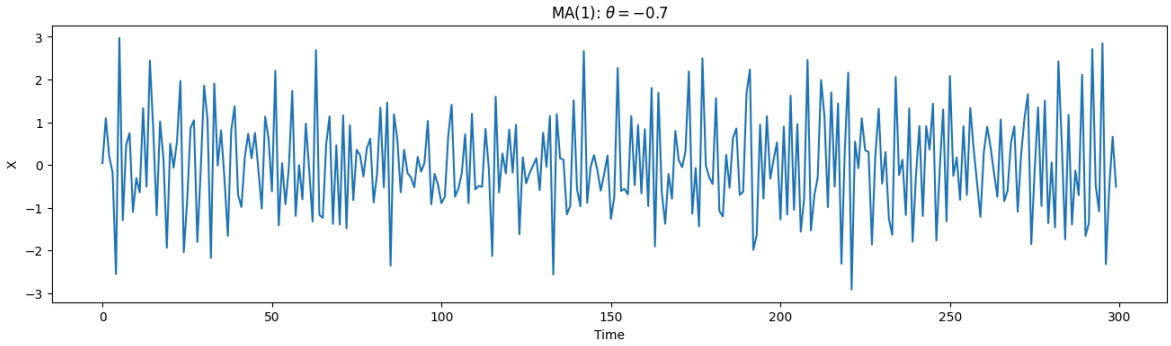
In [10]:
```
# Moving Average examples


# Helper MA1 function
def MA1(theta):
    arparams = np.array([0.0])
    ar = np.r_[1, -arparams]
    maparams = np.array([theta])
    ma = np.r_[1, maparams]  # add zero-lag
    y = arma_generate_sample(ar, ma, nsample=300)
    return y


# MA(1): theta=0.3
plt.figure(figsize=(16, 4))
plt.plot(MA1(0.3))
plt.title("MA(1): $\\theta=0.3$")
plt.xlabel("Time")
plt.ylabel("X")
plt.show()

# MA(1): theta=-0.7
plt.figure(figsize=(16, 4))
plt.plot(MA1(-0.7))
plt.title("MA(1): $\\theta=-0.7$")
plt.xlabel("Time")
plt.ylabel("X")
plt.show()
```





From figure 9, we can see the time series in the bottom plot is more volatile than the time series in the top plot. Now, let's check the mean, autocovariance, and autocorrelation of MA(1) process.

Mean:

$$E(X_t) = E(W_t + \theta W_{t-1}) = 0$$

Autocovariance:

$$\gamma(h) = \begin{cases} (1+\theta^2)\sigma^2 & \text{for } h = 0 \\ \theta\sigma^2 & \text{for } h = 1 \\ 0 & \text{for } h > 1 \end{cases}$$

ACF:

$$\rho(h) = \begin{cases} \frac{\theta}{(1+\theta^2)} & \text{for } h = 1 \\ 0 & \text{for } h > 1 \end{cases}$$

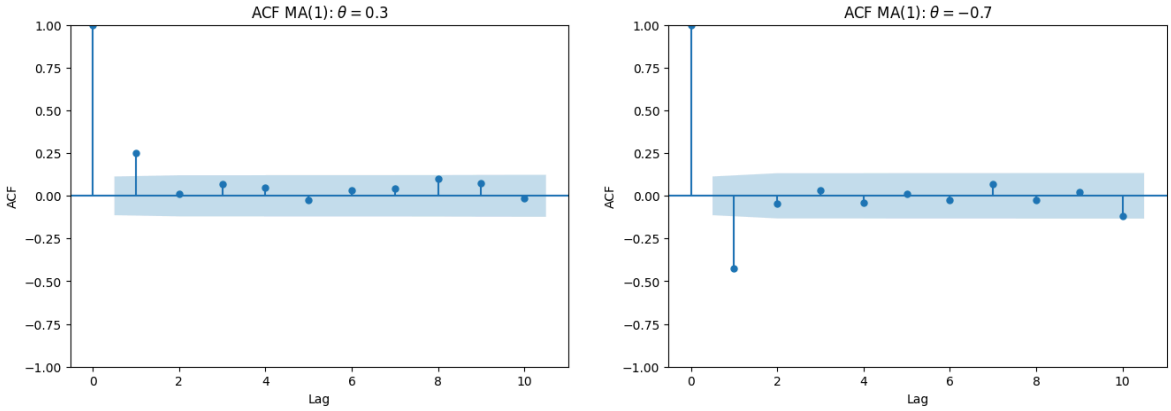
From the above formula, we know the mean and autocovariance for MA time series are independent of time. Hence, MA time series are stationary. Let's look at the ACF plots for MA(1).

Figure 10: ACF of MA(1): $\theta = 0.3$ and MA(2): $\theta = -0.7$

In [11]:
```
# ACF for MA(1) examples
plt.figure(figsize=(16, 5))

# ACF for MA(1): theta=0.3
ax1 = plt.subplot(121)
sm.graphics.tsa.plot_acf(MA1(0.3), title="ACF MA(1): $\\theta=0.3$", lags=10, ax
plt.xlabel("Lag")
plt.ylabel("ACF")

# ACF for MA(1): theta=-0.7
ax2 = plt.subplot(122)
sm.graphics.tsa.plot_acf(MA1(-0.7), title="ACF MA(1): $\\theta=-0.7$", lags=10,
plt.xlabel("Lag")
plt.ylabel("ACF")
plt.show()
```



From figure 10, we can see that autocorrelations are 0 beyond lag 1. This fact is also reflected in the formula of autocovariance and autocorrelation above.

5. Application of Moving Average Model

In this section, we will build a time series model for Google stock price using an MA(1) model. We will first build an MA(1) model using the original Google stock price and then we will build another MA(1) model using the first difference of Google's stock price.

5.1 MA(1) Model with Original Google Stock Price

Figure 11 shows the MA(1) model result for the original Google stock price.

Figure 11: MA(1) Model Result for Original Google Stock Price

In [12]:
```
# MA(1) for Google stock
goog_ma1 = statsmodels.tsa.arima.model.ARIMA(goog.GOOGLE, order=(0, 0, 1)).fit()
print(goog_ma1.params)
```

```
/usr/local/lib/python3.11/site-packages/statsmodels/tsa/base/tsa_model.py:473: Va
lueWarning: A date index has been provided, but it has no associated frequency in
formation and so will be ignored when e.g. forecasting.
  self._init_dates(dates, freq)
/usr/local/lib/python3.11/site-packages/statsmodels/tsa/base/tsa_model.py:473: Va
lueWarning: A date index has been provided, but it has no associated frequency in
formation and so will be ignored when e.g. forecasting.
  self._init_dates(dates, freq)
/usr/local/lib/python3.11/site-packages/statsmodels/tsa/base/tsa_model.py:473: Va
lueWarning: A date index has been provided, but it has no associated frequency in
formation and so will be ignored when e.g. forecasting.
  self._init_dates(dates, freq)
const      1327.678275
ma.L1         0.969855
sigma2    96563.620427
dtype: float64
```

From figure 11, we see the estimated θ is 0.9699 and the intercept is 1327.617. Let's see how the model fits the actual data. Figure 12 shows the plot comparing original Google stock price and fitted stock price.

Figure 12: Original Google Stock Price vs. Fitted Google Stock Price

In [13]:
```
# Plot Google vs fitted Google stock prices
goog_ma1_res = goog_ma1.resid
goog_ma1_fit = goog["GOOGLE"] - goog_ma1_res

goog["GOOGLE"].plot(linewidth=0.8, label="Original Google stock price")
goog_ma1_fit.plot(linewidth=0.8, label="Fitted Google stock price")
plt.xlabel("Time")
plt.ylabel("Price")
plt.legend()
plt.show()
```



From figure 12, we can see the Fitted Google stock price curve (orange) does not follow the actual Google stock price curve (blue) very closely. We know that the original Google stock price is not stationary. Hence, let's perform first differencing of the original stock price and check the first difference time series against an MA(1) model again.

5.2 MA(1) Model with First Difference of Google Stock Price

Figure 13: MA(1) Model Result for First Difference of Google Stock Price

In [14]:
```
# MA(1) for first difference of Google stock price
goog_ma2 = statsmodels.tsa.arima.model.ARIMA(goog.GOOGLE, order=(0, 1, 1)).fit()
print(goog_ma2.params)
```

```
ma.L1      -0.078980
sigma2    532.948967
dtype: float64
```

```
/usr/local/lib/python3.11/site-packages/statsmodels/tsa/base/tsa_model.py:473: Va
lueWarning: A date index has been provided, but it has no associated frequency in
formation and so will be ignored when e.g. forecasting.
  self._init_dates(dates, freq)
/usr/local/lib/python3.11/site-packages/statsmodels/tsa/base/tsa_model.py:473: Va
lueWarning: A date index has been provided, but it has no associated frequency in
formation and so will be ignored when e.g. forecasting.
  self._init_dates(dates, freq)
/usr/local/lib/python3.11/site-packages/statsmodels/tsa/base/tsa_model.py:473: Va
lueWarning: A date index has been provided, but it has no associated frequency in
formation and so will be ignored when e.g. forecasting.
  self._init_dates(dates, freq)
```

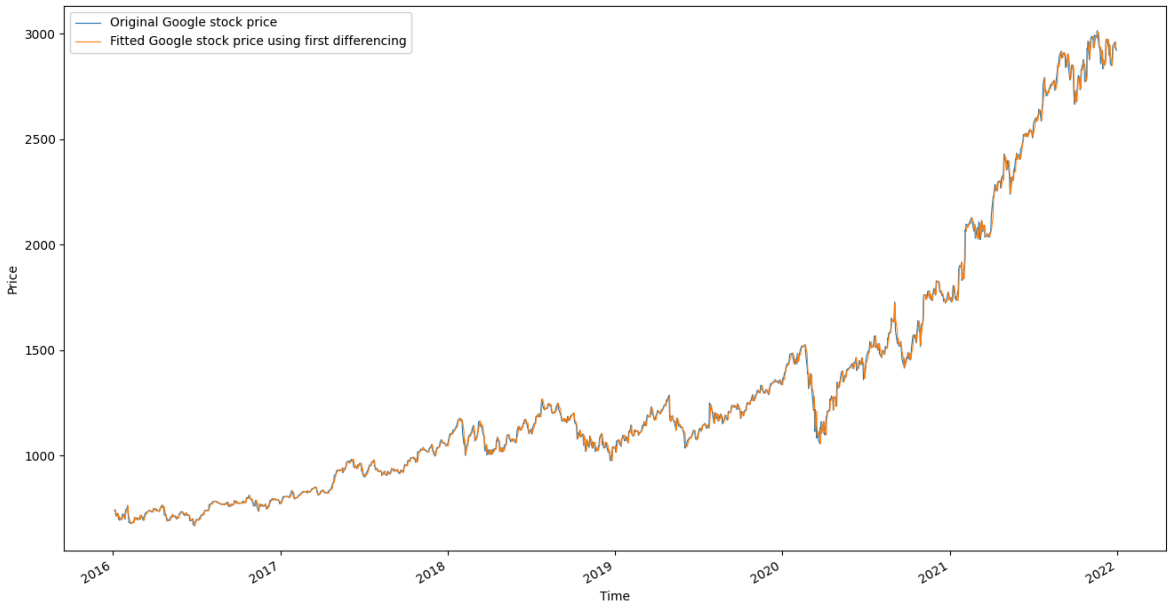
We can see now we only have estimated θ = -0.07898 but no intercept. It is because we are modeling the first difference. Let's check the actual vs. predicted graph.

Figure 14: Google Stock Price vs. Fitted Google Stock Price Using First Differencing

In [15]:
```
# Plotting Google vs fitted Google stock prices using first differencing
goog_ma2_res = goog_ma2.resid
goog_ma2_fit = goog["GOOGLE"] - goog_ma2_res

goog_ma2_res = goog_ma2_res[
    1:
]  # need to skip first element since the model lost one data point
goog_ma2_fit = goog_ma2_fit[
    1:
]  # need to skip first element since the model lost one data point

goog["GOOGLE"].plot(linewidth=0.8, label="Original Google stock price")
goog_ma2_fit.plot(
    linewidth=0.8, label="Fitted Google stock price using first differencing"
)
plt.xlabel("Time")
plt.ylabel("Price")
plt.legend()
plt.show()
```



We can see from figure 14 that first differencing resulted in a considerably better model than for the original dataset. It also highlights the importance of making a time series

stationary before determining a model for the stationary part of the same.

6. Conclusion

In this lesson, we first discussed two methods to remove a trend in a time series. The first method is to introduce a trend variable to a simple regression model. The second method is to use first difference. We then started to go through some time series statistical models. We talked about white noise as an error term for time series. Then, we talked about random walk and random walk with a drift to model non-stationary time series. We then introduced moving average model and applied it to Google stock price. In the next lesson, we will continue our journey and learn more time series statistical models.

Copyright 2025 WorldQuant University. This content is licensed solely for personal use. Redistribution or publication of this material is strictly prohibited.

FINANCIAL ECONOMETRICS

MODULE 4 | LESSON 3

AUTOREGRESSIVE MODEL

Reading Time	90 minutes
Prior Knowledge	Basic Time Series knowledge, White noise, Random walk
Keywords	Autoregressive AR(p) process or model, Autoregressive operator, Mean, Variance, Autocovariance and Autocorrelations for AR(1), Invertible MA, Method of Moments, First order moment, second order moment, Yule-Walker estimation, Maximum Likelihood estimation (MLE)

In the last lesson, we discussed white noise, random walk, and moving average (MA) time series models. In this lesson, we will continue to explore more time series models. Specifically, we will talk about the autoregressive model (AR model). We will go over the definition and properties of the AR model. We will also compare the autoregressive model with the moving average model. We will end the lesson with one application.

In [1]:
```
import matplotlib.pyplot as plt
import numpy as np
import pandas as pd
import pylab
import statsmodels.api as sm
from statsmodels.tsa.arima.model import ARIMA
from statsmodels.tsa.arima_process import arma_generate_sample

plt.rcParams["figure.figsize"] = (16, 9)  # Figure size and width
```

In [2]:
```
# Download the dataset
m4_data = pd.read_csv("M4. goog_eur_10.csv")
dxyr_data = pd.read_csv("M4. dxy_r_data.csv")

# Convert date variable to date format
m4_data["Date2"] = pd.to_datetime(m4_data["Date"], format="%m/%d/%Y")
dxyr_data["Date2"] = pd.to_datetime(dxyr_data["Date"], format="%m/%d/%Y")

# Selecting columns and setting index
goog = m4_data.loc[:, ["Date2", "GOOGLE"]].set_index("Date2")
ust10 = m4_data.loc[:, ["Date2", "UST10Y"]].set_index("Date2")
dxy = dxyr_data[["Date2", "DXY_R"]].set_index("Date2")
```

1. Ordinary Least Square Regression for Time Series Analysis

From previous lessons, we know that often observations in a time series are correlated with each other. If we want to use the past observations to predict today's observations, the first thing that comes to mind is to use OLS (ordinary least square regression). In regular OLS, we have a dependent variable Y and an independent variable X. We can write a linear regression as follows:

$$Y = C + b_1 X_1 + \varepsilon$$

where C is the intercept and ε is the error term.

To apply OLS to a time series X_t as a dependent variable, and its lag 1, X_{t-1} as an independent variable, we can write the OLS as follows:

$$X_t = C + b_1 X_{t-1} + \varepsilon_t$$

where C is the intercept and ε_t is the normal white noise and absolute value of $b_1 < 1$.

We then can apply OLS to estimate the coefficients and the residuals and to conduct model diagnostics. The above regression model for time series X_t is called an **autoregressive (AR) model**.

2. Autoregressive (AR) Model

An autoregressive model is a regression model to predict a time series using a linear combination of its pass values as independent variables, like what we demonstrated in the last section for time series X_t. We set up a regression model to predict X_t using its lag 1 variable X_{t-1} as the independent variable. Because there is only one lag 1 variable on the right side of the equation, we also denote this autoregressive model as AR(1).

We just described the concept of an autoregressive model. Now let's formally define an autoregressive model.

2.1 Definition of Autoregressive Model AR(p)

The formula for an autoregressive model with the order of p, denoted as AR(p), is as follows:

$$X_t = \alpha_1 X_{t-1} + \alpha_2 X_{t-2} + \cdots + \alpha_p X_{t-p} + W_t$$

Where X_t is stationary, $\alpha_1, \alpha_2, \cdots, \alpha_p$ are constants and W_t is normal white noise.

If we use backshift operator B, we can rewrite AR(p) as follows:

$$\alpha(B) X_t = (1 - \alpha_1 B - \alpha_2 B^2 - \cdots - \alpha_p B^p) X_t = W_t$$

$\alpha(B) = (1 - \alpha_1 B - \alpha_2 B^2 - \cdots - \alpha_p B^p)$ is defined as an autoregressive operator.

There are restrictions for the value of coefficients α to make sure X_t can stay stationary.

> For AR(1), $-1 < \alpha_1 < 1$.

> For AR(2), $\begin{cases} -2 < \alpha_1 < 2, \\ -1 < \alpha_2 < 1, \\ \alpha_1 + \alpha_2 < 1, \\ \alpha_2 - \alpha_1 < 1 \end{cases}$.

For a higher degree of AR, the requirement is as follows:

> 1. Set autoregressive operator = 0 as follows:
> $(1 - \alpha_1 B - \alpha_2 B^2 - \cdots - \alpha_p B^p) = 0$. This equation is called a characteristic equation.

2. Solve the characteristic equation for B and get all the roots of B.
3. The absolute values of all the roots of B need to be > 1 to ensure X_t is stationary.

We will talk more about these requirements later in this course.

We can see from the AR definition above that random walk is very similar to AR(1) model except that $|\alpha_1| = 1$. Hence, random walk is not stationary.

2.2 Mean, Variance, ACF, and PACF for AR(1)

AR(1) model can be written as follows:

$$X_t = C + \alpha_t X_{t-1} + W_t$$

Where C is a constant and W_t is normal white noise with mean = 0 and variance = σ^2

Now let's look at AR(1)'s mean, variance, autocovariance, ACF, and PACF.

The **mean** function is:

$$\mu = \frac{C}{1-\alpha_1}$$

When there is no intercept C in the process, μ is 0.

The **variance** is:

$$\frac{\sigma^2}{1-\alpha^2}$$

Autocovariance for X_t and X_s is:

$$\gamma(h) = \left(\frac{\sigma^2}{1-\alpha^2}\right)\alpha^{|h|}, \text{ where } |h| = |s-t|$$

Autocorrelation for X_t and X_s is:

$$\rho(h) = (\alpha^{|h|}), \text{ where } |h| = |s-t|$$

Partial Autocorrelation for X_t and X_s is:

$$\phi_{hh} = \begin{cases} \alpha & \text{for } |h| \leq 1 \\ 0 & \text{for } |h| > 1 \end{cases}$$

2.3 AR(1) and AR(2) Examples

Figure 1: AR(1) with $\alpha = 0.4$ and AR(1) with $\alpha = -0.4$

In [3]:
```
# Simulated AR(1) example with alpha=0.4 and AR(1) example with alpha=-0.4

# generate AR(1) models
np.random.seed(12345)
maparams = np.r_[1, np.array([0.0])]
arparams = np.r_[1, -np.array([0.4])]
arparams_neg = np.r_[1, -np.array([-0.4])]
ar1_p4 = arma_generate_sample(arparams, maparams, 100)  # AR(1): alpha=0.4
ar1_n4 = arma_generate_sample(arparams_neg, maparams, 100)  # AR(1): alpha=-0.4

# Plot
plt.figure(figsize=(16, 5))
ax1 = plt.subplot(121)
plt.plot(ar1_p4)
plt.title("AR(1): $\\alpha=0.4$")
plt.xlabel("Time")
plt.ylabel("X")
```

```
ax2 = plt.subplot(122)
ax2.plot(ar1_n4)
plt.title("AR(1): $\\alpha=-0.4$")
plt.xlabel("Time")
plt.ylabel("X")
plt.plot()
```

Out[3]: []

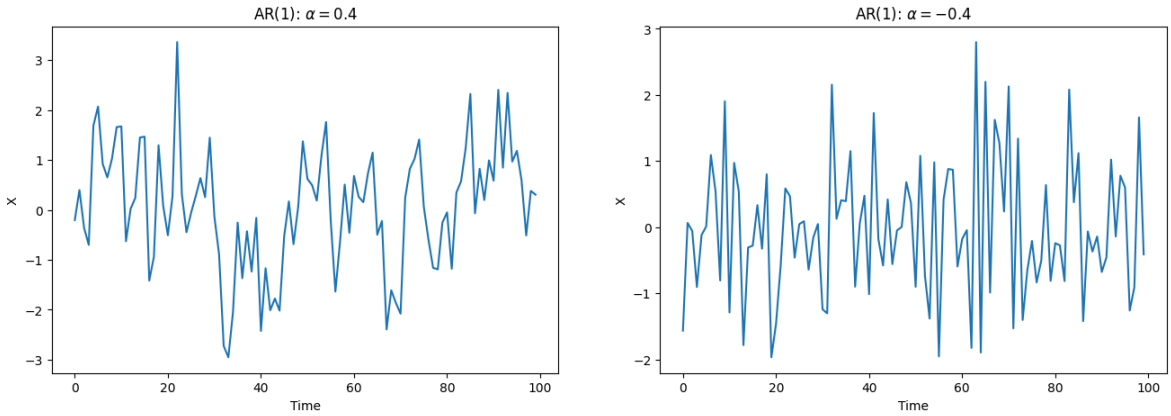


Figure 1 gives us two AR(1) graph examples. They both move around the mean = 0 and within a horizontal band. The AR(1) with negative α is choppier than AR(1) with positive α. We know the correlation of two observations next to each other is autocorrelation $\rho(1)$. For AR(1) model, $\rho(1)$ is α. Hence, when α is positive, the two observations next to each other move in the same direction while when α is negative, the two observations next to each other move in the opposite direction. That is why we see the plot on the left is smoother than the plot on the right.

Figure 2: ACF and PACF of AR(1)

In [4]:
```
# ACF and PACF of AR(1)
fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(16, 5))
sm.graphics.tsa.plot_acf(ar1_p4, title="ACF of AR(1): $\\alpha=0.4$", lags=20, a
sm.graphics.tsa.plot_pacf(ar1_p4, title="PACF of AR(1): $\\alpha=0.4$", lags=20,
plt.show()
```

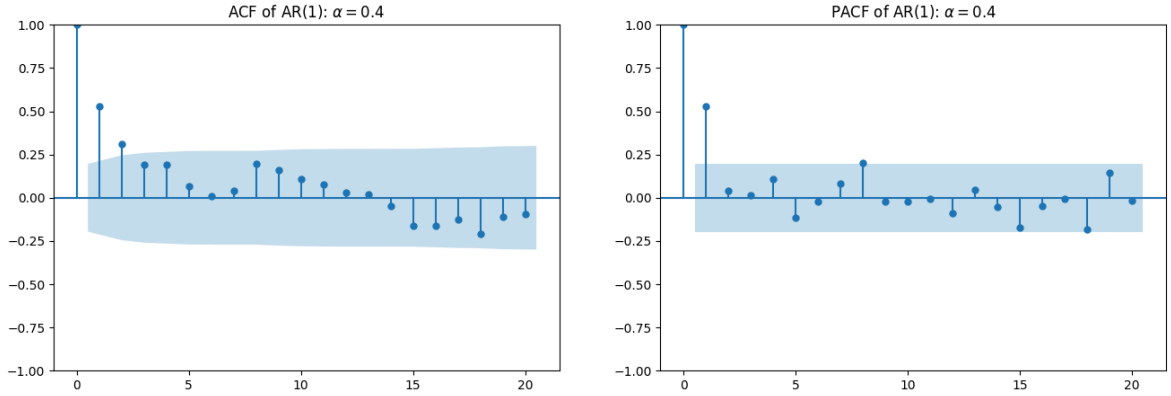


Figure 2 shows the ACF and PACF plots for AR(1) with $\alpha = 0.4$. From both plots, we see ACF and PACF for lag 1 are all significant and their values are between 0.5 and 0.6. Since the example is simulated data from AR(1) with $\alpha = 0.4$, they are random sample data, and ACF and PACF usually are not exactly the same as 0.4.

Now, let's try to simulate AR(2) and see its ACF and PACF plots. The following figure 3 shows the graphs for AR(2).

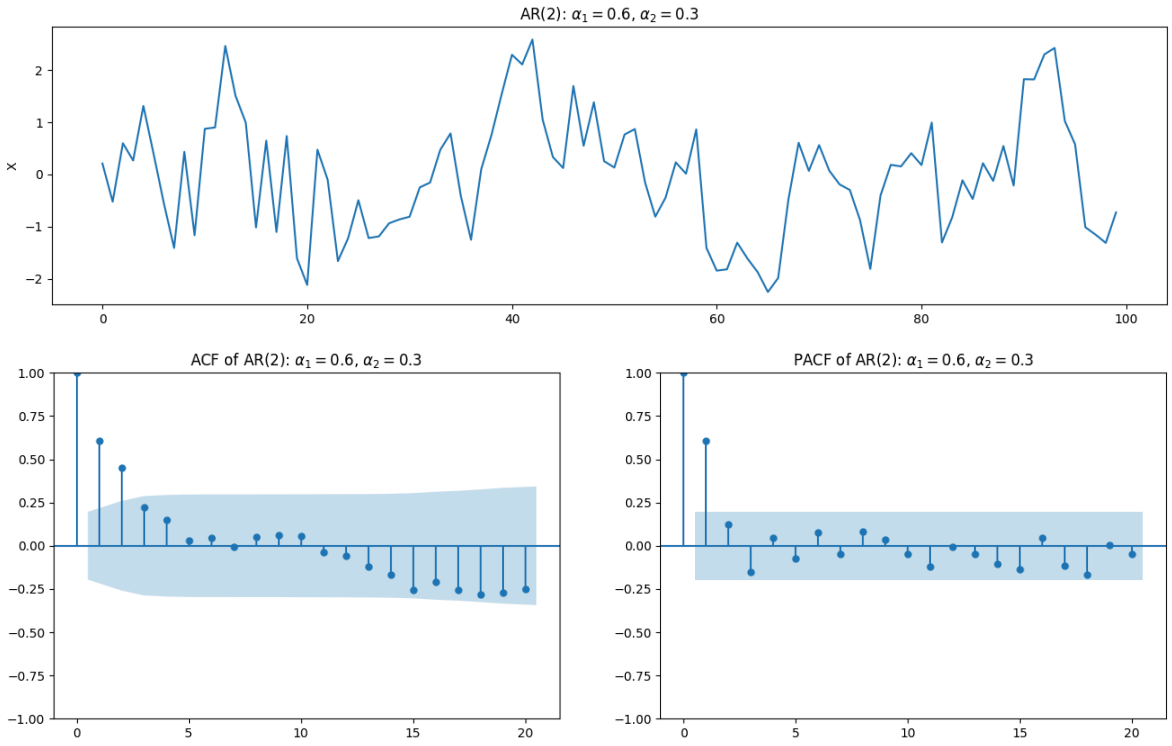
Figure 3: ACF and PACF of AR(2)

In [5]:
```
# ACF and PACF of AR(2)

np.random.seed(77777)
arparams1 = np.r_[1, -np.array([0.6, 0.3])]
maparams1 = np.r_[1, np.array([0.0, 0.0])]
ar2 = arma_generate_sample(arparams1, maparams1, 100)

# Plot AR(2)
plt.figure(figsize=(16, 4))
plt.plot(ar2)
plt.title("AR(2): $\\alpha_1=0.6$, $\\alpha_2=0.3$")
plt.ylabel("X")
plt.show()

# plot ACF and PACF
fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(16, 5))
sm.graphics.tsa.plot_acf(
    ar2, title="ACF of AR(2): $\\alpha_1=0.6$, $\\alpha_2=0.3$", lags=20, ax=ax1
)
sm.graphics.tsa.plot_pacf(
    ar2, title="PACF of AR(2): $\\alpha_1=0.6$, $\\alpha_2=0.3$", lags=20, ax=ax
)
plt.show()
```



Let's focus on the ACF and PACF plots for AR(2) in figure 3. In the ACF plot, we can see the ACF decreases gradually. On the other hand, the PACF in the plot drops to almost 0 after lag 2. This interesting PACF behavior for AR(2) process will help us decide what time series model we will use. We will discuss more about model selection later in this module.

2.4 Use AR to represent MA

One interesting feature about AR process or AR model is that we can represent an MA process using an infinite AR process. Let's use MA(1) as an example.

We have the following MA(1) process:

$$X_t = W_t + \theta W_{t-1}$$

where W is the normal independent white noise with mean = 0 and variance = 1. If and only if $|\theta| < 1$, we can write X_t in an infinite AR form as follows:

$$X_t = \theta X_{t-1} - \theta^2 X_{t-2} + \theta^3 X_{t-3} - \cdots - W_t$$

Here is how to derive the above result:

$$X_t = W_t + \theta W_{t-1}$$

$X_t = (1 + \theta B) W_t$ by using the backshift operator

$W_t = \frac{1}{1+\theta B} X_t$ by rearranging the terms

We also know the formula of infinite geometric series:

$$S_\infty = a + ar + ar^2 + ar^3 + \cdots + ar^\infty = \frac{a}{1-r}$$

when $|r| < 1$.

If $|\theta| < 1$, we can rewrite $\frac{1}{1+\theta B} X_t$ as follows:

$$\frac{X_t}{1+\theta B} = X_t + X_t(-\theta B) + X_t(-\theta B)^2 + X_t(-\theta B)^3 + \cdots + X_t(-\theta B)^\infty$$

$$= X_t - \theta B X_t + \theta^2 B^2 X_t - \theta^3 B^3 X_t + \cdots + \theta^\infty B^\infty X_t$$

We then can plug this back into $W_t = \frac{1}{1+\theta B} X_t$ and rearrange the terms and we will get:

$$X_t = \theta B X_t - \theta^2 B^2 X_t + \theta^3 B^3 X_t + \cdots + W_t$$

Then, we can place the backshift operator back as the subscript for X_t and we get:

$$X_t = \theta X_{t-1} - \theta^2 X_{t-2} + \theta^3 X_{t-3} + \cdots + W_t$$

Now we know that X_t can be written as an infinite AR model from an MA(1) model.

The key point for MA(1) to be able to be represented as an infinite AR process is to make sure its coefficient $|\theta| < 1$. We call this MA(1) **invertible** if the MA(1) process has the absolute value of coefficients less than 1 and can be represented by an infinite AR process.

The other benefit when an MA process is invertible is that the MA process is also **unique**. Let's see an example below to demonstrate this concept.

> MA_A: $X_t = W_t + 5W_{t-1}$ with $W \sim$ normal white noise$(0, 1)$
>
> MA_B: $X_t = W_t + \frac{1}{5}W_{t-1}$ with $W \sim$ normal white noise$(0, 25)$

Both MA_A and MA_B time series have the same autocovariance values shown as follows:

$$\gamma(h) = \begin{cases} 26 & \text{for } h = 0 \\ 5 & \text{for } h = 1 \\ 0 & \text{for } h > 1 \end{cases}$$

In the real world, we only observe the autocovariance values of the time series, not the actual model. We cannot tell if MA_A or MA_B is the correct one. Both MA_A and MA_B are the model candidates for the underlying time series data. So which model should we choose? If we need the MA model to be invertible, then we will limit our model choice to MA_B, which has the absolute value of coefficient less than 1. Then, we call the MA_B model unique.

2.5 ACF and PACF Plots for AR and MA Processes

We briefly talked about the special features ACF and PACF plots exhibit for the AR process earlier. In this section, we are going to compare ACF and PACF plots for both the AR process and MA process together. We are going to use an AR(2) process and an MA(2) process as examples here.

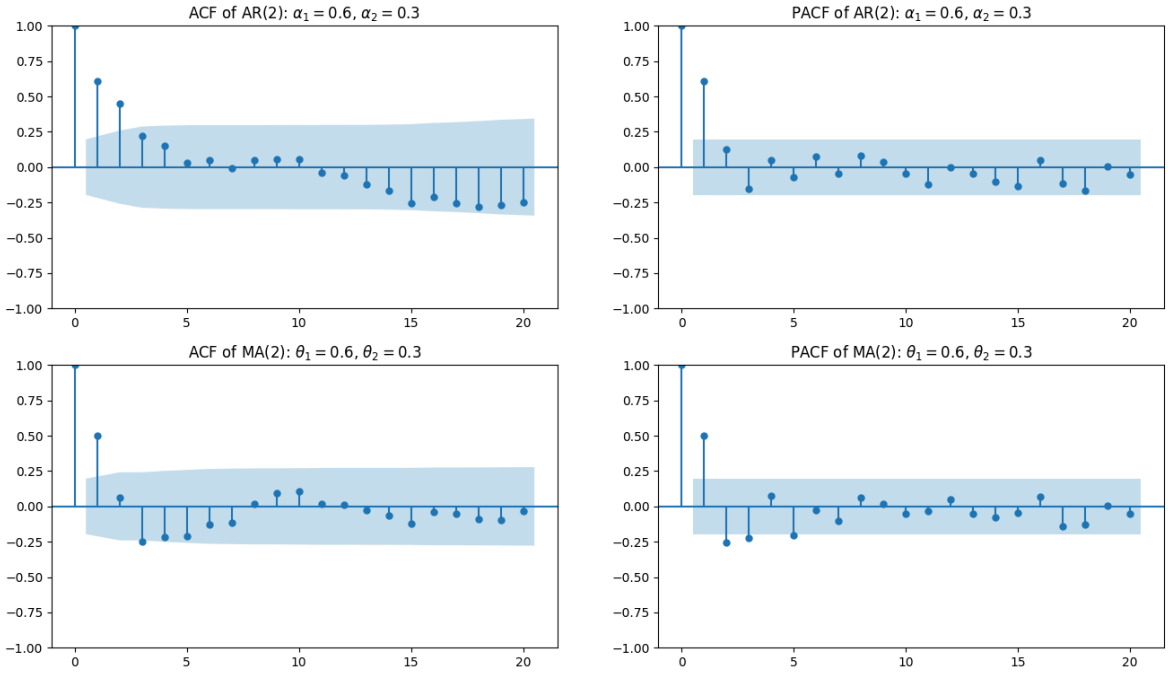
Figure 4: ACF and PACF Plots for AR(2) and MA(2)

In [6]:
```
# ACF and PACF Plots for AR(2) and MA(2)

# MA(2) model
np.random.seed(77777)
arparams2 = np.r_[1, -np.array([0.0, 0.0])]
maparams2 = np.r_[1, np.array([0.6, 0.3])]
ma2 = arma_generate_sample(arparams2, maparams2, 100)


# ACF and PACF of AR(2)
fig, ((ax1, ax2), (ax3, ax4)) = plt.subplots(2, 2)
sm.graphics.tsa.plot_acf(
    ar2, title="ACF of AR(2): $\\alpha_1=0.6$, $\\alpha_2=0.3$", lags=20, ax=ax1
)
sm.graphics.tsa.plot_pacf(
    ar2, title="PACF of AR(2): $\\alpha_1=0.6$, $\\alpha_2=0.3$", lags=20, ax=ax
)

# ACF and PACF of MA(2)
sm.graphics.tsa.plot_acf(
    ma2, title="ACF of MA(2): $\\theta_1=0.6$, $\\theta_2=0.3$", lags=20, ax=ax3
)
sm.graphics.tsa.plot_pacf(
    ma2, title="PACF of MA(2): $\\theta_1=0.6$, $\\theta_2=0.3$", lags=20, ax=ax
)
plt.show()
```



In figure 4 above, the top two plots are ACF and PACF plots for an AR(2) process. The bottom two plots are ACF and PACF plots for an MA(2) process. In an ACF plot, the farthest left bar shows autocorrelation is 1 when $h = 0$, which means it's the correlation for the observation itself. In the PACF plot, it starts with $h = 1$, which means the farthest left bar is for PACF lag 1.

For the AR(2) process, PACF plot shows a drop to almost 0 after lag 2. For the MA(2) process, ACF plot shows a drop to almost 0 after lag 2. This is a particularly important phenomenon. We can actually use this phenomenon to decide the model specification for a time series. If the PACF plot of a time series shows a sudden drop in value to close to 0 after lag p, we can assume the time series follows an AR(p) process. If the ACF plot of a time series shows a sudden drop in value to close to 0 after lag q, we can assume this time series follows an MA(q) process. With the above information, we can use ACF and PACF plots to decide which process and what lag to use to model a time series. Figure 5 summarizes what we just discussed about ACF and PACF plots.

Figure 5: ACF and PACF Plot Features for AR(p) and MA(q)

	ACF Plot Feature	PACF Plot Feature
AR(p)	Tail off	Cut off after p lags
MA(q)	Cut off after q lags	Tail off

2.6 Estimation of AR Model

We have introduced what an AR process model is. We discussed its property and how to select the number of lags p for an AR(p) model. Now we need to estimate the coefficients of the model. There are a few methods to estimate the model. We saw in section 1 that we can use ordinary least square (OLS) method to estimate the model. In

this section, we will go over two other methods here: **method of moments** and **maximum likelihood method**.

2.6.1. Method of Moments (MOM)

Before talking about method of moments, let's briefly review what **moments** are. Moments are statistical parameters used to describe the distribution of a random variable.

The most popular ones are mean and variance. Mean is the first order moment. Variance and covariance are the second order moments. Why do we refer to these moment parameters with order? It is because the functions of these parameters include the expected values of the order of random variable($E(X^n)$). For example, the mean of X is $E(X)$. The variance of X can be written as $E(X^2) - E(X)^2$. Hence, mean is the first order moment and variance is the second order moment for a random variable. Here is the list of popular moments we have used so far:

> - First Order Moment: Mean
> - Second Order Moments: Variance, covariance, correlation, autocovariance, autocorrelation
> - Third Order Moment: Skewness
> - Fourth Order Moment: Kurtosis

Now we know what a moment is. Next, we are going to explain what **method of moments** is. Method of moments is an estimation method that equates a sample moment to its theoretical counterpart and allows us to estimate the parameters for the model. Let's use a very simple example in AR(1) model to explain how it works.

Assume we have a time series X_t and we have removed the trend and the seasonality. The time series is stationary. We have also run ACF and PACF and have decided AR(1) is a good model specification. We assume the mean of X_t is 0 so there is no constant in the AR(1) model. We can write our AR(1) model as follows:

$$X_t = \alpha_t X_{t-1} + W_t$$

Where W_t is normal white noise with mean = 0 and variance = σ^2

From the time series X_t, we can calculate the following information:

> - Sample mean: $\overline{X}$
> - Sample variance: $\widehat{\gamma(0)}$
> - Sample autocovariance of lag 1: $\widehat{\gamma(1)}$

From last section, we know variance and autocovariance formulas are as follows:

The variance is:

$$\frac{\sigma^2}{1-\alpha^2}$$

Autocovariance for X_t and X_s is:

$$\gamma(h) = (\frac{\sigma^2}{1-\alpha^2})\alpha^{|h|}$$

where $|h| = |s-t|$

We can use the above sample information and formulas to solve for AR(1) parameter α.

$$\widehat{\alpha} = \frac{\widehat{\gamma(1)}}{\widehat{\gamma(0)}}$$

2.6.2 Yule-Walker Estimators for AR Model

We introduced MOM in the last section to estimate an AR(1) model. We can generalize the method we used in the last section to solve for a higher order AR(p) model. The method we used above is a very simple case of Yule-Walker estimation. The **Yule-Walker estimation method** connects the coefficients of an AR(p) model to the autocovariances of the time series by using a system of equations called **Yule-Walker equations**.

When we have an AR(p) process as follows:

$$X_t = \alpha_1 X_{t-1} + \alpha_2 X_{t-2} + \cdots + \alpha_p X_{t-p} + W_t$$

Yule-Walker equations can be defined as follows:

$$\begin{aligned}
\gamma(1) &= \alpha_1\gamma(0) + \alpha_2\gamma(-1) + \alpha_3\gamma(-2) + \cdots + \alpha_p\gamma(1-p) \\
\gamma(2) &= \alpha_1\gamma(1) + \alpha_2\gamma(0) + \alpha_3\gamma(-1) + \cdots + \alpha_p\gamma(1-p+1) \\
\gamma(3) &= \alpha_1\gamma(2) + \alpha_2\gamma(1) + \alpha_3\gamma(0) + \cdots + \alpha_p\gamma(1-p+2) \\
&\vdots \\
\gamma(p) &= \alpha_1\gamma(p-1) + \alpha_2\gamma(p-2) + \alpha_3\gamma(p-3) + \cdots + \alpha_p\gamma(0) \\
\sigma^2 &= \gamma(0) - \alpha_1\gamma(1) - \alpha_2\gamma(2) - \cdots - \alpha_p\gamma(p)
\end{aligned}$$

With the above $p+1$ Yule-Walker equation system, we can solve for $\{\alpha_1, \alpha_2, \alpha_3, \cdots, \alpha_p, \sigma^2\}$ $p+1$ parameters. We can see the last equation in the Yule-Walker equation system is an equation to link the white noise variance to model coefficients and autocovariance.

All statistical applications can calculate the estimated coefficients using Yule-Walker equations, so you do not need to calculate them manually.

2.6.3 Maximum Likelihood Method (Box-Jenkins Method)

The next estimation method we will introduce is the **maximum likelihood method (MLE)**. The maximum likelihood method is used to maximize the joint probability of the observed data points with respect to the model parameters. The joint probability is called a **likelihood function**. The likelihood function will usually take the log form so that the function will be easier to handle. Please read the required reading to learn the general definition and properties of the maximum likelihood estimation method.

The maximum likelihood estimation method is the selected estimation method in the Box-Jenkins method. We will talk more about the Box-Jenkins method in the next lesson.

To apply the maximum likelihood method to an AR(1) model, we assume we have the following AR(1) model:

$$X_t = \alpha_t X_{t-1} + W_t$$

Where W_t is normal white noise with mean = 0 and variance = σ^2 and $|\alpha| < 1$

With the above model, we can write our likelihood function as follows:

$$L(\alpha, \sigma^2) = f(x_1, x_2, x_3, \cdots, x_n | \alpha, \sigma^2)$$

Where $x_1, x_2, x_3, \cdots, x_n$ are data points from the time series.

We then maximize $L(\alpha, \sigma^2)$ with respect to α and σ^2 to solve for α. All the statistical applications can solve the optimization for us.

2.6.4 Comparison with Yule-Walker Estimators and Maximum Likelihood Estimators

Now we know how Yule-Walker estimation and maximum likelihood estimation work for an AR model. However, what are the differences between the two methods?

First, Yule-Walker estimation only uses information from first moments, second moments, etc., while the maximum likelihood method uses all data in the likelihood function.

Secondly, when the sample size is larger, the coefficients from the Yule-Walker estimation will have larger variances than coefficients from the maximum likelihood method.

Third, we can only apply the Yule-Walker method to AR models. However, you can apply MLE to most time series models.

However, MLE is computationally complicated while Yule-Walker estimation is relatively straight forward. The choice of estimation method will depend on the computer power and sample size of the data available.

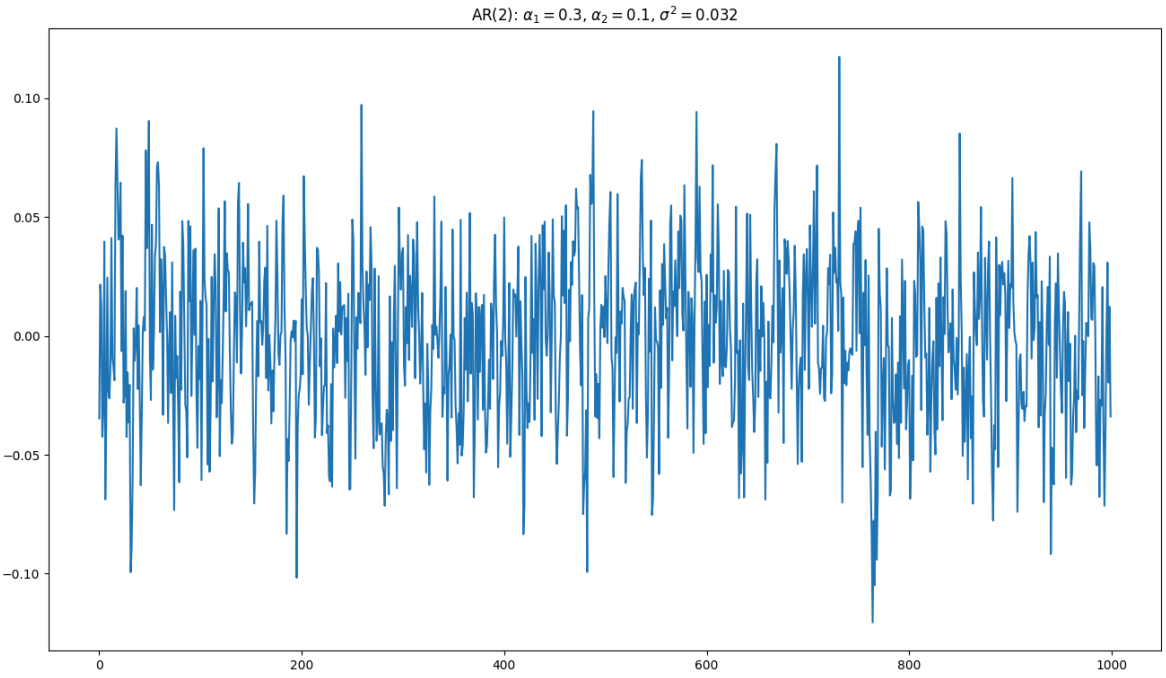
3. Application of Autoregressive Models

In this section, we will try to apply an AR model to simulated time series data. We've simulated an AR(2) time series for demonstration because we can do comparison of the actual parameters and estimated parameters. The analytical process is the same for real-world time series data as the simulated time series data. Figure 6 shows the simulated AR(2) time series plot and its actual parameters.

Figure 6: Simulated AR(2) Time Series

In [7]:
```
# Simulated AR(2) Time Series
np.random.seed(123)
nobs = 1000
arparams3 = np.r_[1, -np.array([0.3, 0.1])]
maparams3 = np.r_[1, np.array([0.0, 0.0])]
sim0 = arma_generate_sample(
    arparams3, maparams3, nobs, scale=0.032
)  # scale=standard deviation

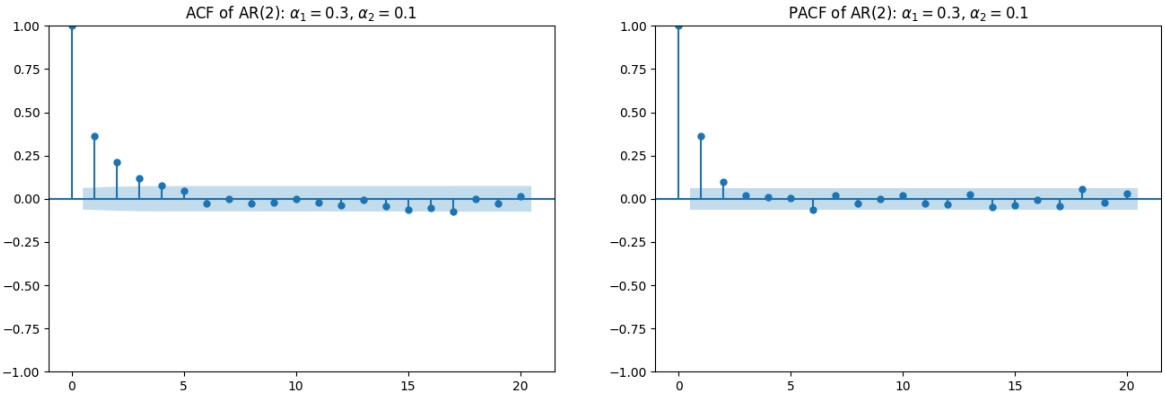
# Plot AR(2)
plt.plot(sim0)
plt.title("AR(2): $\\alpha_1=0.3$, $\\alpha_2=0.1$, $\\sigma^2=0.032$")
plt.show()
```



We next draw ACF and PACF plots as figure 7.

Figure 7: ACF and PACF Plots for Simulated AR(2) Time Series

In [8]:
```
# ACF and PACF for Simulated AR(2)
fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(16, 5))
sm.graphics.tsa.plot_acf(
    sim0, title="ACF of AR(2): $\\alpha_1=0.3$, $\\alpha_2=0.1$", lags=20, ax=ax
)
sm.graphics.tsa.plot_pacf(
    sim0, title="PACF of AR(2): $\\alpha_1=0.3$, $\\alpha_2=0.1$", lags=20, ax=a
)
plt.show()
```



From figure 7, ACF decreases gradually while PACF drops suddenly. This indicates we should use the AR(2) model, which fits the actual time series process.

Now, let's run an AR(2) time series model. Figure 8 shows the AR(2) model result.

Figure 8: AR(2) Model Result

In [9]:
```
# AR(2) Model Result
sim0_fit = ARIMA(sim0, order=(2, 0, 0), trend="n").fit()
print(sim0_fit.summary())
```

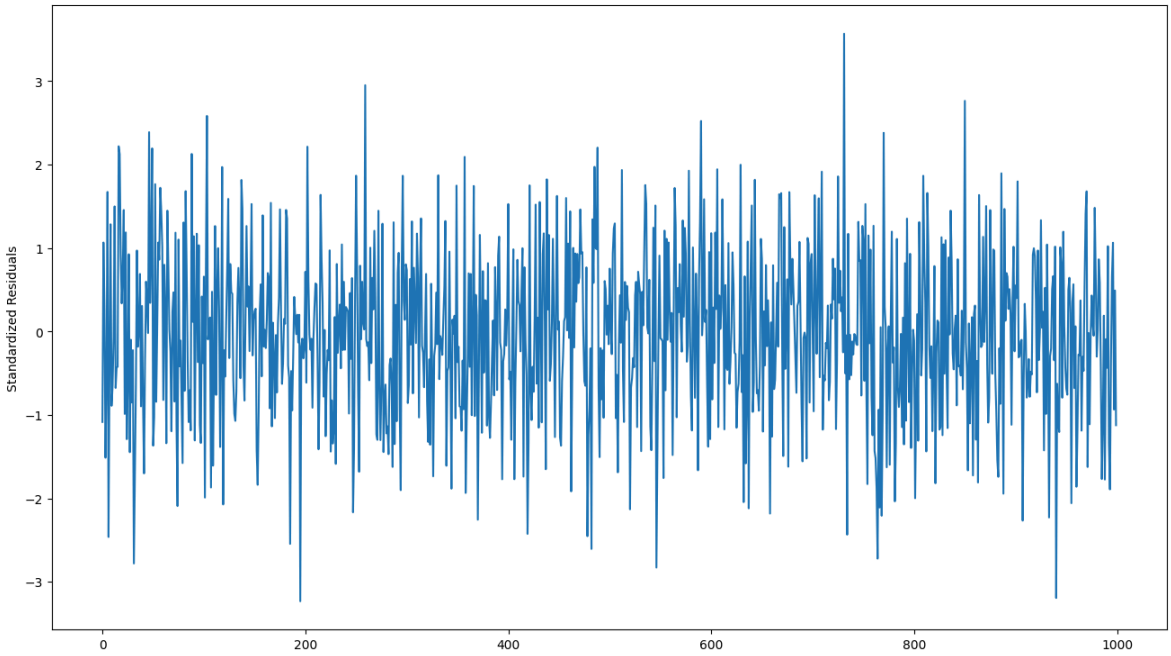
```
                               SARIMAX Results
==============================================================================
Dep. Variable:                      y   No. Observations:                 1000
Model:                 ARIMA(2, 0, 0)   Log Likelihood                2021.897
Date:                Sat, 06 Dec 2025   AIC                          -4037.795
Time:                        19:28:58   BIC                          -4023.072
Sample:                             0   HQIC                         -4032.199
                                - 1000
Covariance Type:                  opg
==============================================================================
                 coef    std err          z      P>|z|      [0.025      0.975]
------------------------------------------------------------------------------
ar.L1          0.3289      0.033      9.995      0.000       0.264       0.393
ar.L2          0.0966      0.033      2.960      0.003       0.033       0.161
sigma2         0.0010   4.62e-05     22.217      0.000       0.001       0.001
===================================================================================
==
Ljung-Box (L1) (Q):                   0.02   Jarque-Bera (JB):                 0.
14
Prob(Q):                              0.89   Prob(JB):                         0.
93
Heteroskedasticity (H):               1.00   Skew:                            -0.
03
Prob(H) (two-sided):                  0.98   Kurtosis:                         2.
98
===================================================================================
==

Warnings:
[1] Covariance matrix calculated using the outer product of gradients (complex-st
ep).
```

From figure 8, we can see both coefficient estimations are close to the actual values and their p-values are all significant. Now, let's check the residuals to see if the residuals follow normal white noise.

Figure 9: Standardized Residuals from AR(2) Model

In [10]:
```
# Standardized Residuals from AR(2) Model
sim0_fit_res = sim0_fit.resid
sim0_fit_res_sta = sim0_fit_res / np.std(sim0_fit_res, ddof=1)
plt.plot(sim0_fit_res_sta)
plt.ylabel("Standardized Residuals")
plt.show()
```

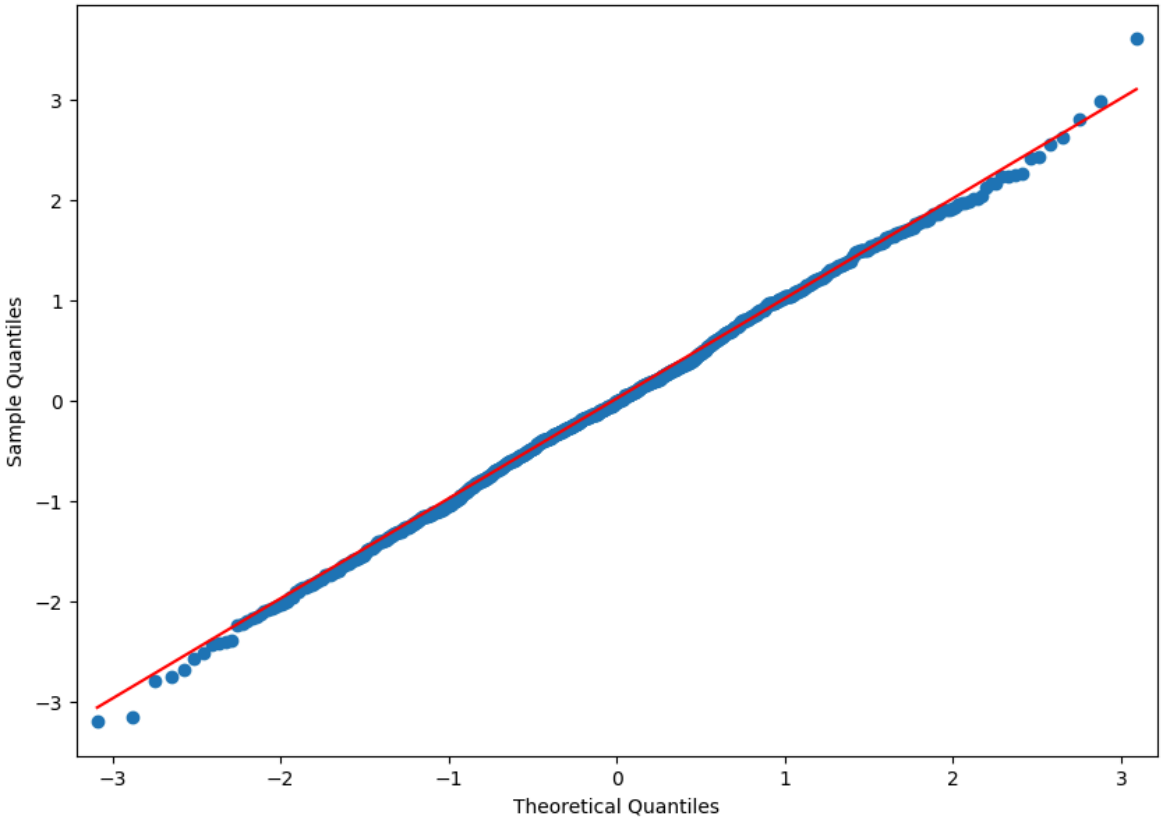


From figure 9, we can see standardized residuals oscillate around 0 and move within a flat channel. It indicates the residuals are stationary.

Now let's check the QQ plot for the normality of the residuals.

Figure 10: QQ Plot for AR(2) Residuals

In [11]:
```
# QQ Plot for AR(2) Residuals
f = sm.qqplot(sim0_fit_res, line="q", fit=True)
f.set_size_inches(10, 7)
pylab.show()
```



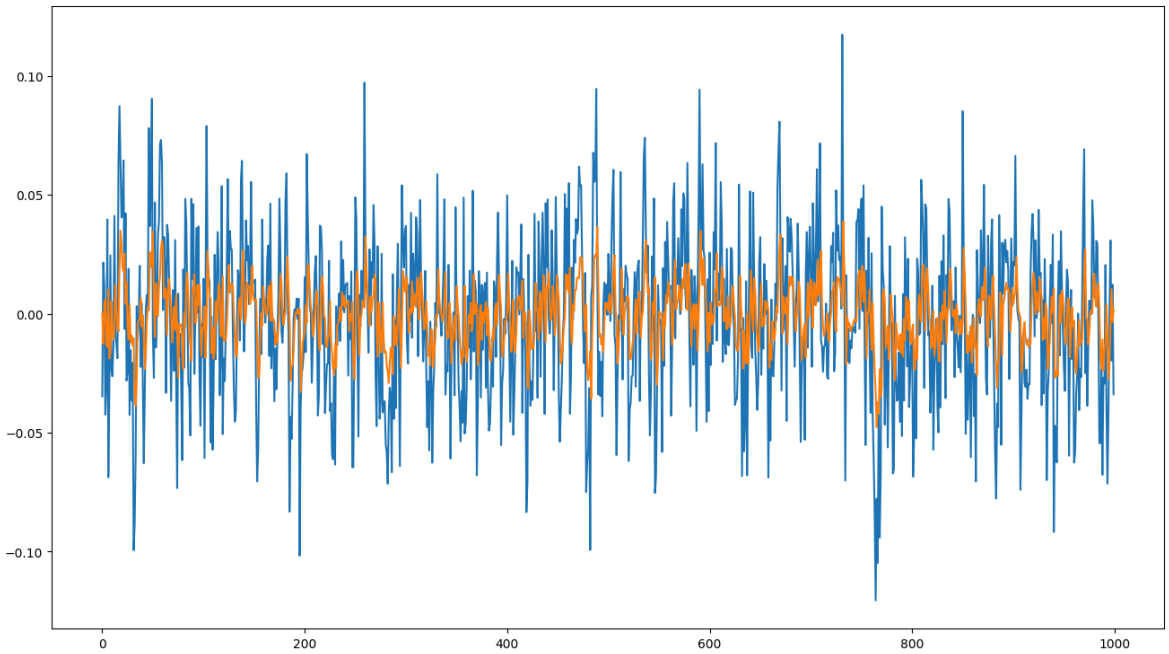
From figure 10, we can see that residuals are normally distributed. Hence, we can confirm that the residuals are normal white noise. Lastly, let's draw a plot to show simulated time series vs. fitted time series.

Figure 11: Simulated Time Series vs. Fitted Time Series

In [12]:
```
# Plot Time Series vs. Fitted Time Series

# TS plot
plt.plot(sim0)

# Fitted TS plot
sim0_ar2_fit = sim0 - sim0_fit_res
plt.plot(sim0_ar2_fit)
plt.show()
```



In figure 11, the black line is the simulated time series, and the orange line is the fitted time series. From the plot, we can see that the fitted line captures the general trend of the simulated time series but not the variation in the simulated time series. The differences are covered by the residuals.

4. Conclusion

In this lesson, we went through the autoregressive model. We first give a brief description of what an AR model is. We then provided the formal definition of an AR model, provided some examples of AR processes, and gave the moments of an AR model. We then moved to discuss how to use ACF and PACF plots to identify an MA model or an AR model. We also explained why and when we can use an infinite AR process to represent an MA model. Then, we talked about two estimation methods for an AR model: the Yule-Walker method and maximum likelihood method. We ended the lesson with a simulated application to show how to model an AR model and conduct post model diagnostics. In the next lesson, we will introduce one comprehensive model and steps to run the model.

Copyright 2025 WorldQuant University. This content is licensed solely for personal use. Redistribution or publication of this material is strictly prohibited.

FINANCIAL ECONOMETRICS

MODULE 4 | LESSON 4

ARIMA MODEL

Reading Time	60 minutes
Prior Knowledge	Basics of time series, Moving average, Autoregressive
Keywords	ARMA model, ARIMA model, Causality, Invertibility, The principle of parsimony, AIC, BIC, Box-Je

In previous lessons, we learned about the basics of time series, moving average models, and autoregressive models. In this lesson, we are going to use all the time series knowledge we have learned and apply it to this lesson. We are going to introduce the ARIMA model. The ARIMA model combines the AR model and MA model as one model for time series analysis. The Box-Jenkins method provides all the modeling steps for the ARIMA model so the Box-Jenkins method and ARIMA model can be used interchangeably. We will talk about what the ARIMA model is and how to apply it to analyze a time series. Before talking about the ARIMA model, let's start with the ARMA model.

```
# !pip install pmdarima
```

```
import matplotlib.pyplot as plt
import numpy as np
import pandas as pd
import statsmodels.api as sm
from pmdarima.arima import auto_arima
from statsmodels.tsa.statespace.sarimax import SARIMAX

plt.rcParams["figure.figsize"] = (16, 9)  # Figure size and width
```

```
# Download the dataset
m4_data = pd.read_csv("M4. goog_eur_10.csv")
dxyr_data = pd.read_csv("M4. dxy_r_data.csv")

# Convert date variable to date format
m4_data["Date2"] = pd.to_datetime(m4_data["Date"], format="%m/%d/%Y")
dxyr_data["Date2"] = pd.to_datetime(dxyr_data["Date"], format="%m/%d/%Y")

# Selecting columns and setting index
goog = m4_data.loc[:, ["Date2", "GOOGLE"]].set_index("Date2")
```

```
ust10 = m4_data.loc[:, ["Date2", "UST10Y"]].set_index("Date2")
dxy = dxyr_data[["Date2", "DXY_R"]].set_index("Date2")
```

1. ARMA Model

ARMA stands for **autoregressive moving average model**. An ARMA model combines an autoregressive model (AR model) and a moving average model (MA model) to model stationary time series.

1.1 Definition of ARMA(p, q) Model

We can define an ARMA(p, q) model as follows:

A time series X_t is ARMA(p, q) if it is stationary and

$$X_t = \alpha_1 X_{t-1} + \cdots + \alpha_p X_{t-p} + W_t + \theta_1 W_{t-1} + \cdots + \theta_q W_{t-q}$$

Where $\alpha_p \neq 0, \theta_q \neq 0$ and W is a normal white noise with mean = 0 and variance = σ^2. The parameters p and q are called the autoregressive order and moving average order.

We can see from the above model, if $q = 0$, then ARMA(p, 0) is an AR(p) model. If $p = 0$, then ARMA(0, q) is an MA(q) model.

1.2 Properties of ARMA(p, q)

ARMA(p, q) can also be written as follows:

$$\alpha(B)X_t = \theta(B)W_t$$

Where:

> $\alpha(B)$ is AR polynomial and is equal to
> $\alpha(B) = 1 - \alpha_1 B - \alpha_2 B^2 - \cdots - \alpha_p B^p$
>
> $\theta(B)$ is MA polynomial and is equal to
> $\theta(B) = 1 + \theta_1 B + \theta_2 B^2 + \cdots + \theta_p B^q$

With the above AR and MA polynomials, we can do the following calculations:

> a. Set the two polynomials to 0 and solve for B.
> b. All the roots of B can be expressed as functions of α and θ coefficients.
> c. Set the absolute values of all the roots of $B > 1$, then convert all the inequalities in terms of α and θ coefficients. Now we have a set of

> inequalities to regulate α and θ coefficients
> d. With these regularity conditions on α and θ coefficients, we can ensure that an ARMA(p, q) time series X_t can only be expressed as a function of its past values and not its future values. This property is called **causality**. The regularity conditions also ensure that $\{W_t\}$ from ARMA(p, q) can be expressed as an infinite AR process. This property is called **invertibility**.

Sometimes, we can find several models that work for our dataset. With several model options, we can choose the simplest one with the fewest parameters. Let's look at one ARMA example. Assume we have an ARMA(2,2) process as follows:

$$X_t = 0.2X_{t-1} + 0.35X_{t-2} + W_t + W_{t-1} + 0.25W_{t-2}$$

We can rewrite the above ARMA(2,2) as follows:

$$(1 - 0.2B - 0.35B^2)X_t = (1 + B + 0.25B^2)W_t$$

Then, we can rewrite them in various factors:

$$(1 - 0.7B)(1 + 0.5B)X_t = (1 + 0.5B)^2W_t$$

Then, the model can be simplified as follows:

$$(1 - 0.7B)X_t = (1 + 0.5B)W_t$$

an ARMA(1,1) model instead of an ARMA(2,2).

The above rule of selecting the model with the least number of coefficients or the simplest model is called **the principle of parsimony**. All rights reserved WQU WorldQuant University QQQQ

2. ARIMA Model

ARIMA Model stands for autoregressive integrated moving average model. Why do we add "integrated" into the name of the ARIMA model? "Integrated" in the ARIMA model means to include the differencing method to remove trend or seasonality in a time series. We learned from the last section that an ARMA model only handles stationary time series. It is different for an ARIMA model. An ARIMA model can handle non-stationary time series.

Let's go over ARIMA's definition:

A time series X_t follows a ARIMA(p, d, q) process if $\nabla^d X_t = (1 - B)^d X_t$ is ARMA(p, q).

In general, we can write the ARIMA(p, d, q) as follows:

$$\nabla^d X_t = \alpha_1 \nabla^d X_{t-1} + \alpha_2 \nabla^d X_{t-2} + \cdots + \alpha_p \nabla^d X_{t-p} + \theta_1 W_{t-1} + \theta_2 W_{t-2} + \cdots$$

We can also rewrite the model as follows:

$$\alpha(B)(1-B)^d X_t = \theta(B)W_t$$

In ARIMA(p, d, q), p is the order of the AR component, d is the order of differencing and q is the order of the MA component.

We have learned a few time series models so far, they can all be special cases of ARIMA model. Figure 1 summarizes these models in terms of ARIMA representation.

Figure 1: Special Cases for ARIMA Model

Time Series Model	ARIMA(p, d, q) Representation
White Noise	ARIMA(0,0,0)
Random Walk	ARIMA(0,1,0) with no constant
Random Walk with a Drift	ARIMA(0,1,0) with a constant
Autoregressive	ARIMA(p,0,0)
Moving Average	ARIMA(0,0,q)

3. Model Selection Criteria: AIC and BIC

Before going into Box-Jenkins method, we would like to go over some metrics used to select the best time series model among several choices.

The first one is **Akaike's information criterion (AIC)**. This is a metric that measures the goodness of fit for a model and can balance the trade off between less precise predictions vs. number of parameters in the model. The lower the AIC, the better the model. AIC is preferred when the sample size is small.

The second one is **Bayesian information criterion (BIC)**. It is also called Schwarz information criterion (SIC). BIC also measures the goodness of fit for a model by balancing the error of the model and the number of parameters. However, BIC puts more emphasis on penalizing number of parameters. Because of this feature in BIC, it usually prefers a simpler model with less parameters. We also look for a lowest BIC when comparing among several model candidates.

In general, the AIC and BIC results would match. However, there are occasions where the two results would not be the same. If the results are different, one can report both. In the following examples, we will use AIC.

4. Box-Jenkins Methodology

The Box-Jenkins method was first introduced by George Box and Gwilym Jenkins. The Box-Jenkins method offers a methodology to derive models for time series data. The

steps in Box-Jenkins method consist of

- model identification,
- model estimation, and
- model diagnostics.

In the rest of the section, we will briefly discuss each step, and we will provide an application in the next section.

4.1 Model Identification

We will use time series plot, ACF plot, and PACF plot and other information to select the order of AR(p), order of differencing (d), or order of MA(q) for a ARIMA model. Many times the plots or information at hand do not give us clear information to identify (p, d, q). For example, if both p and q are greater than 0, both ACF and PACF plots will decline geometrically. In that case, we can create a list of possible ARIMA models with various p, d, q combinations. We will then run each ARIMA model and record their AICs and BICs. Then, we choose the model with the smallest AIC and BIC values.

4.2 Model Estimation

The Box-Jenkins method uses a maximum likelihood method and back-testing to estimate an ARIMA model.

4.3 Model Diagnostics

After the model is built, we will use a residual plot, a residual QQ plot, ACF of residuals, and the Ljung-Box test to check the goodness-of-fit of the model. We use a residual plot to see if there is any pattern that shows up. We are looking for a random pattern if the model is constructed accurately. For a residual QQ plot, we are looking for the normality of residuals. ACF of residuals will give us an idea if there is autocorrelation among error terms from the model. The Ljung-Box test is to check if there is any overall autocorrelation in the residuals. Then, we will show you how to conduct the forecast once the model is finalized.

5. ARIMA Application: Google Stock Price

In this section, we are going to demonstrate how to apply ARIMA to build a model for financial assets. We will use Google's stock price as an example. We will follow the Box-Jenkins methodology to build this model.

5.1 Model Identification and Model Estimation

Model identification has always been a trial-and-error process. We usually try different model candidates until we get a satisfactory one. During the process of searching for a fit model, we will of course estimate a model candidate. Hence, in this section, we will combine model identification and model estimation together.

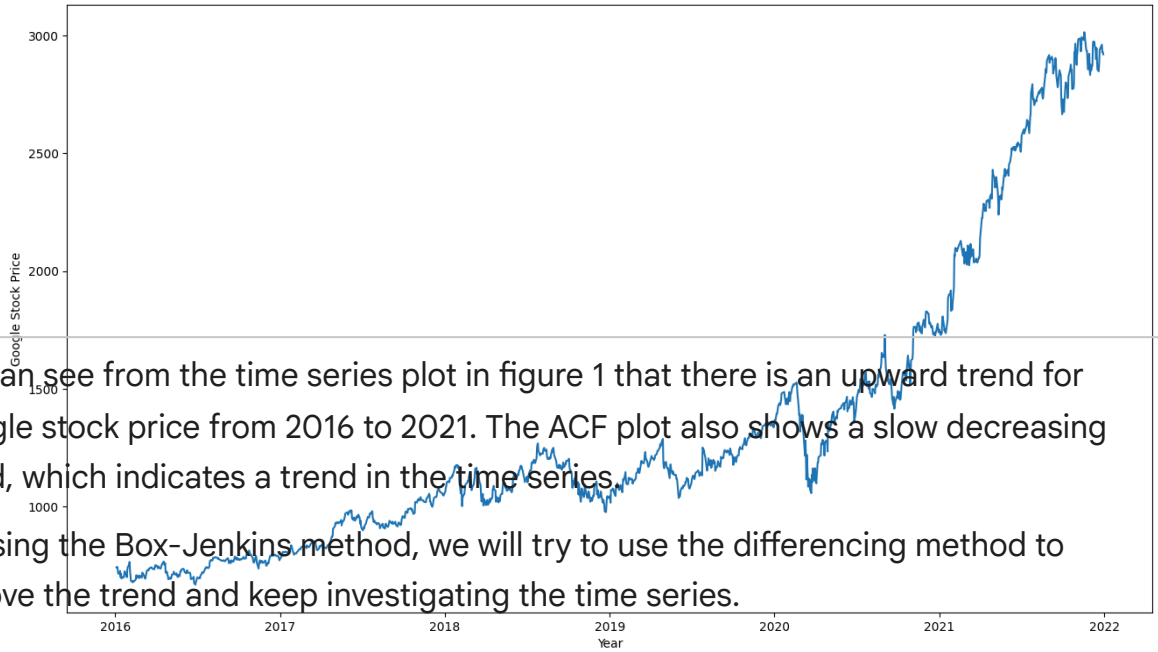
First, let's plot Google's stock time series plot, ACF plot, and PACF plot to take a look at the data.

Figure 2: Time Series Plot, ACF Plot, and PACF Plot for Google Stock Price from 2016 to 2021

```
# Time Series plot of Google stock price with ACF and PACF

# Plot of Google stock price
plt.plot(goog)
plt.xlabel("Year")
plt.ylabel("Google Stock Price")
plt.show()

# plot ACF and PACF
fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(16, 5))
sm.graphics.tsa.plot_acf(goog, title="Google stock ACF", lags=50, ax=ax1)
sm.graphics.tsa.plot_pacf(goog, title="Google stock PACF", lags=50, ax=ax2)
plt.show()
```



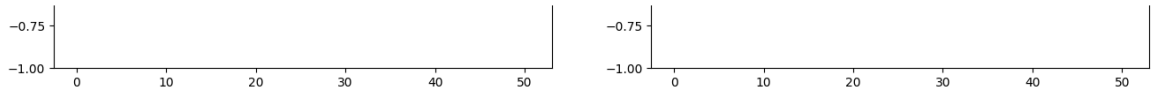
We can see from the time series plot in figure 1 that there is an upward trend for Google stock price from 2016 to 2021. The ACF plot also shows a slow decreasing trend, which indicates a trend in the time series.

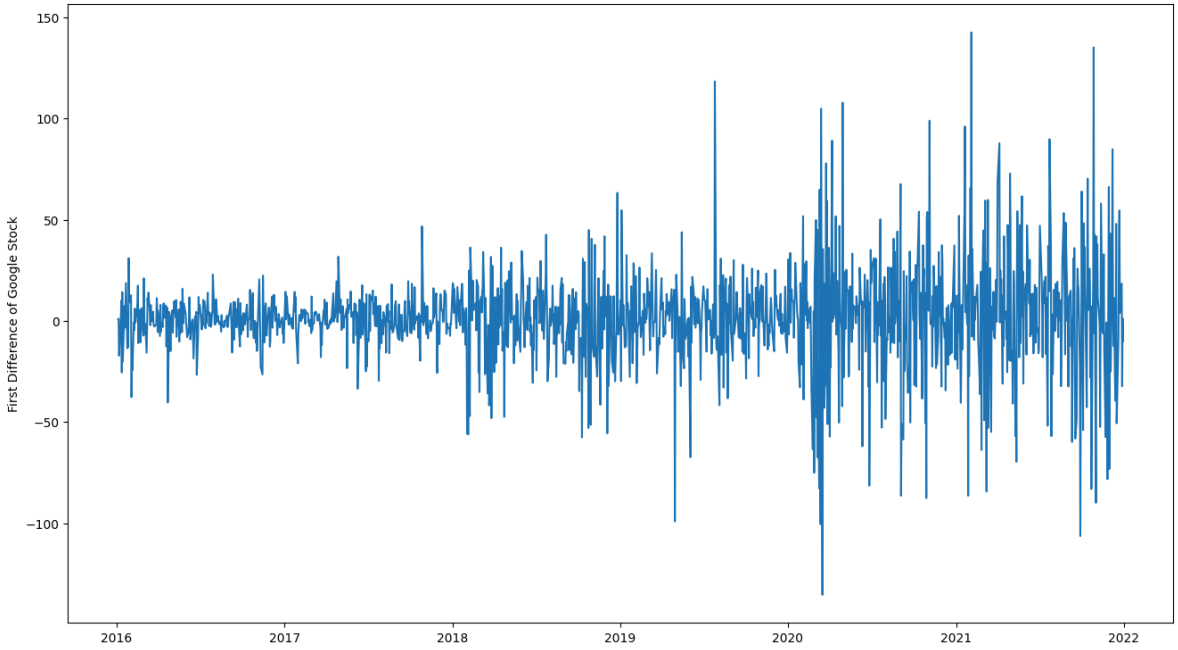
By using the Box-Jenkins method, we will try to use the differencing method to remove the trend and keep investigating the time series.



Figure 3: Plot of First Differencing of Google Stock Price

```
# Plot of First Differencing of Google Stock Price
plt.plot(goog.diff().dropna())
plt.ylabel("First Difference of Google Stock")
plt.show()
```

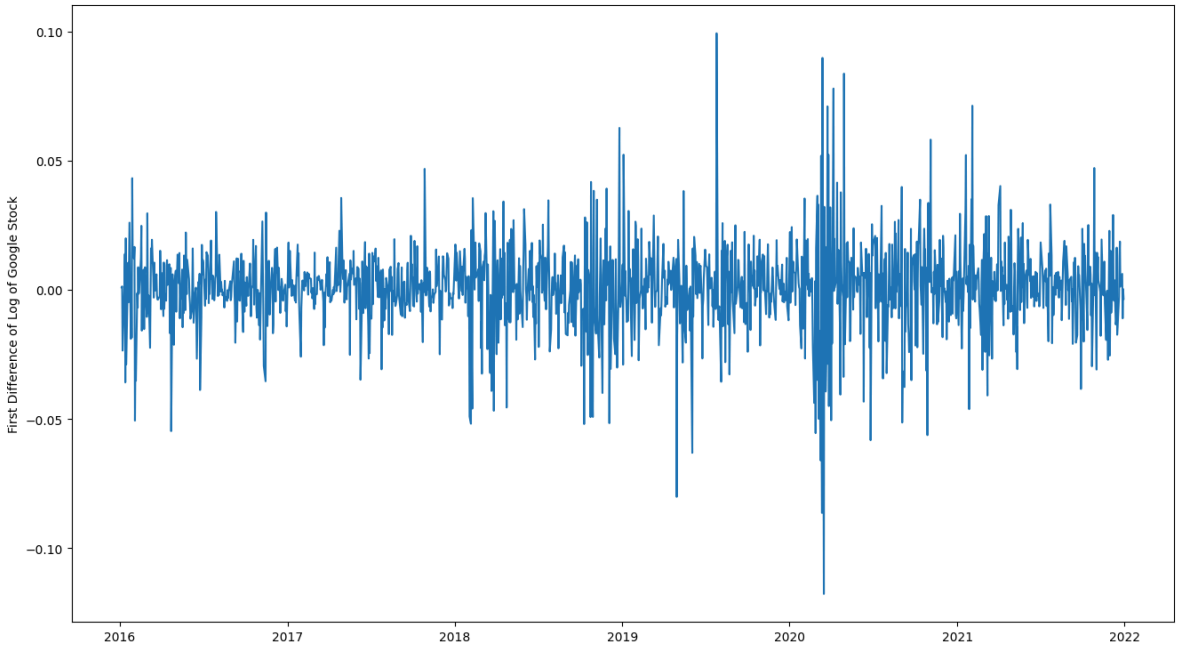




From figure 3, we can see that now the differenced time series oscillates around 0, but the recent movements are more volatile. This indicates that the variance is not constant. It is very common for the financial time series data to have non-constant variance. One way to adjust the time series with non-constant variance is to take the log of the time series. Let's take the log of Google's stock price and do the first differencing. The first difference of the logged Google stock price is very close to Google stock price return, if the return is relatively small. Therefore, in many finance research field, the above method is a way to get the return for a financial asset. Figure 4 shows the result.

Figure 4: Plot First Differencing of Log of Google Stock Price

```
# Plot First Differencing of Log of Google Stock Price
plt.plot(np.log(goog).diff().dropna())
plt.ylabel("First Difference of Log of Google Stock")
plt.show()
```

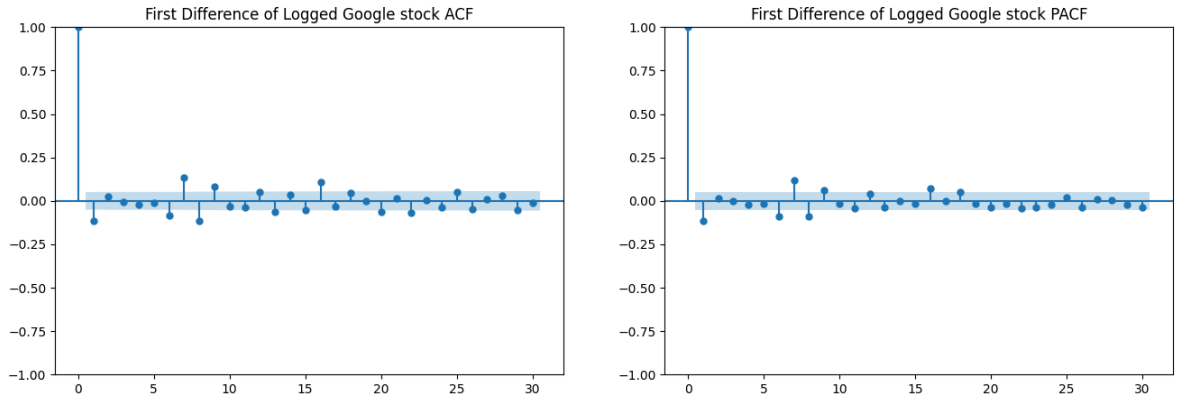


From figure 4, we can see the new time series has a more stable variance even though the second half of the investigated period still has higher variation. We will introduce another time series model to model non-constant variance in the next

module. For now, let's check ACF and PACF plots for the first difference of logged Google stock price.

Figure 5: ACF and PACF Plots for First Difference of Logged Google Stock Price

```
# ACF and PACF Plots for First Difference of Logged Google Stock Price
fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(16, 5))
sm.graphics.tsa.plot_acf(
    np.log(goog).diff().dropna(),
    title="First Difference of Logged Google stock ACF",
    lags=30,
    ax=ax1,
)
sm.graphics.tsa.plot_pacf(
    np.log(goog).diff().dropna(),
    title="First Difference of Logged Google stock PACF",
    lags=30,
    ax=ax2,
)
plt.show()
```



From figure 5, it seems that we can try $p = 1$ and/or $q = 1$ or any other combinations. Fortunately, the statistical software, can automatically test several models and report back the best model with the lowest AIC. The following report shows various ARIMA models and their corresponding AICs.

Figure 6: ARIMA Model Selection Report

```
import itertools
import numpy as np
import statsmodels.api as sm

def best_arima_model(y,
                     p_range=(0, 3),
                     d_range=(0, 3),
                     q_range=(0, 3),
                     with_intercept=False):

    best_aic = np.inf
    best_order = None
    best_model = None

    # grid search over p, d, q
    for p, d, q in itertools.product(
        range(p_range[0], p_range[1] + 1),
        range(d_range[0], d_range[1] + 1),
        range(q_range[0], q_range[1] + 1)
    ):
        try:
            model = sm.tsa.SARIMAX(
                y,
                order=(p, d, q),
                trend="n" if not with_intercept else "c",
                enforce_stationarity=False,
                enforce_invertibility=False
            ).fit(disp=False)

            if model.aic < best_aic:
                best_aic = model.aic
                best_order = (p, d, q)
                best_model = model

        except Exception:
            continue

    print(f"\nBest ARIMA order: {best_order}, AIC: {best_aic}\n")
    return best_model, best_order
```

```
# log transform + drop NA
y = np.log(goog).dropna()

best_model, best_order = best_arima_model(
    y,
    p_range=(0, 3),
    d_range=(0, 3),
    q_range=(0, 3),
    with_intercept=False
)
```

```
/usr/local/lib/python3.12/dist-packages/statsmodels/tsa/base/tsa_model.py:47
  self._init_dates(dates, freq)
/usr/local/lib/python3.12/dist-packages/statsmodels/tsa/base/tsa_model.py:47
  self._init_dates(dates, freq)

Best ARIMA order: (2, 2, 3), AIC: -8118.534432366581

/usr/local/lib/python3.12/dist-packages/statsmodels/base/model.py:607: Conve
  warnings.warn("Maximum Likelihood optimization failed to "
```

```
# Efficient ARIMA model Selection
mod_can_auto = auto_arima(
    np.log(goog).dropna(),  # stepwise=False,
    start_p=0,
    start_d=0,
    start_q=0,
    max_p=3,
    max_d=3,
    max_q=3,
    trace=True,
    with_intercept=False,
    return_valid_fits=True,
)
```

```
Performing stepwise search to minimize aic
 ARIMA(0,1,0)(0,0,0)[0]             : AIC=-8091.969, Time=0.22 sec
 ARIMA(1,1,0)(0,0,0)[0]             : AIC=-8109.119, Time=0.11 sec
 ARIMA(0,1,1)(0,0,0)[0]             : AIC=-8108.303, Time=0.11 sec
 ARIMA(2,1,0)(0,0,0)[0]             : AIC=-8107.506, Time=0.41 sec
 ARIMA(1,1,1)(0,0,0)[0]             : AIC=-8107.472, Time=0.15 sec
 ARIMA(2,1,1)(0,0,0)[0]             : AIC=-8105.502, Time=0.17 sec
 ARIMA(1,1,0)(0,0,0)[0] intercept   : AIC=-8112.960, Time=0.13 sec
 ARIMA(0,1,0)(0,0,0)[0] intercept   : AIC=-8094.611, Time=0.13 sec
 ARIMA(2,1,0)(0,0,0)[0] intercept   : AIC=-8111.185, Time=1.45 sec
 ARIMA(1,1,1)(0,0,0)[0] intercept   : AIC=-8111.181, Time=0.30 sec
 ARIMA(0,1,1)(0,0,0)[0] intercept   : AIC=-8112.242, Time=0.50 sec
 ARIMA(2,1,1)(0,0,0)[0] intercept   : AIC=-8109.167, Time=3.19 sec

Best model:  ARIMA(1,1,0)(0,0,0)[0] intercept
Total fit time: 6.904 seconds
```

In figure 6, we can see the software ran a couple of ARIMA models and reported their AICs.

On the left of the table is the ARIMA model specifications, and on the right is the model AICs. The best model from the software is ARIMA(1,1,0) with intercept. We can see that ARIMA(1,1,0) with intercept has the lowest AIC(-8112.960) among other model candidates. Figure 7 shows the MLE estimation for the ARIMA(1,1,0) model.

Figure 7: ARIMA(1,1,0) Model Estimation

```
# Best ARIMA Model for Google stock price
mod_can_a = SARIMAX(
```

```
    np.log(goog), order=(1, 1, 0), trend="c").fit()  # This is the best mode
print(mod_can_a.summary())
```

```
                               SARIMAX Results
==============================================================================
Dep. Variable:                  GOOGLE   No. Observations:                  150
Model:               SARIMAX(1, 1, 0)   Log Likelihood                4059.48
Date:                Sat, 06 Dec 2025   AIC                          -8112.96
Time:                        19:50:52   BIC                          -8097.01
Sample:                             0   HQIC                         -8107.02
                                - 1505
Covariance Type:                  opg
==============================================================================
                 coef    std err          z      P>|z|      [0.025      0.975
------------------------------------------------------------------------------
intercept      0.0010      0.000      2.394      0.017       0.000       0.00
ar.L1         -0.1160      0.015     -7.500      0.000      -0.146      -0.08
sigma2         0.0003   5.19e-06     51.035      0.000       0.000       0.00
==============================================================================
Ljung-Box (L1) (Q):                   0.00   Jarque-Bera (JB):                 2
Prob(Q):                              0.96   Prob(JB):
Heteroskedasticity (H):               2.98   Skew:
Prob(H) (two-sided):                  0.00   Kurtosis:
==============================================================================

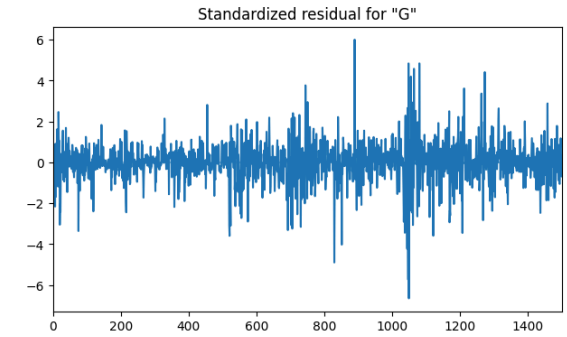
Warnings:
[1] Covariance matrix calculated using the outer product of gradients (comple
/usr/local/lib/python3.12/dist-packages/statsmodels/tsa/base/tsa_model.py:473
  self._init_dates(dates, freq)
/usr/local/lib/python3.12/dist-packages/statsmodels/tsa/base/tsa_model.py:473
  self._init_dates(dates, freq)
```

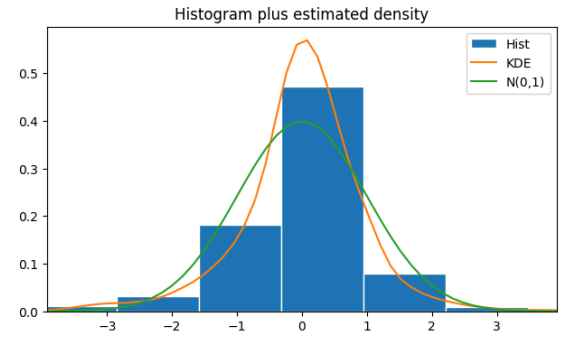
5.2 Model Diagnostics

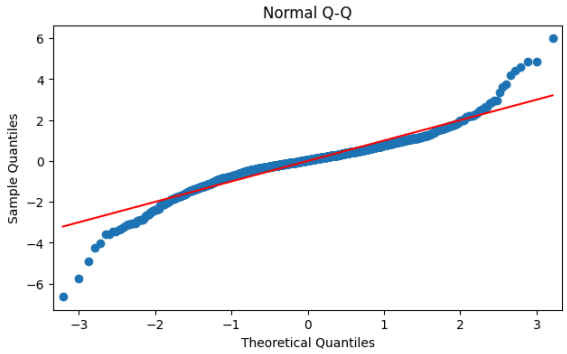
Now we have identified the ARIMA model to model Google stock price and have built and estimated the model. Let's check our model diagnostics to see if there are some potential structural patterns in Google stock price time series that our ARIMA model does not capture.

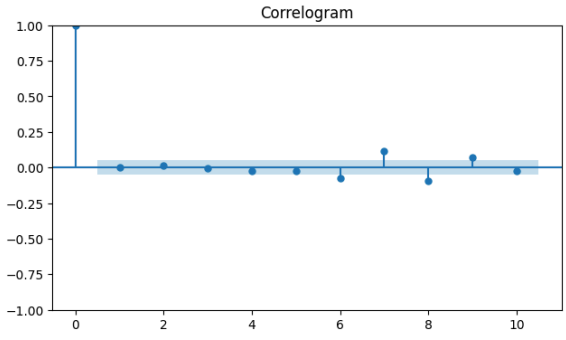
Figure 8: Diagnostic Report for ARIMA(1,1,0) Model

```
# Diagnostic Report for ARIMA(1,1,0) Model
mod_can_a.plot_diagnostics()
plt.show()
```


Standardized residual for "G"

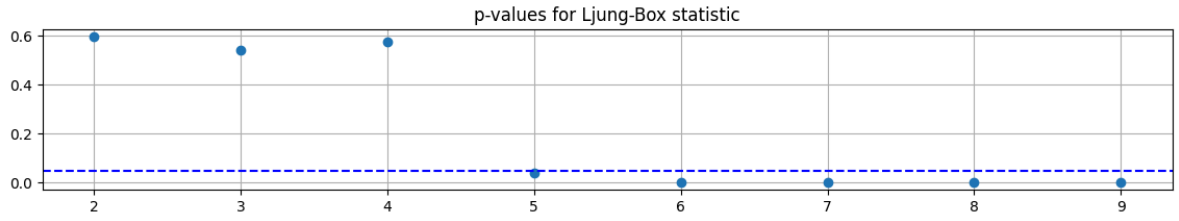

Histogram plus estimated density
Hist
KDE
N(0,1)


Normal Q-Q
Sample Quantiles
Theoretical Quantiles


Correlogram

```
# Ljung-Box test for no serial correlation of standardized residuals
lb_test = mod_can_a.test_serial_correlation(
    method="ljungbox", df_adjust=True, lags=None
)

# plot Ljung-Box test p-values and 0.05 significance line
plt.figure(figsize=(14, 2))
plt.plot(lb_test[0][1], linestyle="", marker="o")
plt.axhline(y=0.05, color="blue", linestyle="--")
plt.title("p-values for Ljung-Box statistic")
plt.grid()
plt.show()
```



In figure 8, we produce a number of graphs for model diagnostics. The first one is the standardized residuals graph. We can see from the residuals graph that there is a period where the variation is higher than other periods. In the next module, we will introduce a method to model non-constant variance for time series data. In the ACF plot for the residuals, the ACFs below lag 5 are not significant. However, lag 6 to lag 9 are still significant. There might be seasonality activity that the model does not capture. From the QQ plot for the residuals, we can see the residuals are not normally distributed since the plot shows fat tails on both ends. The last graph is the Ljung-Box test. What is the Ljung-Box test?

The Ljung-Box test is to test if a group of autocorrelations from model residuals are small enough to claim the residuals are independent. The test conducts n hypothesis tests for the following n autocorrelation groups:

> Group 1: autocorrelation 1
> Group 2: autocorrelation 1, autocorrelation 2
> Group 3: autocorrelation 1, autocorrelation 2, autocorrelation 3
> ⋮
> Group n: autocorrelation 1, autocorrelation 2, autocorrelation 3, ..., autocorrelation n

Each hypothesis test is constructed as follows:

- Null hypothesis H_0: all autocorrelations in the group are small enough so that residuals are independent
- Alternative hypothesis H_1: all autocorrelations in the group are not small enough so that the residuals are not independent

If the p-value from each hypothesis test is > 0.05, we cannot reject the null hypothesis. We then conclude that residuals from this model are independent.

Now that we know what the Ljung-Box test is, we can look at the test graph report in figure 8. From the plot, we see that the groups with lags equal to and less than 5 are all not significant. However, from lag 6 and afterwards, there are some autocorrelations that exist from Ljung-Box Test. Combining this piece of information with ACF plot for residuals, we see there might be a seasonality component that the model neglects to capture.

5.3 Forecast of ARIMA(0,1,1)

Next, once we are satisfied with our model result, we will start to forecast where Google's stock price will be. In figure 9, we forecast Google stock prices for the next 100 days with 80% confidence interval and 95% confidence interval.

Figure 9: Forecast Plot of ARIMA(1,1,0) with 80% Confidence Interval and 95% Confidence Interval

```
# Forecast Plot of ARIMA(1,1,0) with 95% Confidence Interval

# Plot Google data
ffx = goog.copy()
ffx.index = [i for i in range(len(ffx))]  # Set numeric index
ffx.plot(ylabel="Google Stock Price", title="Forecast from ARIMA(1,1,0)", c

# get forecast data for next 100 steps
forecast = mod_can_a.get_forecast(steps=100)
forecast_mean = np.exp(forecast.predicted_mean)  # mean of forecast data
conf_int95 = forecast.conf_int(alpha=0.05)  # 95% confidence interval
conf_int80 = forecast.conf_int(alpha=0.2)  # 80% confidence interval

# plot mean forecast and 95% and 80% confidence intervals
plt.plot(forecast_mean, c="b")
plt.fill_between(
    conf_int95.index,
    np.exp(conf_int95["lower GOOGLE"]),
    np.exp(conf_int95["upper GOOGLE"]),
    color="b",
    alpha=0.3,
)
plt.fill_between(
```

```
    conf_int80.index,
    np.exp(conf_int80["lower GOOGLE"]),
    np.exp(conf_int80["upper GOOGLE"]),
    color="b",
    alpha=0.5,
)
plt.grid()
plt.show()
```

```
/usr/local/lib/python3.12/dist-packages/statsmodels/tsa/base/tsa_model.py:837
  return get_prediction_index(
/usr/local/lib/python3.12/dist-packages/statsmodels/tsa/base/tsa_model.py:837
  return get_prediction_index(
```

